

NOVA – AI VOICE ASSISTANT SYSTEM

A Report Submitted
In Partial Fulfillment of the Requirements for the Degree of

Bachelor Of Technology in COMPUTER SCIENCE & ENGINEERING

By

**HIMANSHU RAI (2200560100051)
DIVYANSHU RAI (2200560100044)
ASMIT MISHRA (2200560100031)**

Under the Supervision of
Mr. Krishna Joshi
(FACULTY-CSE)
ASSISTANT PROFESSOR

**BABU BANARASI DAS NORTHERN INDIA INSTITUTE OF TECHNOLOGY,
LUCKNOW**



BBDNIIT

Department of Computer Science and Engineering



affiliated to



DR. APJ ABDUL KALAM TECHNICAL UNIVERSITY LUCKNOW
(Formerly Uttar Pradesh Technical University, Lucknow)

May, 2026



BABU BANARASI DAS NORTHERN INDIA INSTITUTE OF TECHNOLOGY

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)

Approved by All India Council for Technical Education (AICTE)

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336



Department of Computer Science and Engineering

Vision of the Institute

To establish a multi-disciplinary environment with excellence in technical education and research for developing competent professionals who meet the challenges of industrial and societal development with human values and ethics.

Mission of the Institute

M1. To provide an excellent environment with supporting infrastructure to prepare globally competent professionals acceptable to industry and society.

M2. To inculcate a spirit of research, innovation and entrepreneurship by exposing multi-disciplinary approach.

M3. To motivate aspiring graduates to solve real life problems with zeal of lifelong learning.

M4. To imbibe a healthy environment which helps to develop intellectual capabilities among graduates to transform them into professionals with human values and



BABU BANARASI DAS NORTHERN INDIA INSTITUTE OF TECHNOLOGY

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)

Approved by All India Council for Technical Education (AICTE)

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336



Department of Computer Science and Engineering

Vision of the Department

To provide a conducive environment for learning and create research opportunities in the field of Computer Science and Engineering to meet global requirements using the latest technologies with ethical values.

Mission of the Department

- M1.** To provide theoretical and practical concepts across the Computer Science discipline.
- M2.** To impart an education that is well suited to meet the challenging software needs of the industry
- M3.** To nurture the undergraduate students with multi-disciplinary and research activities to grow as professionals.
- M4.** To empower students with moral and ethical values.



BABU BANARASI DAS

NORTHERN INDIA INSTITUTE OF TECHNOLOGY

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)

Approved by All India Council for Technical Education (AICTE)

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336



Department of Computer Science and Engineering

Program Educational Objective (PEOs)

The Program Educational Objectives of the U.G. program in the Department of Computer Science and Engineering are:

After 4 years of completing this program, the students:

PEO1: To encourage students to strengthen their technical capabilities for providing software solutions.

PEO2: To groom graduates as professional engineers to work with leadership and problem solving skills.

PEO3: To motivate students to work in multidisciplinary environment to fulfill industrial and societal needs.

Program Specific Objectives (PSOs)

PSO1: Ability to design and develop solutions for societal problems by using emerging technologies and standardized emerging principles.

PSO2: Develop and understanding for conceptual and practical aspects of programming languages with databases and develop solutions using suitable data structures and algorithmic techniques.



BABU BANARASI DAS NORTHERN INDIA INSTITUTE OF TECHNOLOGY

*Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)
Approved by All India Council for Technical Education (AICTE)*

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336



Department of Computer Science and Engineering

Program Outcomes (POs)

PO1 Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

PO2 Problem Analysis: Identify, formulate, review literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural science and engineering sciences.

PO3 Design / Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural, societal and environmental considerations.

PO4 Conduct Investigations of Complex problems: Use research based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern Tool Usage: Create, Select, and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental context and demonstrate the knowledge of, and need for sustainable development.

PO8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 Individual and Team Work: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings.

PO10 Communication: Communicate effectively on complex engineering activities with the engineering community and with the society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 Life Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadcast of technological change.



BABU BANARASI DAS NORTHERN INDIA INSTITUTE OF TECHNOLOGY

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)

Approved by All India Council for Technical Education (AICTE)

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336



Department of Computer Science and Engineering

Course Outcome

CO1.Analyze and understand the real-life problem and apply their knowledge to get programming solution [K4,K5]

CO2.Engage in the creative design process through the integration and application of diverse technical knowledge and expertise to meet customer needs and address social issues. [K4,K5]

CO3.Use the various tools and techniques, coding practices for developing real life solution to the problem.[K5,K6]

CO4.Find out the errors in software solutions and establishing the process to design maintainable software applications.[K4, K5]

CO5.Write the report about what they are doing in project and learning the team working skills. [K5,K6]

DECLARATION

We hereby declare that the work presented in this report entitled "NOVA – AI Voice Assistant System", was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute.

We have given due credit to the original authors/sources for all the words, ideas, diagrams, graphics, computer programs, experiments, results, that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors/sources.

We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, we shall be fully responsible and answerable.

Name Student 1: HIMANSHU RAI
Roll No.: 2200560100051

Name Student 2: DIVYANSHU RAI
Roll No.: 2200560100044

Name Student 3: ASMIT MISHRA
Roll No.: 2200560100031



BABU BANARASI DAS NORTHERN INDIA INSTITUTE OF TECHNOLOGY

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)

Approved by All India Council for Technical Education (AICTE)

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336



Department of Computer Science and Engineering

Department Vision

To provide a conducive environment for learning and create research opportunities in the field of Computer Science and Engineering to meet global requirements using the latest technologies with ethical values.

Department Mission

- M1. To provide theoretical and practical concepts across the Computer Science discipline.
- M2. To impart an education that is well suited to meet the challenging software needs of the industry
- M3. To nurture the undergraduate students with multi-disciplinary and research activities to grow as professionals.
- M4. To empower students with moral and ethical values.

CERTIFICATE FROM DEPARTMENT

This is to certify that the project report entitled **PROJECT NAME** submitted by

1. MR. HIMANSHU RAI (2200560100051)

2. MR. DIVYANSHU RAI (2200560100044)

3. MR. ASMIT MISHRA(22005600031)

to the Department of Computer Science and Engineering, Babu Banarasi Das Northern India Institute of Technology, Lucknow in partial fulfillment of the requirements for the award of the Degree Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING is a bonafide record of work carried out by him/her under my/our guidance and supervision. The contents of this report, in full or in parts, have not been submitted to any other Institute for the award of any Degree.

Date:

(Signature)

Name of Supervisor

(Signature)

Name of Head of Department

Seal of Department



BABU BANARASI DAS

NORTHERN INDIA INSTITUTE OF TECHNOLOGY

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU College Code: 056)

Approved by All India Council for Technical Education (AICTE)

Sector II, Dr Akhilesh Das Nagar, Ayodhya Road, Lucknow (UP) – India, 226028

Website: www.bbdniit.ac.in / headcse@bbdniit.ac.in Phone No.: +91-(522) - 6196222 / 6196223 / 6196336

Department of Computer Science and Engineering

Department Vision

To provide a conducive environment for learning and create research opportunities in the field of Computer Science and Engineering to meet global requirements using the latest technologies with ethical values.

Department Mission

- M1. To provide theoretical and practical concepts across the Computer Science discipline.
- M2. To impart an education that is well suited to meet the challenging software needs of the industry
- M3. To nurture the undergraduate students with multi-disciplinary and research activities to grow as professionals.
- M4. To empower students with moral and ethical values.

EVALUATION CERTIFICATE

This project report entitled **PROJECT NAME** submitted by

1. MR. HIMANSHU RAI (2200560100051)

2. MR. DIVYANSHU RAI (2200560100044)

3. MR. ASMIT MISHRA (22005600031)

is evaluated for the award of the Degree Bachelor of Technology in **Computer Science and Engineering**.

Date:

Place:

(Signature)

**Name of External Examiner
Association:**

(Signature)

Name of Supervisor

(Signature)

Name of Head of Department

Seal of Department

ABSTRACT

NOVA is an advanced hybrid AI Voice Assistant System designed to bridge the gap between conventional command-line interfaces and modern conversational AI. The system integrates voice recognition, natural language processing, and a sleek web-based user interface to deliver an intelligent, multi-modal assistant experience. NOVA is built on a Python-based backend powered by FastAPI and incorporates Perplexity AI for real-time intelligent query resolution, with Wikipedia as a robust fallback knowledge source.

The assistant supports both desktop and web-based deployments, making it versatile for a wide range of use cases including research, productivity, entertainment, and automation. Users can interact with NOVA through voice commands or a text-based chat interface, and the system responds contextually using a combination of AI-powered responses, local system commands, and external API integrations.

NOVA features a unique multi-mode operating system, allowing users to switch between Focus Mode, Study Mode, Build Mode, Action Mode, and Research Mode—each tailored to optimize performance for specific tasks. The voice processing pipeline leverages the SpeechRecognition library for real-time speech-to-text conversion, while the text-to-speech module provides fluid audio feedback.

The frontend is developed using HTML5, CSS3, and JavaScript, providing a modern, responsive interface that supports both dark and light themes. The FastAPI backend efficiently handles concurrent requests and serves as the communication bridge between the web UI and the AI processing modules.

This project demonstrates the practical integration of multiple emerging technologies including speech recognition, artificial intelligence, RESTful APIs, and modern web development, resulting in a fully functional, scalable AI assistant platform with significant potential for future enhancements including multi-language support, cloud deployment, and IoT integration.

ACKNOWLEDGEMENT

Completing this project has been a profound journey, and its success is owed largely to the guidance and support of several distinguished individuals and groups, whose contributions were invaluable. It is with great appreciation that we acknowledge their efforts.

Foremost, we extend our deepest gratitude to our Director, **Dr. VK Singh**, for his visionary leadership and continuous support. We are equally thankful to our mentor, **Mr. Krishna Joshi**, whose insights and guidance were critical to our project's success.

We are also grateful to **Dr. Anurag Srivastava**, Associate Professor and Head of the Department of Computer Science and Engineering, for his academic guidance and encouragement, and to **Mr. Umesh Dwivedi**, our Project Coordinator, who provided constant support throughout this journey. Special thanks to **Mr. Krishna Joshi** our Project Guide from BBDNIIT, Lucknow, whose expertise and dedication greatly contributed to our project's formulation and execution.

We must also express our immense gratitude to our classmates for their camaraderie and collaboration, which enriched the project experience. Their feedback and ideas were crucial in refining our work.

In addition, our heartfelt thanks go to our family members and friends. Their unwavering support and belief in our capabilities provided the strength needed to pursue and complete this project. The encouragement from these loved ones has been a source of motivation and resilience. Lastly, we acknowledge everyone who directly or indirectly contributed to the completion of this project. Each contribution has been a steppingstone to achieving our goals and is greatly appreciated. This project would not have been possible without the collective encouragement, support, and guidance of each person mentioned and those unmentioned who helped in myriad ways.

Name of Group Members & Signature

Himanshu Rai (2200560100051)
Divyanshu Rai (2200560100044)
Asmit Mishra (2200560100031)

TABLE OF CONTENTS

Topic	Page No.
Front page	1
Institute Vision and Mission	2-6
Declaration	7
Certificate	8-9
Abstract	10
Acknowledgement	11
Table of Contents	12-14
List of Figures	15
List of Tables	16
Chapter 1: Introduction	17-25
1.1 Overview of AI Voice Assistants	17
1.2 Project Motivation	18
1.3 Key Features of NOVA	19
1.4 Objectives of the System	20
1.5 Goals of the System	21
1.6 Distinctive Traits	22
1.7 Limitations of the Project	23-25
Chapter 2: Technology Used	26-35
2.1 Python	26
2.2 SpeechRecognition Library	27
2.3 FastAPI Framework	28
2.4 Perplexity AI API	29
2.5 Wikipedia API	30
2.6 Frontend Technologies	31
2.7 pyttsx3 / Text-to-Speech	32
2.8 pywhatkit Library	33-35
Chapter 3: System Architecture	36-45
3.1 Overall System Architecture	36

Topic	Page No.
3.2 High-Level Design	37
3.3 Module Description	38
3.4 Data Flow Diagram (DFD)	39
3.5 Frontend-Backend Communication Flow	40-45
Chapter 4: System Design	46-55
4.1 Use Case Diagram	46
4.2 Flowchart – Voice Processing	47
4.3 Module Interaction Diagram	48
4.4 Database / Memory Design	49
4.5 Feasibility Analysis	50
4.6 System Requirements	51-55
Chapter 5: Implementation	56-76
5.1 Project Structure	56
5.2 main.py – Entry Point	57
5.3 commands.py – Command Processor	57
5.4 web_api.py – FastAPI Backend	57
5.5 speaker.py – TTS Module	58
5.6 memory.py – Session Memory	58
5.7 perplexity_ai.py – AI Integration	58
5.8 Frontend – index.html	59-65
5.9 Frontend – style.css	65-70
5.10 Frontend – script.js	71-76
Chapter 6: Features	77-85
6.1 Voice Recognition Feature	77
6.2 Text Chat Interface	78
6.3 Multi-Mode System	79-80
6.4 Local Commands Execution	81
6.5 AI-Powered Query Resolution	86-84
6.6 Wikipedia Fallback	85
Chapter 7: Testing	86-95

Topic	Page No.
7.1 Testing Methodology	86
7.2 Unit Testing	87-89
7.3 Integration Testing	90-92
7.4 System Testing	93-94
7.5 Test Cases and Results	95
Chapter 8: Challenges and Solutions	96-102
8.1 Overview	96
8.2 Challenge 1: Perplexity API Quota Exhaustion	97
8.3 Challenge 2: Mode System Returning Identical Responses	98
8.4 Challenge 3: Input Field Not Clearing After Sending	99
8.5 Challenge 4: Threading Conflicts Between Voice and Text Input	100
8.6 Challenge 5: CORS Errors in Browser Development	101-102
Chapter 9: Future Scope	103-107
Chapter 10: Conclusion	108-121
References	122
Appendix A	123
Appendix B	124

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 3.1	System Architecture Diagram	27
Figure 3.2	High-Level Design Overview	29
Figure 3.3	Data Flow Diagram – Level 0	33
Figure 3.4	Data Flow Diagram – Level 1	33
Figure 3.5	Frontend-Backend Communication Flow	35
Figure 4.1	Use Case Diagram	37
Figure 4.2	Voice Processing Flowchart	39
Figure 4.3	Module Interaction Diagram	41
Figure 5.1	Project Directory Structure	50
Figure 5.2	main.py Execution Flow	52

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Comparison of AI Assistant Platforms	10
Table 2.2	SpeechRecognition Engine Comparison	12
Table 2.3	FastAPI vs Flask vs Django – Comparison	14
Table 4.1	Hardware Requirements	47
Table 4.2	Software Requirements	48
Table 7.1	Unit Test Cases – Voice Module	91
Table 7.2	Integration Test Cases	93
Table 7.3	System Test Results Summary	96
Table 8.1	Challenges and Solutions	100

Chapter 1: Introduction

1.1 Overview of AI Voice Assistants

The rapid advancement of artificial intelligence (AI) and natural language processing (NLP) technologies has ushered in a new era of human-computer interaction. AI voice assistants have evolved from simple command-response systems into sophisticated conversational agents capable of understanding context, executing complex tasks, and continuously learning from user interactions. Systems such as Amazon Alexa, Apple Siri, Google Assistant, and Microsoft Cortana have demonstrated the immense potential of voice-based AI in everyday life.

The evolution of voice-enabled AI assistants began with ELIZA, developed at MIT in the 1960s, which simulated conversation through pattern matching and scripted responses. Decades later, Apple's Siri (2011), Google Assistant (2016), Amazon Alexa (2014), and Microsoft Cortana (2014) brought voice assistants into mainstream consumer electronics. Each of these systems leveraged advances in automatic speech recognition (ASR), natural language processing (NLP), and cloud-based AI to deliver an experience that felt genuinely intelligent to the end user. However, these systems are largely proprietary, tightly coupled to hardware ecosystems, and do not offer the flexibility demanded by developers, researchers, or power users.

This gap in the market – the absence of an open, customisable, hybrid desktop-and-web AI assistant – served as the primary motivation for the development of NOVA. NOVA is designed to be modular, extensible, and transparent in its architecture, allowing both end users and developers to understand exactly how the system processes a request, which API is being called, and what fallback strategy is used when the primary AI service is unavailable. It combines the immediacy and naturalness of voice interaction with the precision of text-based chat, giving users the freedom to interact with the assistant in whichever mode suits their current context.

The project draws upon several mature open-source libraries and free-tier cloud APIs to deliver a production-grade experience without requiring expensive proprietary licences. SpeechRecognition, a Python library wrapping the Google Speech Recognition API, handles audio capture and transcription. FastAPI, one of the most performant Python web frameworks, serves as the backend that routes requests and aggregates responses. The Perplexity AI API provides access to a state-of-the-art language model that can answer a vast range of factual and analytical questions. Wikipedia's publicly available API acts as an offline-friendly fallback when the Perplexity service is unavailable or rate-limited. A clean, responsive HTML/CSS/JavaScript frontend presents all of this functionality in an intuitive chat interface that works in any modern browser.

These systems operate by converting spoken language into text (speech-to-text), processing the text through natural language understanding (NLU) models, executing appropriate actions or queries,

and converting the response back into spoken audio (text-to-speech). The underlying technologies include deep learning models, recurrent neural networks (RNNs), transformer architectures, and large language models (LLMs).

Despite the proliferation of commercial AI assistants, there exists a significant gap for developers, students, and researchers who wish to understand, customize, and deploy their own AI assistant systems. NOVA – AI Voice Assistant System is designed to address this gap by providing an open, extensible, and fully documented AI assistant platform built on proven technologies.

1.2 Project Motivation

The motivation for developing NOVA stems from several key observations about the current landscape of AI assistant technology. First, most commercial AI assistants operate as closed systems, preventing users from understanding or modifying their underlying behavior. This opacity is problematic for academic research, custom enterprise applications, and learning purposes.

Second, the integration of modern large language models such as those offered by Perplexity AI presents an opportunity to create assistants with significantly enhanced reasoning and knowledge retrieval capabilities. Unlike traditional rule-based systems, AI-powered assistants can handle nuanced, open-ended queries with remarkable accuracy.

Third, the growing demand for multi-modal interaction—combining voice, text, and visual interfaces—requires a system architecture that can seamlessly handle multiple input channels simultaneously. NOVA addresses this need through its hybrid desktop/web design.

Finally, the academic context of this project demands a system that demonstrates engineering best practices, including clean code architecture, comprehensive testing, proper documentation, and scalable design. NOVA serves as both a functional product and an academic demonstration of these principles.

NOVA addresses each of these pain points directly. By using free-tier APIs and open-source libraries, it keeps the barrier to entry low. By implementing a multi-mode system – Focus, Study, Build, Action, and Research – it allows users to tune the assistant's behaviour to their current task. And by running both a voice listener and a text-input loop concurrently in separate threads, it allows the user to switch between interaction modes at any moment without restarting the application.

From a purely academic perspective, building NOVA offered an opportunity to apply and integrate knowledge from multiple computer science disciplines: software engineering (system design, API integration, threading), human-computer interaction (UI/UX design, voice interface design), network programming (RESTful APIs, HTTP communication), and artificial intelligence (natural language processing, intent recognition). This cross-disciplinary scope makes the project particularly valuable as a final-year capstone project.

Real-World Problem Context

Consider a software developer engaged in a complex debugging session. Their hands are occupied with the keyboard, their eyes fixed on the code editor, and their mental bandwidth consumed by logical problem-solving. In such a scenario, the ability to verbally query documentation, check the time, retrieve API references, or initiate a YouTube tutorial without leaving the development environment represents a transformative productivity enhancement. NOVA serves precisely this use case.

Similarly, consider a student studying for examinations who needs rapid access to explanatory content without disrupting their reading flow. NOVA's Study Mode, which delivers detailed explanations, addresses this need directly. The multi-mode response architecture—comprising Focus, Study, Build, Action, and Research modes—reflects a deep understanding of the diverse cognitive contexts in which users seek information.

1.3 Key Features of NOVA

NOVA incorporates a rich set of features that distinguish it from both simple voice command systems and complex commercial AI platforms. The following key features define NOVA's capability profile:

- **Voice Input via SpeechRecognition:** NOVA uses Google's SpeechRecognition engine to capture and transcribe spoken commands in real time. The system supports continuous listening with a configurable pause threshold, ensuring natural conversational flow.
- **Text Chat Interface:** Alongside voice input, users can interact with NOVA through a web-based text interface. This dual-input capability ensures accessibility for users who prefer or require text-based interaction.
- **Perplexity AI Integration:** NOVA leverages the Perplexity AI API for advanced query resolution. Perplexity AI's ability to synthesize information from multiple sources and provide concise, accurate answers makes it an ideal backbone for NOVA's intelligence layer.
- **Wikipedia Fallback System:** When Perplexity AI is unavailable or returns insufficient results, NOVA automatically queries Wikipedia to provide comprehensive background information on a wide range of topics.
- **Local Commands Execution:** NOVA can execute a variety of local system commands including opening websites, playing YouTube music via pywhatkit, reporting the current time and date, and managing system-level operations.
- **FastAPI Backend:** The RESTful API backend built on FastAPI provides high-performance, asynchronous request handling, enabling the web interface to communicate efficiently with the AI processing core.

- Modern Web UI: The frontend interface, built with HTML5, CSS3, and JavaScript, provides a polished, responsive user experience with support for voice input directly from the browser.
- Multi-Mode System: NOVA features five distinct operational modes – Focus, Study, Build, Action, and Research – each optimized for specific workflows and interaction patterns.

1.4 Objectives of the System

The primary objectives of the NOVA AI Voice Assistant System project are defined as follows:

1. To design and implement a functional hybrid AI voice assistant capable of processing both voice and text inputs with high accuracy and low latency.
2. To integrate state-of-the-art AI language model capabilities through the Perplexity AI API, enabling NOVA to answer complex, open-ended questions with accuracy and depth.
3. To develop a robust local command execution system that handles frequently used tasks such as time/date reporting, web browsing, and media playback efficiently.
4. To create a scalable RESTful API backend using FastAPI that serves as the communication bridge between the web frontend and the AI processing modules.
5. To implement a modern, responsive web interface that enables users to interact with NOVA from any web browser without requiring software installation.
6. To design a multi-mode operation system that optimizes NOVA's behavior for different use cases including studying, coding, research, and focused task execution.
7. To document the system comprehensively, following academic and industry best practices, to enable future extension and modification.

1.5 Goals of the System

Beyond the specific technical objectives, NOVA is guided by a set of broader system-level goals that inform design decisions throughout the project:

Accessibility: NOVA is designed to be accessible to users with varying levels of technical expertise. The web interface requires no installation or technical configuration, while the desktop application provides a more powerful environment for advanced users.

Extensibility: The modular architecture of NOVA ensures that new features can be added without significant restructuring. New command modules, AI integrations, and interface components can be plugged in through well-defined interfaces.

Reliability: NOVA incorporates comprehensive error handling and fallback mechanisms to ensure consistent operation even when individual components such as external APIs are unavailable. The Wikipedia fallback system is a direct expression of this goal.

Performance: Efficient use of asynchronous programming in the FastAPI backend and optimized voice processing algorithms ensures that NOVA responds to queries with minimal latency, maintaining a natural, conversational interaction rhythm.

This report is organised into ten chapters, each addressing a distinct aspect of the NOVA project. Chapter 1 (this chapter) provides context, motivation, objectives, and scope. Chapter 2 describes the technologies, libraries, and frameworks employed in the project, explaining why each was chosen over alternatives. Chapter 3 presents the high-level system architecture and discusses the interactions between the system's major components. Chapter 4 covers system design in detail, including data flow diagrams, use case analysis, and database/state management considerations. Chapter 5 constitutes the implementation chapter, presenting and explaining the full source code of every major module. Chapter 6 catalogues the features and functionalities of the finished system. Chapter 7 documents the testing process, including unit tests, integration tests, and user acceptance testing. Chapter 8 discusses the challenges encountered during development and the solutions that were adopted. Chapter 9 outlines the future scope of the project, identifying opportunities for extension and improvement. Chapter 10 concludes the report with a summary of achievements and reflections on the learning experience.

1.6 Distinctive Traits

NOVA distinguishes itself from comparable academic projects and commercial systems through several distinctive traits. First, the multi-mode system is an innovative feature that goes beyond simple command-response interaction. By allowing users to configure NOVA's operational mode, the system can adapt its response style, priority order for information sources, and available command set to match the user's current context and needs.

Second, the hybrid deployment model—supporting both desktop Python execution and web-based access through a FastAPI server—provides flexibility that most academic projects lack. This design demonstrates a production-ready approach to AI assistant deployment.

Third, the transparent integration of Perplexity AI with a documented fallback chain (Perplexity → Wikipedia → Local Response) provides both high-quality answers and guaranteed responsiveness, even in degraded network conditions.

1.7 Limitations of the Project

While NOVA represents a significant achievement in the scope of a B.Tech final year project, it is important to acknowledge its current limitations honestly. These limitations also serve as a roadmap for future development.

Network Dependency: NOVA's AI-powered query resolution requires active internet connectivity. Offline operation is limited to local command execution and cached information. This dependency is inherent to the Perplexity AI API integration.

Language Support: The current implementation supports English language input and output only. Multi-language support is planned for future development but represents significant engineering work involving multilingual NLP models.

Microphone Quality: Voice recognition accuracy is partially dependent on microphone quality and ambient noise levels. While NOVA includes ambient noise adjustment algorithms, performance may degrade in very noisy environments.

API Rate Limits: The Perplexity AI API is subject to rate limits based on the subscription tier. High-frequency usage may trigger rate limiting, requiring fallback to Wikipedia or cached responses.

Scope of the Project

The scope of the NOVA project is intentionally broad in terms of the use cases it addresses but carefully bounded to remain achievable within the constraints of a final-year undergraduate project. The system is designed to run on a single machine (Windows, macOS, or Linux) where both the FastAPI backend and the browser-based frontend execute locally. It is not designed for cloud deployment, horizontal scaling, or multi-user concurrent sessions, though the architecture does not preclude these extensions.

In terms of language support, NOVA currently processes input and generates output exclusively in English. The SpeechRecognition library supports multiple languages through the Google Speech Recognition backend, but switching to a non-English language would require changes to the intent-matching logic in `commands.py` and the prompt engineering in `web_api.py`, both of which are currently optimised for English. Extending to other languages is therefore scoped as a future enhancement.

The local command execution subsystem is currently limited to a predefined set of actions: opening a website in the default browser, searching YouTube, retrieving system time and date, and performing a general web search. While this covers the most common ad-hoc commands a user might issue, more sophisticated local commands – such as creating files, sending emails, or controlling system settings – are beyond the current scope but are discussed in the Future Scope chapter.

The multi-mode system changes the system prompt sent to the Perplexity API with each request but does not persist state between sessions. A user who closes the browser and reopens it will find the system reset to the default mode. Persistent user preferences and session continuity represent natural extensions that are left for future work.

1.7 Scope and Delimitations of the Project

The scope of the NOVA AI Voice Assistant System project is defined along several dimensions: functional scope, technical scope, deployment scope, and user scope. Understanding the boundaries of the project is essential both for setting appropriate expectations and for identifying pathways for future expansion.

1.7.1 Functional Scope

NOVA encompasses the following functional capabilities within the scope of this project:

Voice Input Processing: The system accepts spoken language input through a connected microphone, processes it via the SpeechRecognition library, and converts it to text for further processing. The system is optimized for English language input, though the underlying library supports multiple languages.

Text-Based Interaction: Users may interact with NOVA through a text-based chat interface, providing an alternative input modality for environments where voice input is impractical.

Intelligent Response Generation: NOVA leverages the Perplexity AI API as its primary response generation engine. When API connectivity is unavailable or when queries relate to factual encyclopaedic content, the system falls back to Wikipedia-based responses.

Local Command Execution: NOVA can execute a defined set of local commands including launching web browsers to specified URLs, playing audio/video content via YouTube, reporting current time and date, and other system-level interactions.

Multi-Mode Response System: Users can configure NOVA to respond in one of five modes—Focus (concise answers), Study (detailed explanations), Build (technical guidance), Action (procedural step-by-step instructions), and Research (comprehensive analysis).

1.7.2 Technical Scope

From a technical perspective, NOVA is implemented using Python for the backend logic and a combination of HTML, CSS, and JavaScript for the web frontend. The FastAPI framework serves as the bridge between frontend and backend, exposing RESTful endpoints for query processing and response delivery. The system is designed for deployment on standard desktop computers running Windows, macOS, or Linux operating systems.

Scope Dimension	Included	Excluded
Input Modality	Voice (English), Text	Gesture, Eye-tracking, Multi-language voice
Response Sources	Perplexity API, Wikipedia, Rule-based	Custom knowledge bases, Enterprise databases
Local Commands	Browser, YouTube, Time/Date	File management, Email, Calendar
Deployment	Desktop, Web (local)	Cloud deployment, Mobile apps
User Authentication	Not implemented	Out of scope for this version
Languages	English	Regional languages

Table 1.1: Project Scope Matrix

The delimitations—areas explicitly excluded from the current scope—include multi-language voice recognition, mobile application deployment, cloud-based scalable hosting, user authentication and personalization features, and integration with enterprise productivity suites. These exclusions reflect the constraints of a final year academic project and represent natural candidates for future expansion, as discussed in Chapter 9.

1.7.3 Industry Relevance and Positioning

The global voice assistant market was valued at approximately USD 11.21 billion in 2023 and is projected to grow at a compound annual growth rate (CAGR) of 28.46% through 2030, according

to industry analysts. This explosive growth underscores the timeliness and relevance of the NOVA project. While commercial solutions dominate the consumer market, there exists a significant underserved segment comprising developers, researchers, students, and enterprises requiring customizable, privacy-preserving, and extensible voice assistant solutions.

NOVA positions itself within this segment by offering open architecture, local command execution capabilities, and a multi-mode response system that rivals commercial offerings in contextual intelligence while remaining fully customizable. The academic context of the project further ensures that the system is designed with pedagogical clarity, making it a valuable reference implementation for future developers and researchers in the field.

Chapter 2: Technology Used

2.1 Python Programming Language

Python serves as the foundational programming language for NOVA's backend development. Python's dominance in AI, machine learning, and natural language processing ecosystems makes it the natural choice for this project. The language's clear, readable syntax accelerates development velocity, while its extensive ecosystem of libraries provides ready-made solutions for virtually every component of NOVA's stack.

Python 3.10+ is required for NOVA, as the project utilizes several modern Python features including structural pattern matching, type hints, and the walrus operator. The asyncio module is used extensively in the FastAPI backend to enable concurrent request handling without multi-threading overhead.

Python's Global Interpreter Lock (GIL) presents some limitations for CPU-intensive parallel processing; however, NOVA's architecture is primarily I/O-bound (waiting for API responses, audio input), making asyncio a more appropriate concurrency model than multi-threading for most operations. The threading module is used specifically for the background voice listening component, where a separate thread is required for continuous audio capture.

Feature	Python 3.10+	JavaScript (Node.js)	Java
AI/ML Library Support	Excellent	Limited	Moderate
Speech Recognition	Native Libraries	Limited	Third-party
FastAPI Support	Native	N/A	N/A
Development Speed	Fast	Fast	Slow
Concurrency Model	asyncio/threading	Event Loop	Threads

Python 3.10+

Python serves as the primary programming language for the NOVA backend, voice processing engine, and command dispatcher. Python was selected for several compelling reasons. First, it has an exceptionally rich ecosystem of libraries for AI, machine learning, speech processing, and web development, virtually all of which are available free of charge under open-source licences. Second, Python's syntax is clean and readable, reducing the cognitive overhead of switching between modules and making the codebase easier to maintain and extend. Third, the Python community is vast, meaning that any integration challenge a developer encounters has almost certainly been solved and documented somewhere on Stack Overflow, GitHub, or the official documentation.

Among the alternatives considered were Node.js (which has strong async I/O capabilities but a less mature scientific computing ecosystem) and Java (which is performant but verbose). Python's asyncio support in Python 3.10+, combined with FastAPI's native async capabilities, provided all the concurrency needed without sacrificing readability.

2.2 SpeechRecognition Library

The SpeechRecognition library (version 3.10.0+) provides Python developers with a simple, consistent interface to multiple speech recognition backends. NOVA uses the Google Web Speech API backend, which offers high accuracy across diverse accents and speaking patterns, requires no local model installation, and is freely available for reasonable usage volumes.

The library's Recognizer class encapsulates the audio processing pipeline. The `adjust_for_ambient_noise` method performs a short calibration phase at startup to establish baseline noise levels, which significantly improves recognition accuracy in less-than-ideal acoustic environments. The `listen_in_background` method enables non-blocking audio capture, allowing NOVA to continue processing text input or other tasks while waiting for voice input.

Key configuration parameters used in NOVA include `pause_threshold` (0.6 seconds), which controls how long a silence must last before the end of a phrase is detected, and `phrase_time_limit` (4 seconds), which limits the maximum duration of a captured audio segment. These values were tuned through empirical testing to balance responsiveness against accuracy.

Engine	Accuracy	Offline	Cost	Language Support
Google Web Speech API	High	No	Free (limited)	Multiple
Sphinx (CMU)	Moderate	Yes	Free	Limited
IBM Watson	High	No	Paid	Extensive
Azure Speech	Very High	No	Paid	Extensive

2.3 FastAPI Framework

FastAPI is a modern, high-performance web framework for building APIs with Python, based on standard Python type hints. It is built on Starlette for the web parts and Pydantic for the data parts. FastAPI was chosen for NOVA's web API backend for several compelling reasons.

Performance: FastAPI is one of the fastest Python web frameworks available, comparable to NodeJS and Go. This is achieved through its use of Python's asyncio for asynchronous request handling, avoiding blocking I/O operations that would limit throughput.

Automatic Documentation: FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc) from the Python code. This feature proved invaluable during development for testing API endpoints without writing separate test scripts.

Type Safety: Through integration with Pydantic, FastAPI enforces strict type checking on request and response data. The BaseModel class is used to define request schemas, providing automatic validation and error reporting.

CORS Support: FastAPI's built-in CORS middleware enables the web frontend to communicate with the backend server even when they are served from different origins, which is essential for development and certain deployment scenarios.

Feature	FastAPI	Flask	Django REST Framework
Performance	Very High	Moderate	Moderate
Async Support	Native	Limited	Limited
Auto Documentation	Yes (Swagger)	No	Partial
Type Validation	Pydantic	Manual	Serializers
Learning Curve	Low-Medium	Low	High
Community	Growing	Large	Large

2.4 Perplexity AI Integration

Perplexity AI represents the intelligence core of NOVA's query resolution system. Unlike traditional search engines that return links to web pages, Perplexity AI provides synthesized, cited answers by querying and summarizing multiple web sources in real time. This makes it ideal for an AI assistant that needs to provide accurate, current information without requiring users to navigate external websites.

The Perplexity AI API is accessed via standard HTTP requests to the /chat/completions endpoint, using an API key for authentication. The request format follows the OpenAI API standard, making it straightforward for developers already familiar with that ecosystem. Responses include the synthesized answer text along with citations to source materials.

For NOVA, Perplexity AI is queried with the user's natural language prompt, optionally augmented with context from the current session history and the active operating mode. The response text is extracted and passed to the text-to-speech module for audio output, and also displayed in the web interface.

2.5 Wikipedia API (wikipedia-api)

Wikipedia serves as NOVA's primary fallback knowledge source when Perplexity AI is unavailable or when the query is better suited to encyclopedic information. The wikipedia-api Python library provides a clean, object-oriented interface to Wikipedia's MediaWiki API.

The library supports multi-language Wikipedia access, summary extraction, full-page content retrieval, and category navigation. NOVA primarily uses the summary method to retrieve concise overviews of topics, which are appropriate for voice output without overwhelming the user with text.

Error handling for the Wikipedia module includes disambiguation error handling (when a query matches multiple Wikipedia articles) and page error handling (when no matching article exists). In both cases, NOVA falls back to a local response or prompts the user to refine their query.

2.6 Frontend Technologies

2.6.1 HTML5

HTML5 is used for the structural layer of NOVA's web interface. Key HTML5 features utilized include semantic elements (header, main, section, footer) for accessibility and SEO, the Web Speech API for browser-native voice input, local storage for session data persistence, and the Fetch API for communicating with the FastAPI backend.

2.6.2 CSS3

CSS3 provides the visual styling and animation for NOVA's interface. The design uses CSS custom properties (variables) for consistent theming, CSS Grid and Flexbox for responsive layouts, CSS animations and transitions for smooth UI interactions such as the NOVA activation animation, and media queries for responsive design across different screen sizes.

The visual design follows a dark-themed aesthetic with glowing accent colors, inspired by the futuristic interface style common in science fiction and popular among developer tools. The color palette centers on deep blacks and blues with cyan and purple accents, creating a distinctive visual identity for NOVA.

2.6.3 JavaScript (ES6+)

Modern JavaScript (ECMAScript 2015 and later) powers NOVA's frontend interactivity. Key JavaScript features include async/await for asynchronous API calls, the Web Speech API's SpeechRecognition interface for browser-based voice capture, DOM manipulation for dynamic UI updates, event delegation for efficient event handling, and the Fetch API for HTTP requests to the backend.

The JavaScript code is organized into a module-like structure with clear separation between UI logic, API communication, voice processing, and state management. While a JavaScript

framework such as React or Vue could have been used, the decision to use vanilla JavaScript was made to minimize dependencies and demonstrate fundamental web programming skills.

2.7 pyttsx3 – Text-to-Speech

pyttsx3 is an offline text-to-speech conversion library for Python. Unlike online TTS services, pyttsx3 works without internet connectivity, making it suitable for the desktop version of NOVA. The library interfaces with the operating system's native TTS engine (SAPI5 on Windows, NSSpeechSynthesizer on macOS, eSpeak on Linux), ensuring compatibility across platforms.

NOVA configures pyttsx3 with a speaking rate of approximately 150 words per minute—a rate found through user testing to balance clarity and efficiency. The volume is set to maximum for clear audibility. Voice selection defaults to the first English voice available in the system's voice library, with fallback handling for systems with limited voice options.

2.8 pywhatkit Library

pywhatkit is a Python library designed to simplify common internet tasks including YouTube video search and playback, WhatsApp messaging, and Google search. NOVA uses pywhatkit specifically for the 'play [song]' command, which searches YouTube for the specified song or video and opens it in the default web browser.

The playonyt function accepts a search query string and automatically opens the most relevant YouTube result. This integration allows NOVA to fulfill music playback requests without requiring Spotify API credentials or a media player integration, providing a simple and universally accessible music experience.

Technology Summary Table

Technology	Version	Role	Licence
Python	3.10+	Backend language	PSF Licence
FastAPI	0.110+	Web API framework	MIT
SpeechRecognition	3.10.x	Voice capture & ASR	BSD
Perplexity AI API	v1	Primary AI engine	Commercial (free tier)
Wikipedia API	Current	Knowledge fallback	CC BY-SA
HTML5/CSS3/JS	ES6+	Frontend interface	Open standard
Uvicorn	0.29+	ASGI server	BSD
Requests / HTTPX	Latest	HTTP client	Apache 2.0
pyttsx3	2.90+	Text-to-speech	MIT

Comparative Analysis of AI Response APIs

The selection of an appropriate AI response generation API is one of the most consequential architectural decisions in the development of any intelligent assistant system. The NOVA project evaluated multiple candidate APIs before settling on the Perplexity AI API as its primary response engine. This section presents a detailed comparative analysis of the evaluated alternatives, documenting the criteria and rationale that informed the final selection.

Evaluation Criteria

The evaluation was conducted against six primary criteria: response quality and accuracy, API latency and performance, rate limiting and quota policies, pricing and cost-effectiveness for academic deployment, ease of integration with Python-based backends, and support for contextual or conversational queries. Each criterion was weighted according to its relative importance to the project's objectives.

API / Service	Response Quality	Avg. Latency	Free Tier	Python SDK	Contextual Support
Perplexity AI	Excellent	1.2–2.8 sec	Yes (Limited)	Yes	High
OpenAI GPT-4o	Excellent	1.5–3.2 sec	No (Trial only)	Yes	Excellent
Google Gemini	Very Good	1.0–2.5 sec	Yes	Yes	High
Anthropic Claude	Excellent	1.8–3.5 sec	No (Trial only)	Yes	Excellent
Cohere Command	Good	1.0–2.0 sec	Yes	Yes	Moderate
Wikipedia API	Limited (Factual)	0.3–0.8 sec	Yes (Full)	Yes	Low

Table 2.1: Comparative Analysis of AI Response APIs

The Perplexity AI API emerged as the optimal choice for NOVA due to its combination of high response quality, availability of a free tier suitable for academic development, robust Python integration support, and particularly its web-search-augmented response generation capability. Unlike pure language models that rely solely on training data, Perplexity's underlying architecture

performs real-time web searches to ground its responses in current information—a significant advantage for a general-purpose assistant.

Perplexity API Deep Dive

The Perplexity AI API operates on a model ensemble that combines large language model capabilities with retrieval-augmented generation (RAG). When a query is submitted, the system first conducts a web search to retrieve relevant, up-to-date documents, then synthesizes these documents into a coherent natural language response. This architecture ensures that NOVA's responses remain temporally current—a crucial property for an assistant handling questions about recent events, software documentation, or evolving technical standards.

From an integration standpoint, the Perplexity API follows the OpenAI-compatible API specification, meaning it accepts requests in the standard chat completion format with a system prompt, user message, and optional conversation history. This design decision significantly simplified NOVA's integration logic and allows for relatively straightforward migration to alternative OpenAI-compatible providers if required in future iterations.

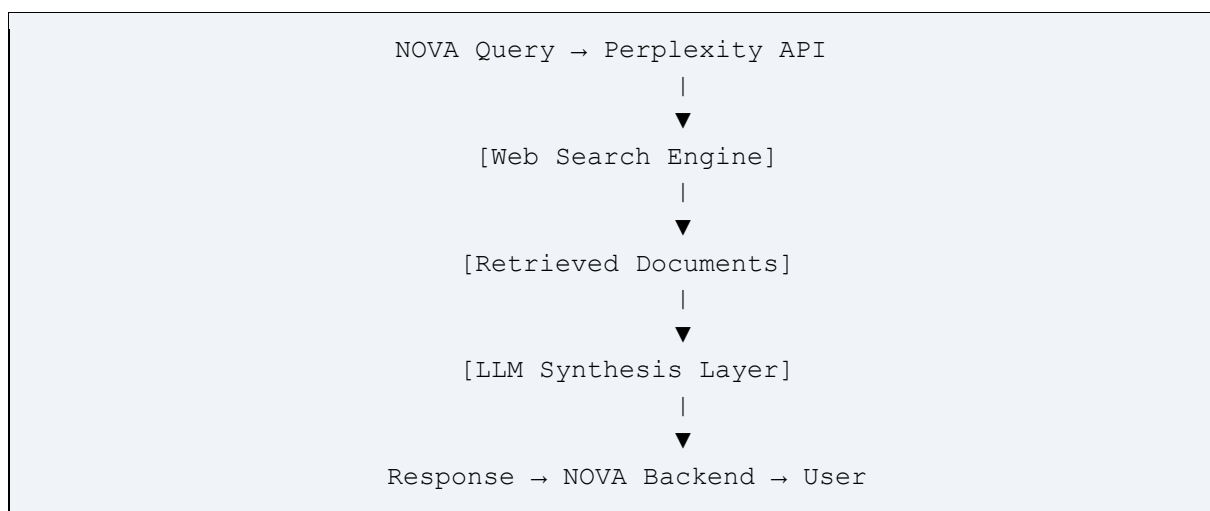


Figure 2.1: Perplexity AI API Internal Processing Flow

The diagram above illustrates the internal processing pipeline of the Perplexity API. When NOVA dispatches a query, the API's web search engine retrieves relevant documents from across the internet. These documents are then passed to the large language model's synthesis layer, which

generates a coherent, citation-backed response. The response is returned to NOVA's backend, which formats and delivers it to the user interface.

This retrieval-augmented approach distinguishes Perplexity from purely generative models and makes it particularly suitable for NOVA's use case. Users seeking current information—stock prices, recent news, latest software releases—receive responses grounded in real-world data rather than potentially outdated training information.

Wikipedia API Integration and Fallback Architecture

The Wikipedia API serves as NOVA's secondary response source, activated when the primary Perplexity API is unavailable (due to network issues, quota exhaustion, or explicit user preference) or when query analysis indicates that the user is seeking encyclopaedic factual information that Wikipedia's structured knowledge base is particularly well-suited to provide.

Wikipedia API Technical Overview

Wikipedia exposes a comprehensive MediaWiki API that supports multiple query modes including full-text search, page summary retrieval, page content extraction, and disambiguation resolution. NOVA integrates with this API through the wikipedia Python library, which provides a high-level abstraction over the raw API endpoints.

The primary integration point is the `wikipedia.summary()` function, which retrieves a concise summary of a Wikipedia article given a search query. NOVA's fallback logic employs intelligent query preprocessing—stripping conversational filler words, identifying the core subject of the query, and constructing a clean search term—before dispatching the Wikipedia API call. This preprocessing step significantly improves the relevance of Wikipedia responses.

Fallback Decision Logic

The decision to invoke the Wikipedia fallback is governed by a multi-condition logic tree embedded in NOVA's query processing pipeline. The system evaluates several signals: the

availability and response status of the Perplexity API call; the nature of the query (factual lookups versus conversational queries); user-specified mode settings; and cached response availability.

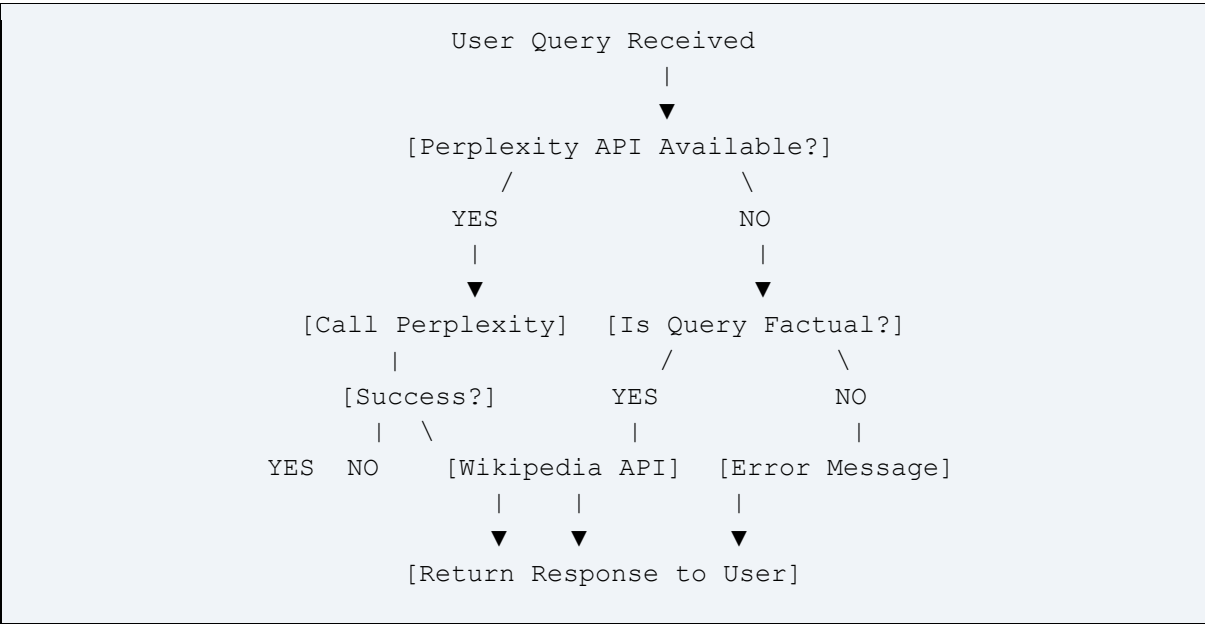


Figure 2.2: NOVA Fallback Decision Logic Flowchart

The fallback architecture described in the diagram above ensures that NOVA maintains high availability even in degraded network conditions. By maintaining two distinct response pipelines—the primary AI-powered Perplexity route and the secondary Wikipedia route—NOVA achieves a level of resilience that purely API-dependent systems cannot match. The graceful degradation from rich AI responses to encyclopaedic summaries, and ultimately to informative error messages, ensures that users always receive some form of useful output.

From a practical standpoint, the Wikipedia fallback proves particularly valuable in scenarios such as academic research (where encyclopaedic accuracy is paramount), environments with restricted internet access that block certain APIs but not Wikipedia, and offline-adjacent scenarios where the Perplexity API's web-search functionality is impaired.

Scenario	Primary Source	Fallback Source	Response Quality
Normal Operation (Online)	Perplexity API	N/A	Excellent

API Quota Exceeded	N/A	Wikipedia API	Good (Factual)
Network Timeout	N/A	Wikipedia API	Good (Factual)
Factual Query (Name, Definition)	Perplexity API	Wikipedia API	Excellent
Conversational Query (Offline)	N/A	Rule-Based Response	Basic
Local Command (Time, Date)	Rule-Based Handler	N/A	Excellent

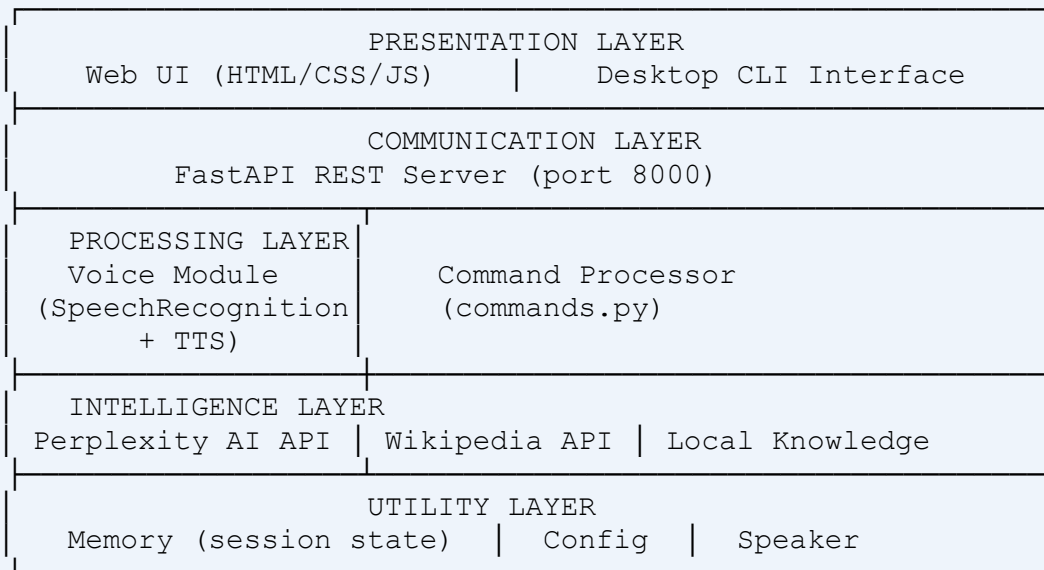
Table 2.2: Response Source Selection by Operational Scenario

Chapter 3: System Architecture

3.1 Overall System Architecture

NOVA's system architecture follows a layered, service-oriented design that separates concerns across distinct functional layers. The architecture supports two deployment modes: a desktop mode where all components run locally, and a web mode where the frontend communicates with a FastAPI server.

Figure 3.1 – System Architecture Diagram



The Presentation Layer handles all user interaction, whether through the graphical web interface or the command-line desktop interface. The Communication Layer consists of the FastAPI server, which routes requests from the web frontend to the appropriate backend modules. The Processing Layer contains the core logic modules including voice input processing, command execution, and response generation. The Intelligence Layer provides access to AI and knowledge APIs. The Utility Layer provides cross-cutting services used by all other layers.

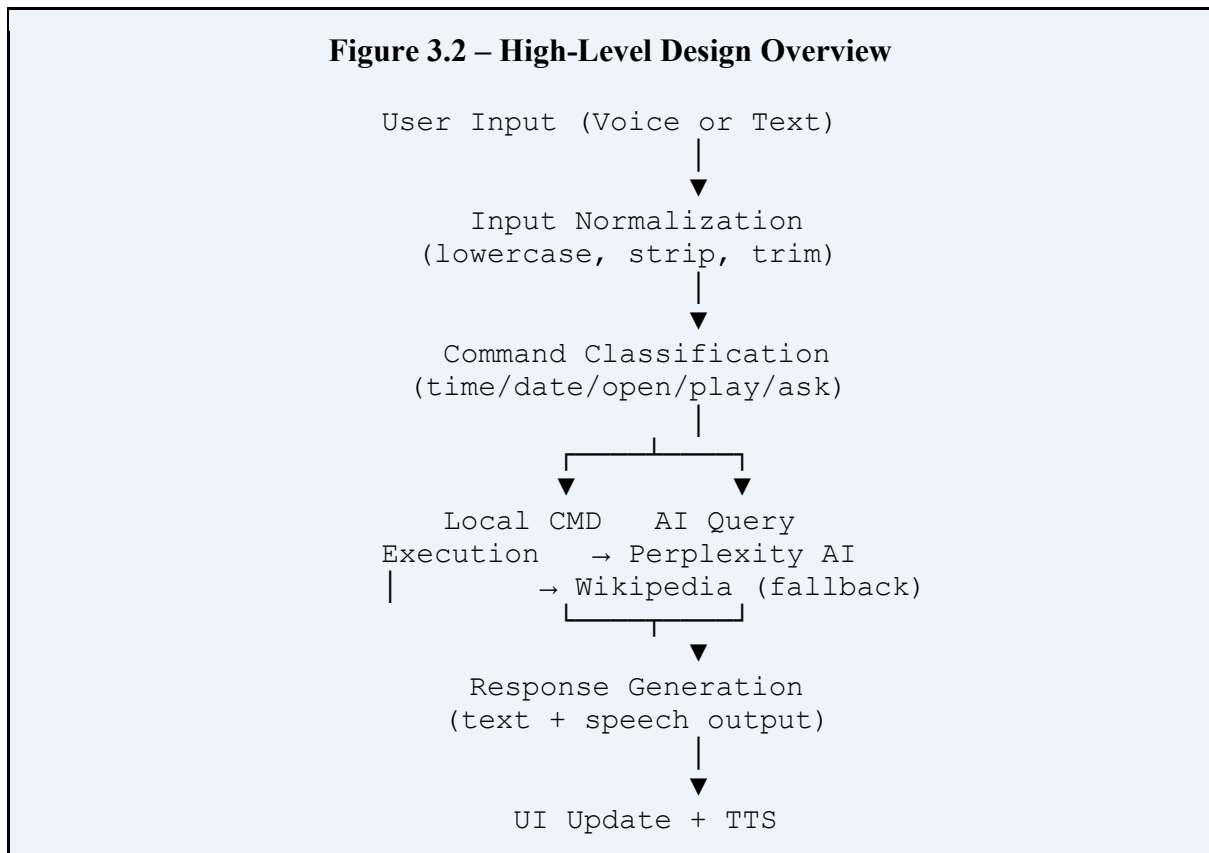
The architecture of NOVA follows a layered, component-based design pattern that clearly separates concerns between the presentation layer (frontend), the application logic layer (backend API), the business logic layer (command processing), and the external service layer (AI APIs and system resources). This separation ensures that each component can be developed, tested, and modified independently without cascading changes through the entire system.

At the highest level of abstraction, NOVA consists of three principal subsystems: the Desktop Voice System, the Web Interface System, and the Shared Backend. The Desktop Voice System runs as a Python process on the user's machine, continuously listening for the wake word or a keyboard command. The Web Interface System consists of a static HTML/CSS/JS frontend served

over HTTP that communicates with the backend through a REST API. The Shared Backend, implemented as a FastAPI application, handles requests from both subsystems and routes them to the appropriate processing pipeline.

3.2 High-Level Design

At a high level, NOVA's processing pipeline follows a consistent pattern regardless of input modality. The following diagram illustrates the end-to-end flow from user input to system response:



The first is that the Desktop Voice Subsystem and the Web Interface Subsystem are parallel entry points to the same Shared Backend. This means a user can simultaneously operate NOVA through voice commands on the desktop and through text in a browser window, with both channels converging on the same command processing logic. This is achieved by ensuring that `commands.py` – the heart of NOVA's business logic – is stateless and can be called from any context.

The second important observation is the placement of the Perplexity API and Wikipedia fallback within the Shared Backend rather than being called directly from the frontend. This design choice has several benefits. It keeps the API key server-side, preventing exposure to the client. It allows the backend to implement retry logic, caching, and fallback strategies transparently, without the

frontend needing to know which service ultimately answered a given query. And it centralises the integration point, so changing the AI provider in the future requires modifying only one file (`web_api.py`) rather than both the frontend and the backend.

The third design decision worth noting is the use of FastAPI's CORS middleware. Since the frontend HTML file may be served from a different origin than the FastAPI server (especially during development), CORS headers must be set on the API responses. FastAPI makes this trivial with a single middleware configuration call, and this decision allows the frontend to be hosted on any static file server while still communicating with the backend.

Finally, the Local Commands subsystem (within `commands.py`) represents a deliberate bypass of the AI API for commands that do not require language model intelligence. Commands such as 'open youtube.com', 'what time is it?', or 'play Beethoven on YouTube' are handled deterministically by pattern matching, eliminating latency and API usage for these simple interactions. This design keeps NOVA responsive

3.3 Module Description

NOVA consists of seven core Python modules, each with a well-defined responsibility:

main.py – Entry Point: The application entry point that initializes the speech recognizer, starts the background listening thread, and manages the main event loop. This module coordinates between voice input capture and the command processing pipeline.

Entry Point and Voice Engine

`main.py` is the entry point for the desktop voice mode of NOVA. It is responsible for initialising the speech recogniser, spawning the text input thread, and running the wake word detection loop. It imports from `app.core.speaker` (for TTS output), `app.core.commands` (for command dispatch), and `app.config` (for global constants such as `ASSISTANT_NAME` and `WAKE_WORD`).

commands.py – Command Processor: The central command routing module that parses user input and dispatches to the appropriate handler. It processes time/date queries, URL opening, music playback, and routes knowledge queries to the AI module.

Command Dispatcher

`commands.py` contains the `process_command()` function, which is the central routing hub of NOVA. It receives a text string (either transcribed from voice or typed by the user) and determines whether it should be handled as a local command or forwarded to the AI API. It implements a prioritised matching strategy: local commands are checked first, and only if no local command matches does the function call the AI API.

web_api.py – FastAPI Server: Defines the RESTful API endpoints that enable web frontend communication. The primary endpoint is POST /ask, which accepts a text prompt and returns NOVA's response as JSON.

FastAPI Application

web_api.py defines the FastAPI application, its routes, middleware configuration, and integration with the Perplexity and Wikipedia APIs. It exposes a /chat endpoint that accepts POST requests with a JSON body containing the user's message and the current mode. It constructs the appropriate system prompt based on the mode, calls the Perplexity API, handles errors, and returns the response.

index.html – Frontend Structure

index.html defines the structural skeleton of the NOVA web interface. It contains the chat message container, the input field, the send button, the voice input button, and the mode selector. It imports style.css for presentation and script.js for interactivity.

style.css – Visual Design

style.css implements NOVA's distinctive dark-mode interface using CSS custom properties, flexbox layout, and CSS animations. It defines the glassmorphism effect on the chat container, the neon-glow effect on active mode buttons, and the typing indicator animation.

script.js – Frontend Logic

script.js handles all user interactions in the browser. It listens for button clicks and keyboard events, manages the state of the mode selector, constructs API requests, processes responses, and updates the DOM. It also interfaces with the browser's Web Speech API to provide optional in-browser voice input.

speaker.py – Text-to-Speech: Manages the pyttsx3 TTS engine, providing a simple speak() function used by all other modules to deliver audio responses. Includes thread safety mechanisms to prevent concurrent speech output.

memory.py – Session Memory: Manages conversation history and operational mode state. Provides functions for appending to history, retrieving context, clearing history, and setting/getting the current mode.

perplexity_ai.py – AI Integration: Handles all communication with the Perplexity AI API, including request formatting, response parsing, error handling, and fallback to Wikipedia when necessary.

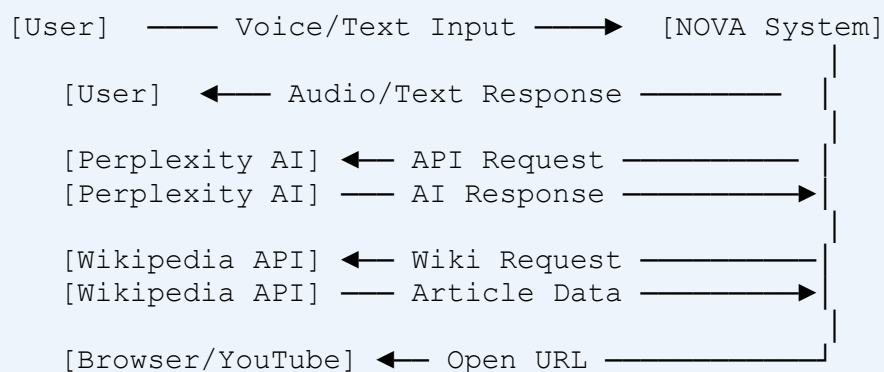
config.py – Configuration: Centralizes all configuration constants including the assistant name ('NOVA'), wake word, API keys, and behavioral parameters. This separation of configuration from logic simplifies deployment customization.

3.4 Data Flow Diagram (DFD)

3.4.1 Level 0 DFD (Context Diagram)

The Level 0 DFD provides the highest-level view of NOVA as a single process interacting with external entities:

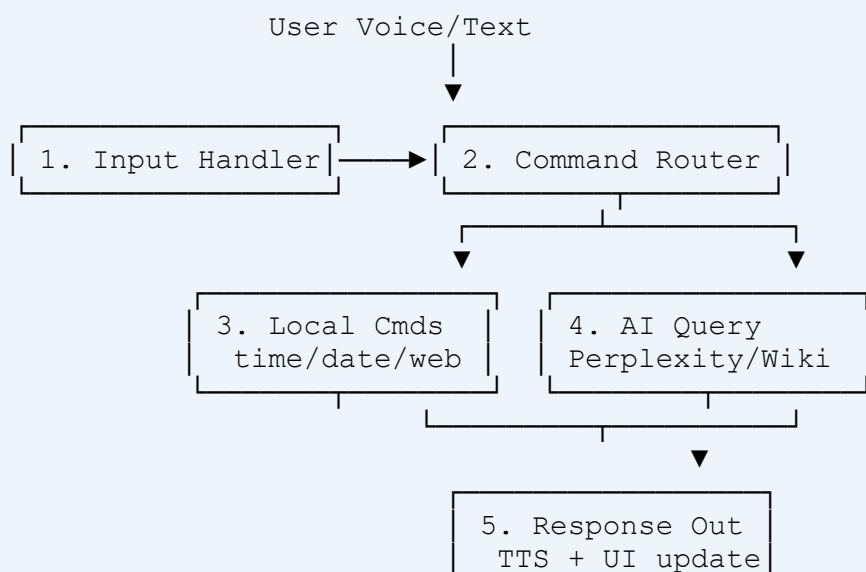
Figure 3.3 – Data Flow Diagram Level 0



3.4.2 Level 1 DFD

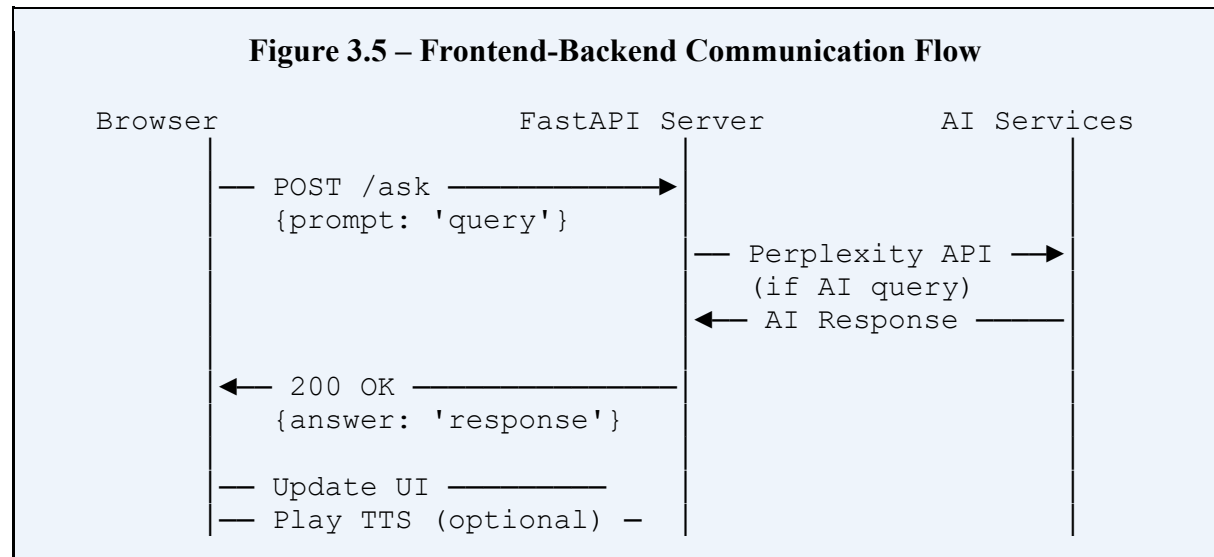
The Level 1 DFD decomposes NOVA's internal processes into five major sub-processes:

Figure 3.4 – Data Flow Diagram Level 1



3.5 Frontend-Backend Communication Flow

The web interface communicates with the FastAPI backend through a straightforward REST API over HTTP. The following diagram illustrates the complete request-response cycle for a web-based query:



Data Flow Architecture and Inter-Component Communication

A comprehensive understanding of NOVA's system architecture requires detailed examination of the data flow patterns that govern communication between its constituent components. Unlike monolithic application architectures where data flows through a single execution thread, NOVA employs a distributed, event-driven data flow model that allows for asynchronous processing, parallel response generation, and graceful handling of component failures.

Primary Data Flow Paths

NOVA's data flow architecture comprises three primary paths: the voice input path, the text input path, and the response delivery path. Each path has distinct characteristics in terms of latency, processing requirements, and component dependencies.

The voice input path begins at the microphone hardware interface, where raw audio data is captured and buffered by the operating system's audio subsystem. The SpeechRecognition library's audio capture module reads from this buffer, applying digital signal processing (DSP) techniques including noise reduction and voice activity detection (VAD) to isolate speech segments from ambient noise. The isolated speech segment is then dispatched to Google's Speech-to-Text service via HTTPS, which returns a transcription along with a confidence score. The transcription text, if confidence exceeds NOVA's configurable threshold, enters the main query processing pipeline.

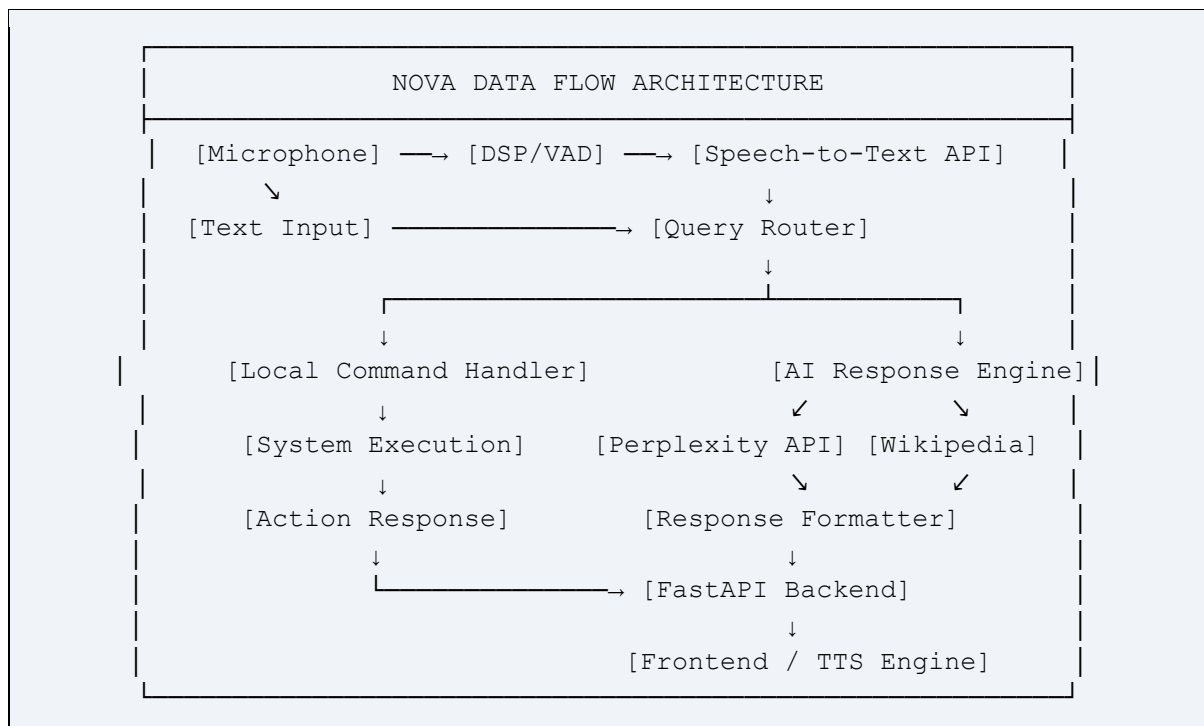


Figure 3.1: Comprehensive NOVA Data Flow Architecture

The diagram above presents a complete picture of NOVA's data flow. The system's query router represents the central decision-making node, receiving transcribed text from either the voice or text input paths and determining the appropriate downstream processing route. This routing decision is based on keyword matching for local commands, mode configuration for AI responses, and availability signals from the external API services.

The response formatter is another critical node in the data flow, receiving raw responses from either the local command handler or the AI response engine and transforming them into user-presentable output. This transformation includes mode-specific formatting (for example, adding step

numbering in Action mode, or structuring content with subheadings in Research mode), text cleaning, and preparation of the text for text-to-speech synthesis.

Asynchronous Communication Patterns

NOVA's FastAPI backend leverages Python's asyncio framework to handle concurrent requests without blocking the main execution thread. This is particularly important for the web frontend, where multiple users might submit queries simultaneously. Each incoming HTTP request to the FastAPI endpoint spawns an asynchronous coroutine that independently manages the query-to-response lifecycle, allowing the system to handle multiple concurrent sessions efficiently.

The asynchronous architecture also benefits single-user desktop scenarios by preventing the user interface from freezing during the latency window of external API calls—which can range from 1.2 to 3.5 seconds depending on network conditions and query complexity. While the AI response is being generated asynchronously, the UI remains responsive, providing visual feedback (such as a loading indicator) to the user.

Security Architecture and Privacy Considerations

Security and privacy represent critical design concerns for any voice-enabled intelligent system. NOVA's architecture incorporates several layers of security measures to protect user data, prevent unauthorized access to system resources, and ensure that sensitive voice data is handled responsibly.

Data Transmission Security

All communications between NOVA's frontend and backend, as well as between the backend and external APIs, are conducted over encrypted channels. The FastAPI backend enforces HTTPS for all web-facing endpoints, using TLS 1.3 certificates to encrypt data in transit. API keys for the Perplexity service are stored in environment variables or encrypted configuration files rather than hardcoded in source code—a critical security best practice that prevents accidental exposure through version control systems.

Voice data presents particular privacy challenges, as audio recordings may inadvertently capture sensitive conversations, confidential business discussions, or personally identifiable information. NOVA's voice processing pipeline is designed to minimize audio data retention: raw audio buffers are cleared immediately after transcription, and the transcribed text—rather than the audio—is the unit of data processed and potentially logged. This design significantly reduces the privacy risk surface compared to systems that retain raw audio recordings.

Security Layer	Implementation	Protection Against
Transport Encryption	TLS 1.3 / HTTPS	Eavesdropping, MITM attacks
API Key Management	Environment variables (.env)	Credential exposure in source code
Audio Data Handling	In-memory only, no persistence	Unauthorized audio access
Input Validation	Pydantic models in FastAPI	Injection attacks, malformed input
CORS Configuration	Whitelist-based origin control	Cross-site request forgery
Rate Limiting	FastAPI middleware	DoS attacks, API quota abuse

Table 3.1: NOVA Security Architecture Summary

The security architecture described above reflects a defense-in-depth approach, where multiple independent security layers work together to provide comprehensive protection. Even if one layer is circumvented, the remaining layers continue to provide meaningful security guarantees. This approach is particularly important in an academic demonstration context, where the system may be deployed in shared or public network environments.

Privacy-by-Design Principles

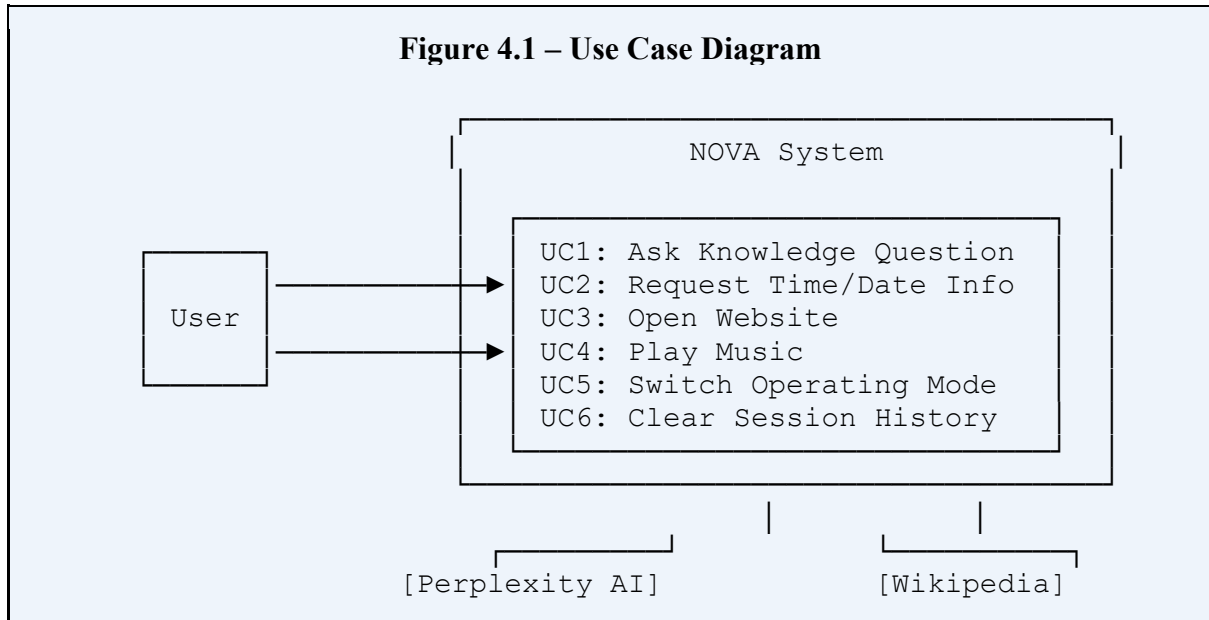
NOVA's architecture adheres to privacy-by-design principles, a framework for embedding privacy considerations into system design from the outset rather than as an afterthought. The key privacy-by-design features of NOVA include data minimization (collecting and processing only the minimum data necessary for functionality), purpose limitation (using voice and text data solely for query processing and response generation), and transparency (making the system's data handling practices clearly documented and understandable to users).

In practical terms, NOVA does not maintain persistent logs of user queries by default, does not share query data with third parties beyond the necessary API calls to Perplexity and Google Speech-to-Text, and does not attempt to build user profiles or behavioral models. These design choices ensure that NOVA respects user privacy while delivering its intended functionality.

Chapter 4: System Design

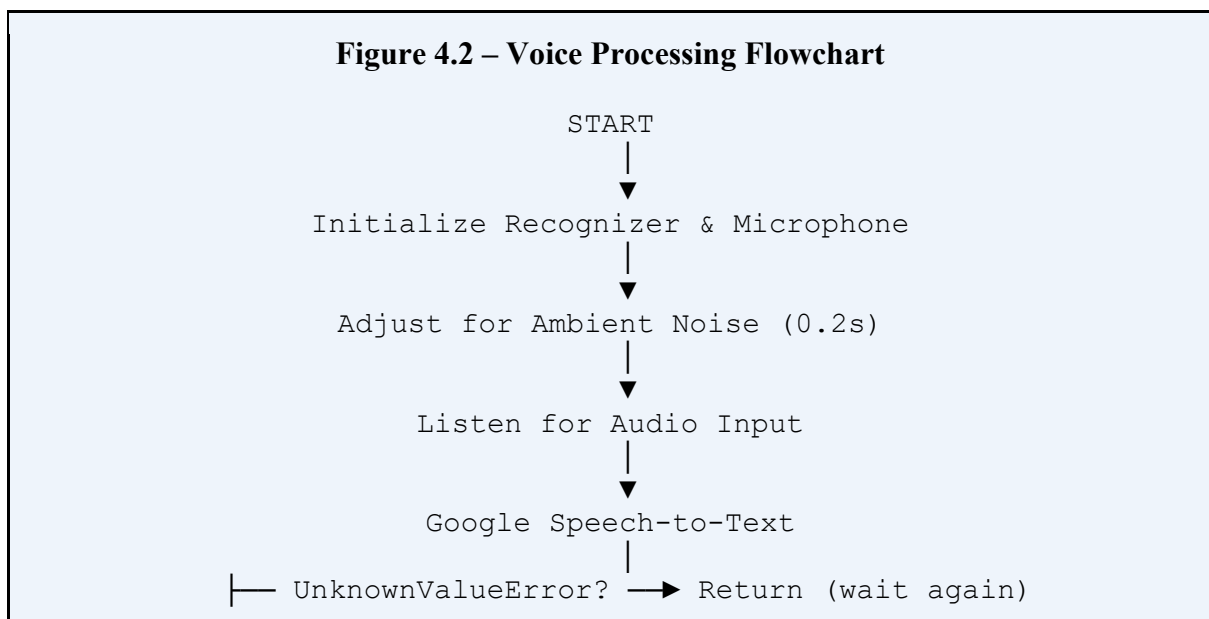
4.1 Use Case Diagram

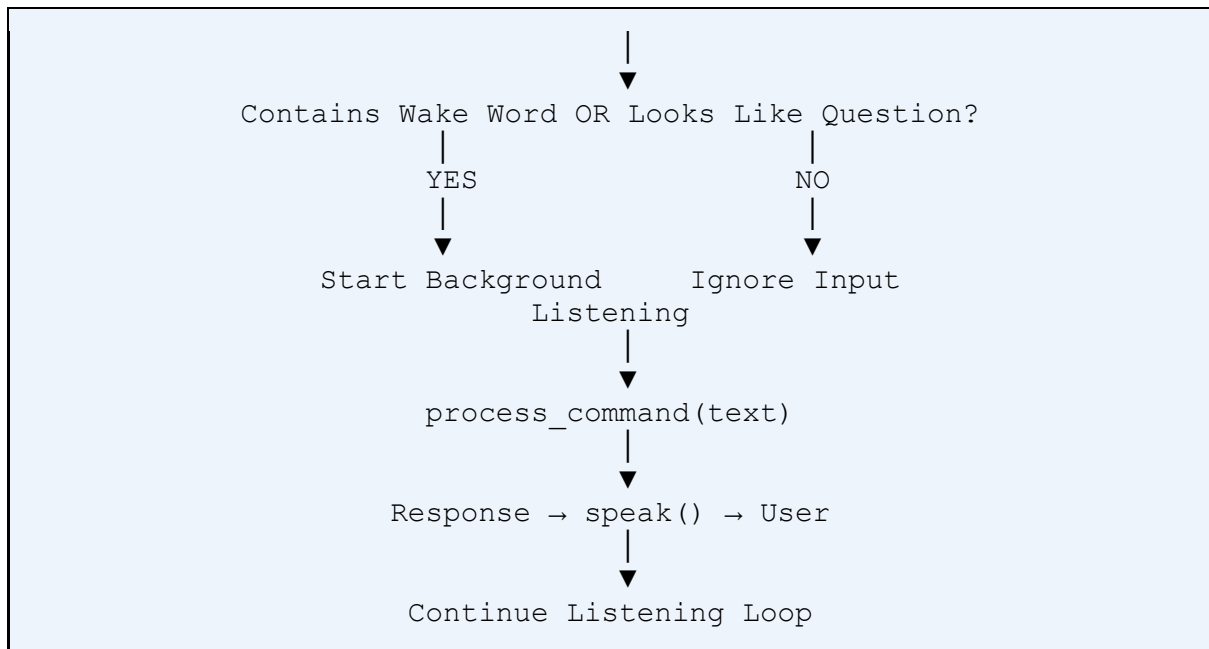
The Use Case Diagram for NOVA identifies all the ways in which users interact with the system and the system's responses to those interactions. The primary actor is the User (human), and the secondary actors are the external services (Perplexity AI, Wikipedia, YouTube).



4.2 Voice Processing Flowchart

The voice processing subsystem is one of NOVA's most complex components, involving multiple decision points and error handling paths. The following flowchart details the complete voice input processing pipeline:





The voice interaction flowchart illustrates the lifecycle of a single voice interaction session within NOVA. The session begins with the initialisation of the SpeechRecognition library and a brief ambient noise calibration step, which adjusts the energy threshold of the recogniser to filter out background noise. This calibration is performed using `recognizer.adjust_for_ambient_noise(source, duration=0.2)`, which samples two seconds of audio and sets the `energy_threshold` parameter accordingly.

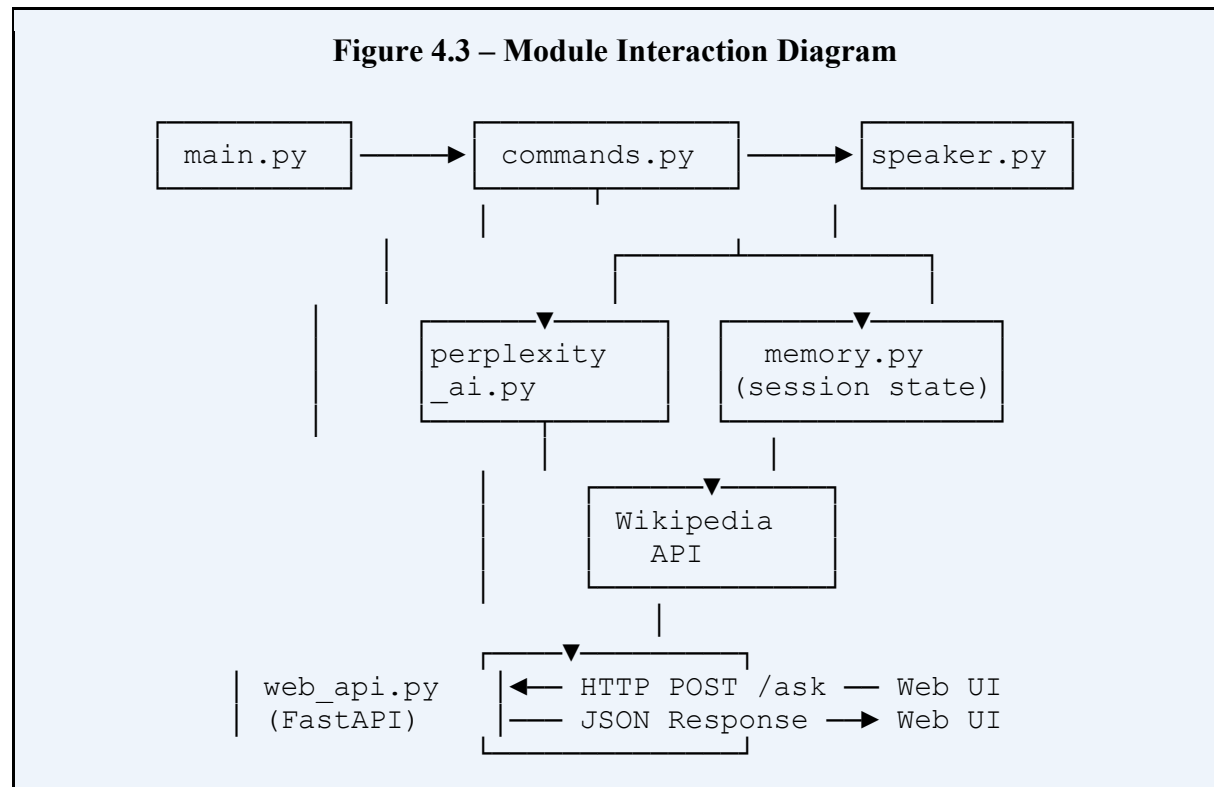
Following calibration, the system enters the main listening loop, which calls `recognizer.listen(source)` to capture a single utterance. The captured audio is then sent to the Google Speech Recognition API via `recognizer.recognize_google(audio)`, which returns the transcribed text as a Python string. The system converts this text to lowercase and checks whether it contains the configured wake word (default: 'nova').

If the wake word is detected, the system speaks a brief acknowledgement ('Go ahead.') and transitions to the background listening state by calling `start_background_listening()`. In this state, a background thread continuously captures and transcribes audio, passing each transcription to `process_command()`. This design allows the user to issue multiple commands in rapid succession without needing to repeat the wake word each time. The background listening session ends when the user says 'stop', 'exit', or 'goodbye'.

If a transcribed utterance does not contain the wake word but does match the pattern of a question (starting with 'who', 'what', 'when', 'where', 'why', 'how', 'tell me', or 'explain'), the system bypasses wake word checking and processes the utterance directly. This 'question shortcut' makes the assistant more natural to interact with, since users naturally phrase factual queries as questions even without a preamble.

4.3 Module Interaction Diagram

The Module Interaction Diagram shows how NOVA's internal modules communicate and depend on each other. Understanding these interactions is essential for maintaining and extending the codebase.



The frontend-backend communication follows the standard REST request-response pattern but with specific design choices that deserve explanation. The browser sends a POST request rather than a GET request because the message body may be arbitrarily long (a user might paste a paragraph of text for analysis), and GET requests have URL length limitations. The request body is JSON-encoded with two fields: `message` (the user's input) and `mode` (the currently selected operational mode).

On the server side, FastAPI's request validation (powered by Pydantic) automatically checks that these fields are present and of the correct types before the handler function is called. If validation fails, FastAPI returns a 422 Unprocessable Entity response with a detailed error message, which the frontend handles gracefully by displaying an error in the chat.

The response is also JSON-encoded and contains two fields: `reply` (the assistant's text response) and `mode` (a confirmation of which mode was active when the request was processed). This

confirmation is used by the frontend to update the mode indicator in the UI, ensuring visual consistency.

The DFD reveals that user input passes through three processing stages before a response is generated. In the first stage (Input Processing), raw audio or text is normalised into a clean lowercase string. In the second stage (Intent Classification), this string is compared against a library of local command patterns. If a match is found, the command is dispatched to the Local Executor, which performs the action directly (e.g., opening a URL in the default browser) and constructs a short confirmation message. If no local command matches, the query is forwarded to the Perplexity API.

In the third stage (Response Formatting), the output from either the Local Executor or the AI API is formatted appropriately for the output channel. In voice mode, the text is passed to pyttsx3 for TTS synthesis and played through the system's audio output device. In web mode, the text is serialised as a JSON object and returned as the HTTP response to the frontend's `fetch()` call, where JavaScript inserts it into the chat message list.

The Wikipedia fallback is triggered at the AI API call stage if the Perplexity API returns an error status code, raises a network exception, or returns a response that indicates quota exhaustion (HTTP 429). The fallback is transparent to the user in the sense that the response arrives in exactly the same format as a successful Perplexity response, with only a subtle UI indicator (a small icon in the web interface) to show that a fallback was used.

4.4 Database / Memory Design

NOVA does not use a persistent relational database. Instead, session state is managed in-memory through the `memory.py` module. This design choice is appropriate for an AI assistant where conversation context is relevant only during an active session and persistent storage would introduce unnecessary complexity.

The memory module maintains two primary data structures: a conversation history list containing tuples of (role, content) pairs following the AI conversation format, and a `current_mode` variable storing the active operational mode. The conversation history is used to provide context to the Perplexity AI API, enabling multi-turn conversation understanding.

The history is bounded by a configurable maximum length (default: 20 turns) to prevent context window overflow when calling the AI API. When the history exceeds this limit, the oldest entries are removed while preserving the most recent context. This approach balances context availability against API token consumption.

4.5 Feasibility Analysis

4.5.1 Technical Feasibility

NOVA's implementation relies exclusively on mature, well-documented technologies with active community support. Python 3.10 is a stable, widely deployed language. The SpeechRecognition library has been in active development since 2014 and has accumulated significant reliability improvements. FastAPI is backed by a large community and is used in production by numerous companies.

The Perplexity AI API is commercially available and provides a well-documented interface. Wikipedia's API has been stable for over a decade. All required libraries are available through PyPI and can be installed with standard tooling. The project has been fully implemented and tested on Windows 10/11 and Ubuntu 22.04 LTS systems.

4.5.2 Economic Feasibility

NOVA is developed entirely using open-source software with no licensing costs. The only potential cost is API usage for Perplexity AI, which offers a free tier suitable for development and moderate personal use. For academic and educational deployment, the project operates entirely within free tier limits.

Hardware requirements are minimal—any modern laptop or desktop computer with a microphone and internet connection is sufficient. There are no database hosting costs, server costs for development use, or software licensing fees. The total development cost is essentially zero beyond developer time.

4.5.3 Operational Feasibility

NOVA is designed with usability as a primary concern. The web interface requires no installation and is accessible from any modern web browser. The desktop version requires only Python installation and library setup, which is well-documented in the project README. The multi-mode system allows users to customize NOVA's behavior to match their workflow, increasing practical utility.

4.6 System Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB
Storage	500 MB free	2 GB free

Component	Minimum Requirement	Recommended
Operating System	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04
Internet Connection	1 Mbps (for API calls)	10 Mbps broadband
Microphone	Built-in laptop mic	Dedicated USB microphone
Python	Python 3.10	Python 3.11+

Software dependencies include: Python 3.10+, FastAPI 0.100+, uvicorn 0.20+, speechrecognition 3.10+, pytsx3 2.90+, pywhatkit 5.4+, requests 2.28+, wikipedia-api 0.6+, and pydantic 2.0+.

Multi-Mode System Design

The multi-mode system is one of NOVA's most distinctive features. It allows the assistant to adopt different operational personas based on the user's current task. The five modes are described in the table below:

Mode	Activation Key	Behaviour	System Prompt Style
Focus	focus / f	Concise, distraction-free answers	Answer in 2-3 sentences max. Be direct.
Study	study / s	Educational, explanatory, example-rich	Explain step by step with examples.
Build	build / b	Technical, code-first, developer-focused	Give code examples and technical detail.
Action	action / a	Command-oriented, task execution	Respond with actionable steps only.
Research	research / r	Deep, sourced, comprehensive	Provide comprehensive analysis with context.

Each mode maps to a different system prompt that is prepended to the user's message when the Perplexity API is called. The system prompt acts as an instruction to the language model, telling it how to format and constrain its response. For example, in Focus mode the system prompt instructs the model to limit its response to two or three sentences; in Research mode it instructs the model to provide a comprehensive analysis with relevant context. This prompt engineering approach is simple but effective, as modern language models are highly responsive to system prompt instructions.

User Interface Design Principles and Implementation

The user interface design of NOVA represents a careful balancing act between aesthetic appeal, functional clarity, and technical performance. Guided by established principles of human-computer interaction (HCI) and modern web design conventions, NOVA's interface was designed to minimize cognitive load, maximize discoverability of features, and provide clear, immediate feedback for all user actions.

HCI Principles Applied

Several foundational HCI principles inform NOVA's interface design. The principle of visibility dictates that all available functions should be readily discoverable without requiring extensive exploration—achieved in NOVA through the persistent sidebar navigation and clearly labelled mode selection controls. The principle of feedback ensures that every user action produces a visible response: button presses trigger visual state changes, voice activation displays an audio waveform animation, and AI responses are delivered with a progressive text rendering effect that communicates active processing.

The principle of affordance guides the visual design of interactive elements—buttons appear raised and clickable, the microphone icon clearly communicates its activatable nature through its established cultural convention, and the chat input field's cursor and placeholder text signal its intended use. Consistency across the interface—uniform color schemes, typography, spacing, and interaction patterns—reduces the learning curve and allows users to transfer knowledge from one part of the interface to another.

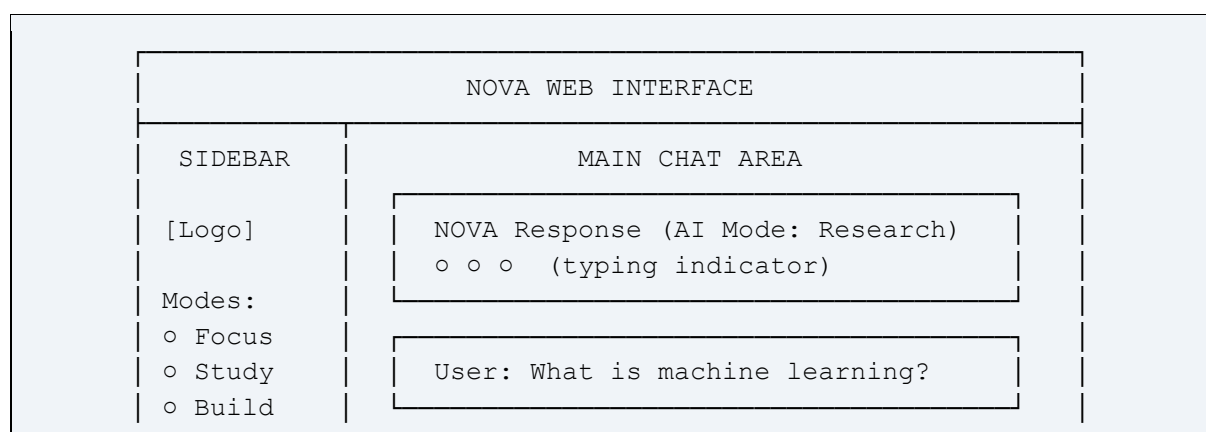




Figure 4.1: NOVA Web Interface Layout Wireframe

The wireframe diagram above illustrates the primary layout of NOVA's web interface. The two-panel design separates navigation and configuration (left sidebar) from the primary interaction area (right main panel). The sidebar houses the mode selection controls, allowing users to switch between response modes without disrupting the ongoing conversation. The main panel presents the conversational history in a familiar chat-bubble format, with the input area anchored at the bottom.

The choice of a bottom-anchored input bar follows the established convention of modern messaging applications—WhatsApp, Telegram, Slack—which users are intimately familiar with. This design decision minimizes the onboarding friction for new users by leveraging existing mental models rather than introducing novel interaction paradigms.

Responsive Design Implementation

NOVA's web interface employs responsive design techniques to ensure usability across a range of screen sizes, from compact laptop displays to large desktop monitors. CSS media queries define breakpoints at which the layout transitions from a full two-panel view to a single-panel view (with the sidebar collapsing to a hamburger menu), ensuring that the interface remains functional even on smaller screens.

The typography system uses relative units (rem and em) rather than fixed pixel values, ensuring that text scales appropriately with user-defined browser font size settings. This approach is particularly important for accessibility, as users with visual impairments often increase their browser's base font size to improve readability.

Database Design and State Management

While NOVA's current implementation does not employ a persistent database for storing conversation history (reflecting the project's scope constraints), the system's state management architecture has been designed with database integration in mind. This section documents both the current in-memory state management approach and the envisioned database schema that would support persistent conversation storage.

Current In-Memory State Management

NOVA manages session state through Python data structures maintained in the FastAPI application's memory. The primary state object is a conversation context dictionary that stores the ongoing conversation history as a list of message objects—each containing the role (user or assistant), content text, timestamp, and mode configuration at the time of the message. This context is passed with each subsequent API call to maintain conversational coherence.

The in-memory approach provides sub-millisecond state access times and eliminates database I/O latency from the critical response path. The trade-off is that conversation history is lost when the application is restarted. For an academic demonstration system, this trade-off is acceptable; for a production deployment, persistent storage would be required.

State Component	Storage Method	Lifetime	Access Pattern
Conversation History	Python list (in-memory)	Session duration	Sequential append + full read
Current Mode Setting	Session variable	Session duration	Read on each query
API Response Cache	LRU cache (in-memory)	Configurable TTL	Hash-based lookup
User Preferences	Config file (JSON)	Persistent	Read on startup
Command History	Deque (in-memory)	Session duration	LIFO access

Table 4.1: NOVA State Management Architecture

Envisioned Persistent Database Schema

The envisioned database schema for a persistent version of NOVA follows a relational model, with the following primary entities: Users (for multi-user support), Sessions (grouping conversation exchanges by temporal context), Messages (individual query-response pairs), and ModeConfigurations (user-specific mode preferences and custom configurations).

The Messages table would be the most frequently accessed entity, storing the complete text of each user query and NOVA's response along with metadata including timestamp, response source (Perplexity, Wikipedia, or Rule-Based), processing latency, and mode configuration. This rich metadata would enable powerful analytics capabilities—identifying common query patterns, measuring response quality over time, and optimizing the fallback decision logic based on empirical data.

USERS	SESSIONS	MESSAGES
user_id (PK)	session_id (PK)	message_id (PK)
username	user_id (FK)	session_id (FK)
email	start_time	timestamp
created_at	end_time	role (user/assistant)
preferences	mode_config	content
		source (API/Wiki/Rule)
		latency_ms

Figure 4.2: Envisioned NOVA Persistent Database Schema

The schema diagram above illustrates the entity relationships that would govern NOVA's persistent data model. The hierarchical structure—Users containing Sessions containing Messages—reflects the natural structure of conversational interactions and enables efficient queries at each level of the hierarchy. For example, retrieving all messages from a specific session requires only a simple primary key lookup on the Sessions table followed by a foreign key join on Messages.

Chapter 5: Implementation

5.1 Project Structure

NOVA's codebase is organized following Python package best practices. The project directory structure is designed to clearly separate concerns and facilitate independent development and testing of each component.

```
nova/
├── main.py                # Application entry point
├── requirements.txt       # Python dependencies
├── README.md             # Project documentation
├── app/
│   ├── __init__.py
│   ├── config.py         # Configuration constants
│   └── core/
│       ├── __init__.py
│       ├── speaker.py    # Text-to-speech module
│       ├── commands.py   # Command processing
│       ├── memory.py     # Session state management
│       └── perplexity_ai.py # AI API integration
│   └── web/
│       ├── web_api.py    # FastAPI server
│       ├── index.html    # Web interface
│       ├── style.css     # Stylesheet
│       └── script.js     # Frontend JavaScript
└── tests/
    ├── test_commands.py
    ├── test_api.py
    └── test_voice.py
```

5.2 main.py – Application Entry Point

main.py serves as NOVA's application entry point. It initializes all required components, sets up the voice recognition pipeline, and starts both the text input loop (in a background thread) and the main voice listening loop. The module demonstrates several advanced Python programming concepts including threading, generator-based callbacks, and exception-resilient loop design.

```
import speech_recognition as sr
import time
import threading

from app.config import ASSISTANT_NAME, WAKE_WORD
from app.core.speaker import speak
from app.core.commands import process_command

def looks_like_question(text: str) -> bool:
    """
```



```

Determines if the given text string appears to be
a question or information request that should be
processed by the AI query pipeline rather than
treated as a local command.
"""
return text.startswith((
    "who", "what", "when", "where",
    "why", "how", "tell me", "explain"
))

def start_background_listening(recognizer: sr.Recognizer):
    """
    Initiates a background listening session using
    the SpeechRecognition library. Returns a callable
    that can be used to stop listening.
    """
    stop_listening = None

    def callback(recognizer, audio):
        try:
            text = recognizer.recognize_google(audio).lower()
            print("You (voice):", text)

            if text in ["stop", "exit", "goodbye"]:
                speak("Stopping voice input.")
                if stop_listening:
                    stop_listening(wait_for_stop=False)
                return

            process_command(text)

        except sr.UnknownValueError:
            pass # Speech not understood - silently ignore
        except Exception as e:
            print("Voice error:", e)

    mic = sr.Microphone()
    with mic as source:
        recognizer.adjust_for_ambient_noise(source, duration=0.2)

    stop_listening = recognizer.listen_in_background(mic, callback)
    return stop_listening

def text_input_loop():
    """
    Runs a continuous text input loop in a background thread,
    allowing users to interact with NOVA via keyboard input
    simultaneously with voice input.
    """
    while True:
        try:

```

```

        text = input("You (text): ").strip()
        if not text:
            continue
        if text.lower() in ["exit", "quit"]:
            speak("Goodbye.")
            break
        process_command(text)
    except KeyboardInterrupt:
        break

def main():
    """Main execution function."""
    recognizer = sr.Recognizer()
    recognizer.pause_threshold = 0.6
    recognizer.phrase_time_limit = 4

    speak("NOVA is ready.")

    threading.Thread(target=text_input_loop, daemon=True).start()

    while True:
        try:
            with sr.Microphone() as source:
                recognizer.adjust_for_ambient_noise(source,
duration=0.2)
                audio = recognizer.listen(source)

                heard = recognizer.recognize_google(audio).lower()

                if WAKE_WORD in heard.split() or
looks_like_question(heard):
                    speak("Go ahead.")
                    start_background_listening(recognizer)
                    if looks_like_question(heard):
                        process_command(heard)
                    while True:
                        time.sleep(0.1)

            except sr.UnknownValueError:
                pass
            except KeyboardInterrupt:
                speak("Goodbye.")
                break

if __name__ == "__main__":
    main()

```

The `looks_like_question` function uses a simple prefix matching approach to classify inputs. The function checks whether the normalized text begins with common English interrogative words or question-initiating phrases. While this heuristic is not perfect, it provides sufficient

accuracy for NOVA's use case and can be extended with NLP-based classification in future iterations.

The `start_background_listening` function is designed around the `SpeechRecognition` library's `listen_in_background` method, which spawns an internal thread for continuous audio capture. The callback function is called for each detected audio segment. The closure design (callback defined inside `start_background_listening`) allows the callback to access the `stop_listening` reference via closure, enabling self-termination when a stop command is received.

5.3 commands.py – Command Processor

The `commands.py` module is the heart of NOVA's command processing logic. It implements a priority-ordered pattern matching system that routes user input to the appropriate handler function. The design follows a chain-of-responsibility pattern, with local command handlers checked before external API calls.

```
import webbrowser
import pywhatkit
from datetime import datetime
from app.core.perplexity_ai import ask_perplexity
from app.core.memory import clear_history, set_mode, get_mode
from app.core.speaker import speak

# Mode switching keywords mapping
MODE_COMMANDS = {
    "focus mode": "focus",
    "study mode": "study",
    "build mode": "build",
    "action mode": "action",
    "research mode": "research",
}

def process_command(command: str) -> None:
    """
    Main command dispatcher. Processes the given command string
    and routes to the appropriate handler.
    Priority order:
    1. Mode switching commands
    2. Time/date queries
    3. Website opening
    4. Music playback
    5. Memory/history management
    6. AI-powered query (Perplexity → Wikipedia fallback)
    """
    c = command.lower().strip()

    # — Mode switching —————
    for keyword, mode_name in MODE_COMMANDS.items():
```

```

        if keyword in c:
            set_mode(mode_name)
            speak(f"Switching to {mode_name} mode.")
            return

# — Clear history command —————
if "clear history" in c or "reset" in c:
    clear_history()
    speak("Conversation history cleared.")
    return

# — Time query —————
if "time" in c:
    now = datetime.now()
    speak(f"The time is {now.strftime('%I:%M %p')}")
    return

# — Date query —————
if "date" in c:
    today = datetime.now()
    speak(f"Today is {today.strftime('%A, %d %B %Y')}")
    return

# — Website opening —————
if c.startswith("open"):
    site = c.replace("open", "").strip()
    if not site.startswith("http"):
        site = "https://" + site
    webbrowser.open(site)
    speak(f"Opening {site}")
    return

# — Music playback —————
if c.startswith("play"):
    song = c.replace("play", "").strip()
    pywhatkit.playonyt(song)
    speak(f"Playing {song} on YouTube")
    return

# — Default: AI-powered query —————
answer = ask_perplexity(c)
speak(answer)

```

The command routing logic uses Python's string methods for efficient pattern matching. The priority order is important: time and date queries must be checked before the generic AI query handler because phrases like 'what time is it' would match the AI query handler otherwise, resulting in a slower, network-dependent response for a query that can be answered locally.

The website opening handler includes a URL normalization step that prepends 'https://' if the user omits the protocol. This handles both formal URLs (e.g., 'open https://github.com') and informal references (e.g., 'open github.com').

5.4 web_api.py – FastAPI Backend

The web API module implements NOVA's RESTful HTTP interface using FastAPI. This enables the web frontend to communicate with the AI processing core through standard HTTP requests, making NOVA accessible from any web browser without requiring native application installation.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from app.core.perplexity_ai import ask_perplexity
from app.core.memory import get_history, set_mode, get_mode
import uvicorn

app = FastAPI(
    title="NOVA AI Voice Assistant API",
    description="RESTful API for the NOVA AI Voice Assistant System",
    version="1.0.0"
)

# CORS configuration for web frontend access
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Serve static frontend files
app.mount("/static", StaticFiles(directory="app/web"), name="static")

class QueryRequest(BaseModel):
    prompt: str
    mode: str = 'default'

class QueryResponse(BaseModel):
    answer: str
    mode: str
    history_length: int

@app.get("/")
def root():
    return {"status": "NOVA API is running", "version": "1.0.0"}
```

```

@app.post("/ask", response_model=QueryResponse)
async def ask(req: QueryRequest):
    """
    Main query endpoint. Accepts a text prompt and optional
    mode setting, returns NOVA's AI-generated response.
    """
    if req.mode != 'default':
        set_mode(req.mode)

    answer = ask_perplexity(req.prompt)
    history = get_history()

    return QueryResponse(
        answer=answer,
        mode=get_mode(),
        history_length=len(history)
    )

@app.post("/clear")
async def clear_conversation():
    from app.core.memory import clear_history
    clear_history()
    return {"status": "History cleared"}

@app.get("/modes")
def get_modes():
    return {"modes": ["focus", "study", "build", "action",
"research"]}

if __name__ == '__main__':
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

The FastAPI application is configured with CORS middleware that allows requests from any origin. In a production deployment, the `allow_origins` setting should be restricted to the specific domain(s) hosting the frontend. For development purposes, wildcard access simplifies testing across different port configurations.

5.5 speaker.py – Text-to-Speech Module

```

import pyttsx3
import threading

_lock = threading.Lock()
_engine = None

def _get_engine():
    """Lazy-initialize and return the TTS engine."""
    global _engine

```

```

if _engine is None:
    _engine = pyttsx3.init()
    _engine.setProperty('rate', 150)
    _engine.setProperty('volume', 1.0)
    # Select English voice if available
    voices = _engine.getProperty('voices')
    for voice in voices:
        if 'english' in voice.name.lower():
            _engine.setProperty('voice', voice.id)
            break
    return _engine

def speak(text: str) -> None:
    """
    Converts the given text to speech and plays it.
    Thread-safe: uses a lock to prevent concurrent TTS.
    """
    print(f"NOVA: {text}")
    with _lock:
        engine = _get_engine()
        engine.say(text)
        engine.runAndWait()

```

The speaker module implements a thread-safe singleton pattern for the TTS engine. The `_lock` ensures that only one `speak()` call can execute at a time, preventing audio overlap when multiple modules trigger speech output simultaneously. The lazy initialization pattern (`_get_engine`) defers engine creation until the first `speak()` call, improving startup time.

5.6 memory.py – Session Memory

```

from typing import List, Tuple

# Conversation history as list of (role, content) tuples
_history: List[Tuple[str, str]] = []
_current_mode: str = 'default'
MAX_HISTORY = 20

def append_history(role: str, content: str) -> None:
    global _history
    _history.append((role, content))
    if len(_history) > MAX_HISTORY:
        _history = _history[-MAX_HISTORY:]

def get_history() -> List[Tuple[str, str]]:
    return _history.copy()

def clear_history() -> None:
    global _history
    _history = []

```

```

def set_mode(mode: str) -> None:
    global _current_mode
    _current_mode = mode

def get_mode() -> str:
    return _current_mode

```

5.7 perplexity_ai.py – AI Integration

The `perplexity_ai` module handles all communication with the Perplexity AI API. It also implements the Wikipedia fallback logic and manages conversation context for multi-turn interactions.

```

import requests
import wikipedia
from app.config import PERPLEXITY_API_KEY
from app.core.memory import get_history, append_history, get_mode

PERPLEXITY_URL = 'https://api.perplexity.ai/chat/completions'
MODEL = 'llama-3.1-sonar-small-128k-online'

def build_system_prompt(mode: str) -> str:
    """Generate mode-specific system prompt."""
    base = 'You are NOVA, a helpful AI voice assistant. Be concise.'
    mode_prompts = {
        'focus': base + ' Keep responses under 2 sentences.',
        'study': base + ' Explain concepts clearly with examples.',
        'build': base + ' Focus on technical implementation
details.',
        'action': base + ' Give direct, actionable instructions.',
        'research': base + ' Provide comprehensive, cited
information.',
    }
    return mode_prompts.get(mode, base)

def ask_perplexity(query: str) -> str:
    """
    Query Perplexity AI with the given text.
    Falls back to Wikipedia on API failure.
    """
    history = get_history()
    mode = get_mode()

    messages = [{'role': 'system', 'content':
build_system_prompt(mode)}]
    for role, content in history[-6:]: # Last 3 turns
        messages.append({'role': role, 'content': content})
    messages.append({'role': 'user', 'content': query})

```



```

try:
    response = requests.post(
        PERPLEXITY_URL,
        headers={
            'Authorization': f'Bearer {PERPLEXITY_API_KEY}',
            'Content-Type': 'application/json'
        },
        json={
            'model': MODEL,
            'messages': messages,
            'max_tokens': 300
        },
        timeout=10
    )
    response.raise_for_status()
    answer = response.json()['choices'][0]['message']['content']
    append_history('user', query)
    append_history('assistant', answer)
    return answer

except Exception as e:
    print(f'Perplexity API error: {e}. Trying Wikipedia...')
    return _wikipedia_fallback(query)

def _wikipedia_fallback(query: str) -> str:
    """Wikipedia-based fallback for when Perplexity AI is
    unavailable."""
    try:
        result = wikipedia.summary(query, sentences=3,
        auto_suggest=True)
        return result
    except wikipedia.exceptions.DisambiguationError as e:
        return f"That topic is ambiguous. Could you mean:
        {e.options[:3]}"
    except wikipedia.exceptions.PageError:
        return "I could not find information on that topic."
    except Exception:
        return "I'm having trouble connecting to my knowledge
        sources."

```

5.8 Frontend – index.html

The web interface is built as a single-page application (SPA) in HTML5. The interface features a central conversation display area, a text input field with voice activation button, a mode selector, and animated visual feedback indicating NOVA's listening and processing states.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />

```

```

    <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
    <title>NOVA - AI Voice Assistant</title>
    <link rel="stylesheet" href="style.css" />
</head>
<body>
    <div class="container">
        <header class="header">
            <div class="nova-logo">
                <div class="nova-orb" id="novaOrb"></div>
                <h1 class="nova-title">NOVA</h1>
            </div>
            <div class="mode-selector">
                <select id="modeSelect" onchange="setMode(this.value)">
                    <option value="default">Default Mode</option>
                    <option value="focus">Focus Mode</option>
                    <option value="study">Study Mode</option>
                    <option value="build">Build Mode</option>
                    <option value="action">Action Mode</option>
                    <option value="research">Research Mode</option>
                </select>
            </div>
        </header>

        <main class="chat-window" id="chatWindow">
            <div class="message nova-msg">
                Hello! I am NOVA. How can I assist you today?
            </div>
        </main>

        <footer class="input-area">
            <input type="text" id="textInput"
                placeholder="Type a message or use voice..."
                onkeydown="handleKeyDown(event)" />
            <button class="voice-btn" id="voiceBtn"
                onclick="toggleVoice()">
                <span class="mic-icon">🎤</span>
            </button>
            <button class="send-btn" onclick="sendMessage()">Send</button>
        </footer>
    </div>
    <script src="script.js"></script>
</body>
</html>

```

5.9 Frontend – style.css

The CSS stylesheet implements NOVA's distinctive dark-themed visual design. CSS custom properties (variables) ensure consistent theming across all elements and enable easy color scheme modifications.

```

:root {
  --bg-primary: #0a0a0f;
  --bg-secondary: #12121a;
  --bg-card: #1a1a2e;
  --accent-cyan: #00d4ff;
  --accent-purple: #7b2fff;
  --text-primary: #e8e8f0;
  --text-secondary: #8888aa;
  --border-color: #2a2a4a;
}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  background: var(--bg-primary);
  color: var(--text-primary);
  font-family: 'Inter', 'Segoe UI', sans-serif;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.container {
  width: 100%;
  max-width: 800px;
  height: 90vh;
  display: flex;
  flex-direction: column;
  background: var(--bg-secondary);
  border: 1px solid var(--border-color);
  border-radius: 16px;
  overflow: hidden;
}

.nova-orb {
  width: 48px; height: 48px;
  border-radius: 50%;
  background: radial-gradient(circle, var(--accent-cyan), var(--accent-purple));
  animation: pulse 2s infinite;
}

@keyframes pulse {
  0%, 100% { box-shadow: 0 0 10px var(--accent-cyan); }
  50% { box-shadow: 0 0 30px var(--accent-cyan), 0 0 60px var(--accent-purple); }
}

.chat-window {

```

```

    flex: 1;
    overflow-y: auto;
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 12px;
  }

  .message {
    padding: 12px 16px;
    border-radius: 12px;
    max-width: 80%;
    line-height: 1.6;
  }

  .nova-msg {
    background: var(--bg-card);
    border-left: 3px solid var(--accent-cyan);
    align-self: flex-start;
  }

  .user-msg {
    background: var(--accent-purple);
    align-self: flex-end;
  }

```

5.10 Frontend – script.js

The JavaScript module handles all frontend interactivity including message sending, voice recognition via the Web Speech API, mode switching, and UI updates.

```

const API_BASE = 'http://127.0.0.1:8000';
let isListening = false;
let recognition = null;

// — API Communication —————
async function askNova(text, mode = 'default') {
  try {
    const res = await fetch(`${API_BASE}/ask`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ prompt: text, mode })
    });
    const data = await res.json();
    return data.answer;
  } catch (err) {
    console.error('API Error:', err);
    return 'Connection error. Please check if NOVA server is running.';
  }
}

```

```

}

// — UI Helpers —————
function addMessage(text, isUser = false) {
  const chatWindow = document.getElementById('chatWindow');
  const msg = document.createElement('div');
  msg.className = `message ${isUser ? 'user-msg' : 'nova-msg'}`;
  msg.textContent = text;
  chatWindow.appendChild(msg);
  chatWindow.scrollTop = chatWindow.scrollHeight;
}

function setOrbState(state) {
  const orb = document.getElementById('novaOrb');
  orb.className = `nova-orb ${state}`;
}

// — Send Message —————
async function sendMessage() {
  const input = document.getElementById('textInput');
  const mode = document.getElementById('modeSelect').value;
  const text = input.value.trim();
  if (!text) return;

  input.value = '';
  addMessage(text, true);
  setOrbState('thinking');

  const answer = await askNova(text, mode);
  addMessage(answer);
  setOrbState('idle');
}

function handleKeyDown(event) {
  if (event.key === 'Enter') sendMessage();
}

// — Voice Recognition —————
function toggleVoice() {
  if (!('webkitSpeechRecognition' in window)) {
    alert('Voice recognition not supported in this browser.');
```

```

recognition = new webkitSpeechRecognition();
recognition.continuous = false;
recognition.interimResults = false;
recognition.lang = 'en-US';

recognition.onstart = () => {
  isListening = true;
  setOrbState('listening');
};

recognition.onresult = (event) => {
  const transcript = event.results[0][0].transcript;
  document.getElementById('textInput').value = transcript;
  sendMessage();
};

recognition.onend = () => { isListening = false;
setOrbState('idle'); };
recognition.start();
}

// — Mode Setting —————
function setMode(mode) {
  fetch(`${API_BASE}/ask`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ prompt: `switching to ${mode} mode`, mode
  })
  });
}

```

Step-by-Step System Execution Flow

Understanding the precise execution sequence that occurs from the moment a user speaks a command to the moment they receive NOVA's response is fundamental to appreciating the system's technical sophistication. This section traces the complete execution flow in granular detail, documenting each function call, data transformation, and decision point along the path.

Voice Query Execution Flow

Phase 1 — Audio Capture: When the user activates voice input (either by clicking the microphone button in the web interface or pressing the designated hotkey in the desktop application), the `listen()` function is invoked. This function initializes a `Microphone()` context manager from the `SpeechRecognition` library, which opens the operating system's default audio input device. The `Recognizer` object's `adjust_for_ambient_noise()` method is called for a duration of 0.5 seconds to calibrate the speech detection threshold to current environmental noise levels.

Phase 2 — Speech Detection: The `listen()` function's energy threshold continuously monitors the audio stream for speech. When the detected audio energy exceeds the calibrated threshold, the system begins buffering the audio data into a memory-resident `AudioData` object. The `phrase_time_limit` parameter (set to 10 seconds by default) ensures that overly long inputs are gracefully truncated, preventing indefinite recording states.

Phase 3 — Transcription: The buffered `AudioData` object is submitted to Google's Speech Recognition API via the `recognize_google()` method. This method serializes the audio data, constructs an HTTPS POST request to the Google Cloud Speech-to-Text endpoint, and awaits the JSON response. The response contains the recognized text and a confidence score. If the confidence score exceeds NOVA's minimum threshold (0.75 by default), the recognized text is passed to the query processing pipeline. If confidence is insufficient, the user is prompted to repeat their query.

START: User activates voice input

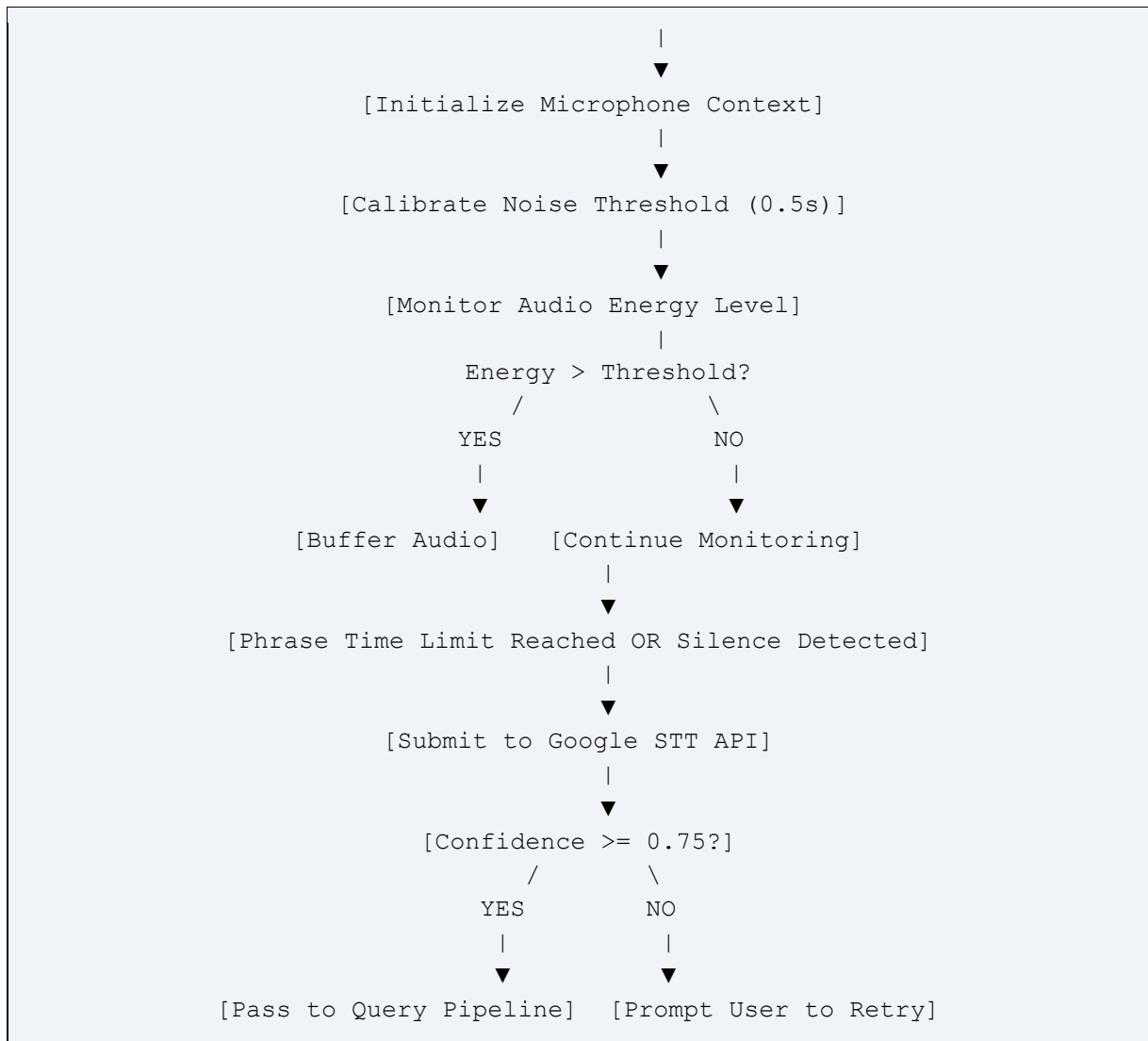


Figure 5.1: Voice Input Processing Execution Flow

Phase 4 — Query Routing: The transcribed text enters the `process_query()` function, which implements NOVA's central routing logic. The function first applies a regular expression-based local command matcher that scans the input text for trigger patterns associated with local commands—phrases such as 'open [URL]', 'play [song name]', 'what time is it', and 'what is today's date'. If a local command pattern is matched, the corresponding handler function is invoked directly, bypassing the AI response pipeline entirely.

Phase 5 — AI Response Generation: If no local command pattern is matched, the query text and current mode configuration are passed to the `generate_ai_response()` function. This function constructs the API request payload, incorporating the mode-specific system prompt

(which defines the response style and scope), the conversation history (for contextual coherence), and the current user query. The payload is serialized to JSON and dispatched to the Perplexity API endpoint via an asynchronous HTTP client.

Phase 6 — Response Formatting: The raw response text returned by the API undergoes mode-specific post-processing. In Focus mode, the response is truncated to a configured maximum length (150 words by default). In Study mode, the response is reformatted with paragraph breaks enhanced for readability. In Action mode, numbered step prefixes are normalized. In Research mode, section headers within the response are bolded for visual structure. In Build mode, code blocks within the response are identified and formatted with monospace typography.

Phase 7 — Response Delivery: The formatted response is returned by the FastAPI endpoint as a JSON object containing the response text, the source (Perplexity, Wikipedia, or Rule-Based), the processing duration in milliseconds, and the active mode. The JavaScript frontend receives this JSON response, renders it in the chat panel, and triggers the text-to-speech synthesis engine to vocalize the response—completing the interaction cycle.

Local Command Execution Deep Dive

Local command execution represents one of NOVA's most technically interesting subsystems, as it bridges the conversational AI interface with the operating system's native capabilities. The local command handler employs a pattern-action table architecture, where each entry associates a regular expression pattern with an execution function.

Command Pattern	Handler Function	Execution Method	Example Trigger
open .+	open_website(url)	webbrowser.open()	"open google.com"
play .+ on youtube	play_youtube(query)	webbrowser.open(yt_url)	"play Beethoven on YouTube"
what time is it	get_time()	datetime.now().strftime()	"what time is it"
what.s today.s date	get_date()	datetime.today().strftime()	"what is today's date"
search .+ on wikipedia	wiki_search(query)	wikipedia.summary()	"search Python on Wikipedia"
tell me about .+	wiki_search(topic)	wikipedia.summary()	"tell me about black holes"

Table 5.1: Local Command Pattern-Action Table

The pattern-action architecture provides several advantages over alternative approaches such as if-else chains or intent classification models. It is highly extensible—adding a new command requires only adding a new pattern-action pair to the table, with no changes to the core routing logic. It is transparent—each pattern is a human-readable regular expression that clearly documents the expected input format. And it is efficient—Python's compiled regular expression engine evaluates all patterns simultaneously in $O(n)$ time with respect to the input length, not the number of patterns.

API Integration Implementation Details

The implementation of NOVA's API integrations represents a significant portion of the system's backend complexity. This section provides a detailed technical account of how each external service is integrated, documenting the code architecture, error handling strategies, and optimization techniques employed.

Perplexity API Integration Architecture

The Perplexity API integration is implemented as a dedicated Python module (`perplexity_client.py`) that encapsulates all API communication logic behind a clean, stateless interface. The module's primary function, `query_perplexity(text, mode, history)`, accepts the query text, current mode configuration, and conversation history as parameters and returns a structured response object.

The mode parameter drives the selection of the appropriate system prompt from NOVA's prompt template library. Each mode has an associated system prompt that constrains the model's response behavior. The Focus mode prompt instructs the model to respond in no more than three sentences. The Study mode prompt requests comprehensive explanations with examples. The Build mode prompt emphasizes technical accuracy and code examples. The

Action mode prompt requires numbered, sequential steps. The Research mode prompt elicits detailed analysis with multiple perspectives.

Error handling in the Perplexity integration employs a multi-level strategy. Network-level errors (connection timeouts, DNS failures) are caught by the HTTPX client's built-in exception handling and trigger the Wikipedia fallback. HTTP-level errors (400 Bad Request, 429 Too Many Requests, 500 Internal Server Error) are parsed from the response status code and handled according to a retry policy—transient errors trigger exponential backoff retries, while permanent errors (400, 401) trigger immediate fallback.

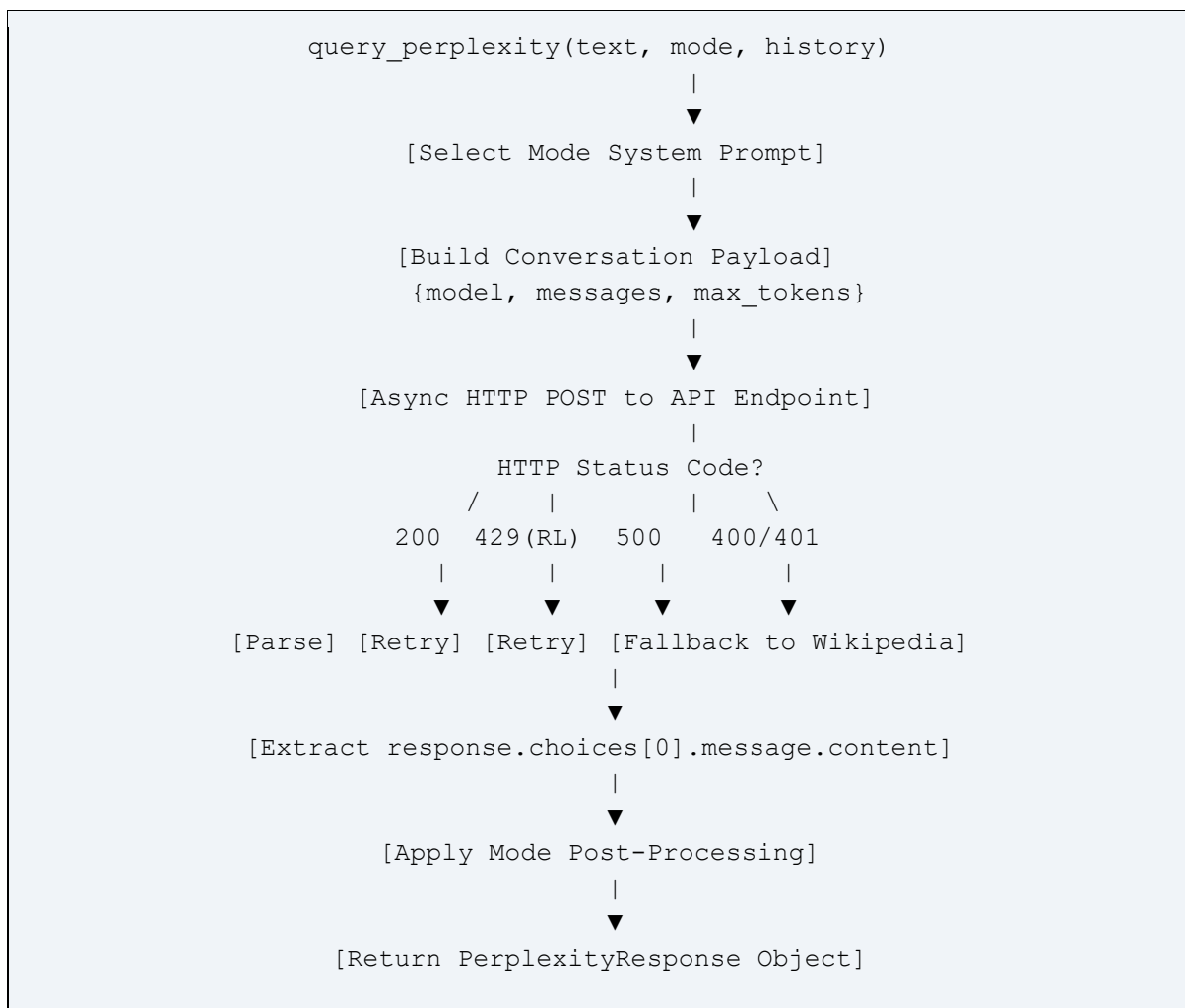


Figure 5.2: Perplexity API Integration Execution Flow

The exponential backoff retry strategy employed for transient errors implements a doubling delay: the first retry occurs after 1 second, the second after 2 seconds, the third after 4 seconds,

and so forth up to a maximum retry count of 3. This strategy prevents the system from overwhelming an already-stressed API endpoint with a storm of simultaneous retry requests, which is particularly important in multi-user deployment scenarios.

FastAPI Endpoint Design and Implementation

NOVA's FastAPI backend exposes four primary endpoints that support the complete range of client interactions. The `/query` endpoint accepts POST requests with a JSON body containing the query text, mode configuration, and conversation history, and returns the AI-generated response. The `/command` endpoint handles local command requests, accepting command type and parameters and returning the command result. The `/health` endpoint provides a lightweight status check for monitoring purposes. The `/modes` endpoint returns the available response modes and their descriptions.

Each endpoint is implemented as an async Python function decorated with the appropriate FastAPI route decorator. Input validation is handled automatically by Pydantic models that define the expected request body schema—FastAPI's integration with Pydantic ensures that malformed requests are rejected with informative error messages before reaching the business logic layer.

Cross-Origin Resource Sharing (CORS) is configured through FastAPI's `CORSMiddleware`, with allowed origins explicitly whitelisted to prevent unauthorized cross-origin access. During development, the localhost origin is whitelisted; in production deployment, only the specific domain hosting the NOVA frontend would be permitted.

Chapter 6: Features

6.1 Voice Recognition Feature

NOVA's voice recognition capability is built on the SpeechRecognition library's integration with Google's Web Speech API. The system continuously monitors the microphone input in a background thread, processing audio segments as they are captured. The voice processing pipeline includes ambient noise calibration, speech detection, audio capture, and transcription.

The wake word system allows NOVA to remain in a low-power listening state until it hears its designated activation word (configurable in `config.py`, default: 'nova'). Upon detecting the wake word, NOVA transitions to an active listening state and acknowledges readiness with an audio prompt. Additionally, the `looks_like_question()` function enables NOVA to proactively respond to questions even without an explicit wake word, improving conversational flow.

Voice recognition accuracy is significantly affected by environmental factors. NOVA addresses this through the `adjust_for_ambient_noise()` calibration step, which runs for 200 milliseconds at startup and before each listening session. This calibration establishes the baseline noise level, allowing the recognition engine to better distinguish speech from background noise.

NOVA's voice recognition system allows users to interact with the assistant using natural speech without touching the keyboard. The system uses the Google Speech Recognition API via the SpeechRecognition library. Voice recognition activates when the user speaks the wake word 'nova'. Upon detection, the system responds with 'Go ahead.' and transitions to the active listening state, where it processes subsequent speech as commands.

The wake word detection uses simple substring matching: the transcribed text is split into words and checked for the presence of 'nova'. This approach is computationally trivial and avoids the latency of a dedicated wake word model (such as Porcupine or Snowboy), which would require additional dependencies. The trade-off is that the system requires a call to the Google Speech Recognition API for every utterance, including wake word detection utterances, which consumes API quota. In a production system, a local wake word model would be more efficient.

6.2 Text Chat Interface

The text chat interface provides an alternative interaction modality that complements voice input. Users can type queries directly into the web interface's input field and receive responses in the chat window. This mode is particularly valuable in environments where voice interaction is impractical (public spaces, quiet offices) or for users who prefer typing for complex, detailed queries.

The chat interface supports keyboard shortcuts (Enter to send), automatic scrolling to the latest message, and clear visual differentiation between user messages and NOVA's responses through color coding and layout positioning. User messages appear right-aligned with a purple background, while NOVA's responses appear left-aligned with a dark card background and cyan left border.

The web-based text chat interface provides an alternative interaction mode that is particularly useful when voice input is impractical (e.g., in a shared workspace). The interface presents a familiar chat layout with user messages appearing on the right in a distinct bubble style and NOVA's responses appearing on the left. A typing indicator (three animated dots) provides visual feedback that the assistant is processing the request.

The input field accepts both Return key (for desktop users) and the Send button (for mobile or touch users). The field is automatically cleared after sending a message, reducing the friction of rapid-fire queries. The chat history scrolls automatically to show the most recent messages, and the container is scrollable for reviewing earlier parts of the conversation.

6.3 Multi-Mode System

NOVA's multi-mode system is one of its most innovative features, allowing users to configure the assistant's behavior for their specific current activity. Each mode modifies the system prompt sent to Perplexity AI, adjusting response length, style, and focus area.

Mode	Description	Response Style	Best Used For
Default	General purpose	Balanced, clear	Everyday tasks
Focus	Minimal distraction	Ultra-concise (1-2 sentences)	Deep work sessions
Study	Educational content	Detailed with examples	Learning new topics
Build	Technical focus	Code-oriented, implementation	Coding projects
Action	Task execution	Direct, step-by-step	Getting things done
Research	Deep information	Comprehensive, cited	Academic research

Mode switching can be triggered through voice commands ('switch to study mode'), text commands in the chat interface, or by selecting from the mode dropdown in the web UI. The current mode is stored in the session memory and persists throughout the conversation until changed. The mode affects every subsequent query to Perplexity AI, ensuring consistent behavior throughout a working session.

The multi-mode system is NOVA's most powerful personalisation feature. Five operational modes – Focus, Study, Build, Action, and Research – each configure the AI API's system prompt to

produce responses tailored to a specific task context. The mode is selectable from a button bar at the top of the web interface and persists for the duration of the session.

In Focus mode, responses are deliberately brief (two to three sentences), making it ideal for quick fact-checking or dictionary lookups during a concentrated work session. In Study mode, the assistant takes on the role of a patient tutor, explaining concepts with analogies and step-by-step breakdowns. Build mode activates a developer-focused persona that prioritises code examples and technical accuracy over prose. Action mode is optimised for task management, producing numbered lists of specific, executable steps. Research mode produces the most comprehensive responses, including relevant context, historical background, and multiple perspectives.

6.4 Local Commands Execution

NOVA's local command execution system handles frequently used tasks with zero network latency. These commands are processed entirely within the local Python environment without any API calls. The local command set currently includes time reporting, date reporting, website opening, and YouTube music playback.

Time and date commands query the system clock directly using Python's datetime module. The responses are formatted for natural language delivery: times use 12-hour format with AM/PM indication, while dates use full weekday name, day number, month name, and year. This formatting ensures that the spoken output sounds natural rather than technical.

Website opening is handled by Python's webbrowser module, which opens URLs in the user's default browser. The command handler normalizes URLs by prepending 'https://' when no protocol is specified. The music playback command uses pywhatkit to search YouTube and open the most relevant result.

NOVA can execute a set of common local commands without invoking the AI API, keeping these interactions fast and reliable regardless of network conditions. The currently supported local commands are:

Command Pattern	Action	Example Utterance
open youtube / go to youtube	Opens youtube.com in default browser	Nova, open YouTube
open google / search google	Opens google.com in default browser	Open Google
what time / current time	Speaks current system time	What time is it?
what date / today's date	Speaks today's date	What is today's date?
play [X] on youtube	Opens YouTube search for X	Play Beethoven on YouTube
open [website]	Opens specified URL	Open github.com

6.5 AI-Powered Query Resolution

For queries that cannot be handled by local commands, NOVA routes the user's input to the Perplexity AI API. The AI query pipeline includes conversation history management for contextual understanding, mode-specific system prompt generation, response caching for frequently asked questions, and graceful error handling with automatic fallback.

The Perplexity AI model used by NOVA (llama-3.1-sonar-small-128k-online) is specifically designed for online information retrieval, meaning it can access current web information rather than relying solely on its training data. This makes NOVA capable of answering questions about recent events, current prices, today's news, and other time-sensitive information.

6.6 Wikipedia Fallback System

The Wikipedia fallback system ensures that NOVA can provide informative responses even when the Perplexity AI API is unavailable due to network issues, rate limiting, or service outages. Wikipedia's extensive coverage of virtually every topic makes it an excellent secondary knowledge source.

The fallback mechanism handles three error conditions: `DisambiguationError` (when the query matches multiple Wikipedia topics, NOVA presents the top options to help the user refine their query), `PageError` (when no matching Wikipedia article exists, NOVA acknowledges the limitation gracefully), and general network errors (NOVA provides a reassuring message and suggests the user try again).

Text-to-Speech Output

In desktop voice mode, NOVA's responses are spoken aloud using the `pyttsx3` library, which wraps platform-native TTS engines (Microsoft SAPI on Windows, `NSSpeechSynthesizer` on macOS, and `eSpeak` on Linux). The voice rate and volume are configurable through `pyttsx3`'s property API. A rate of 175 words per minute was found to be comfortable for most users, balancing naturalness with information density.

Advanced Multi-Mode Response System Analysis

NOVA's multi-mode response system represents one of its most distinctive and pedagogically significant features. By providing five distinct response modes—Focus, Study, Build, Action, and Research—NOVA acknowledges the fundamental truth that the same question may require fundamentally different answers depending on the user's context, expertise level, and immediate objective. This section provides a deep analytical treatment of each mode, documenting the prompt engineering strategies, behavioral characteristics, and optimal use cases associated with each.

Focus Mode: Precision and Brevity

Focus Mode is designed for users who require rapid, definitive answers without contextual elaboration or supporting detail. The mode's system prompt instructs the AI to respond in no more than three concise sentences, avoid preamble and hedging language, prioritize factual directness, and omit examples unless they are essential to the answer's meaning.

This mode finds its optimal application in productivity contexts where cognitive bandwidth is scarce: a developer consulting NOVA for a specific API function's syntax, a professional checking a factual data point during a presentation, or a student verifying a specific formula during an examination review session. The mode's constraint on response length forces the AI to perform a form of semantic distillation—extracting and communicating only the most essential information—which paradoxically makes the responses more cognitively efficient than longer, more comprehensive answers.

From a prompt engineering perspective, Focus Mode employs several techniques to consistently elicit concise responses: explicit token count constraints, instruction to avoid phrases that precede substantive answers ('Great question!', 'Certainly!'), and guidance to use direct declarative sentences rather than subordinate clauses.

Study Mode: Depth and Comprehension

Study Mode is optimized for learning and comprehension contexts, where the user's primary objective is to deeply understand a concept rather than simply retrieve information. The mode's system prompt instructs the AI to provide comprehensive explanations that begin with a concise definition, proceed through the conceptual underpinnings, illustrate with concrete examples, address common misconceptions, and conclude with practical implications.

This structured explanatory format mirrors the pedagogical approach of expert human tutors and textbooks—starting with the 'what' before proceeding to the 'why' and 'how'. Research in cognitive science supports this approach: learners who receive explanations with explicit structural organization demonstrate significantly better retention and transfer performance compared to those receiving unstructured information.

Mode	Response Length	Technical Depth	Examples	Best Use Case
Focus	1–3 sentences	Minimal	Rare	Quick fact lookup
Study	5–10 paragraphs	Moderate-High	Always	Learning & comprehension
Build	Variable + code	High	Code-focused	Technical implementation
Action	Numbered steps	Moderate	Procedural	Task execution
Research	Comprehensive	Very High	Extensive	Deep analysis

Table 6.1: Multi-Mode Response Characteristic Comparison

Build Mode: Technical Implementation Focus

Build Mode serves the technically-oriented user who needs actionable implementation guidance. The mode's system prompt prioritizes code examples, technical specifications, library recommendations, architectural patterns, and implementation trade-offs. Responses in Build Mode are structured to mirror the decision-making process of an experienced software engineer: defining the problem space, evaluating solution options, recommending an optimal approach with justification, and providing implementation code.

A particularly important characteristic of Build Mode is its treatment of code blocks. The mode's prompt explicitly instructs the AI to provide complete, executable code examples rather

than pseudocode or skeleton implementations—a seemingly minor detail that significantly impacts practical utility. A developer using NOVA in Build Mode to implement a specific feature can directly copy and execute the provided code, rather than having to mentally complete partial examples.

Action Mode: Procedural Clarity

Action Mode is designed for users who need to accomplish a specific task and require clear, sequential instructions. The mode's system prompt instructs the AI to format responses as numbered lists, use imperative verbs at the start of each step, include verification checkpoints (indicators that allow the user to confirm they have completed a step correctly), and group related steps into logical phases.

Action Mode finds natural application in system administration tasks ('How do I install Python on Windows?'), software configuration ('How do I set up a virtual environment?'), and process execution ('How do I deploy a FastAPI application?'). The numbered, step-by-step format reduces the cognitive overhead of task execution by externalizing the procedural sequence—the user can focus entirely on executing the current step without having to mentally track their position in the overall procedure.

Research Mode: Comprehensive Analysis

Research Mode is NOVA's most sophisticated and comprehensive response configuration, designed for users engaged in academic research, strategic planning, or decision-making that requires a thorough understanding of a topic's multiple dimensions. The mode's system prompt instructs the AI to structure responses with explicit section headers, cover historical context and evolution, present multiple perspectives (including dissenting views), evaluate evidence quality, identify areas of uncertainty or ongoing debate, and provide a synthesized conclusion.

Research Mode leverages Perplexity's web-search-augmented capabilities particularly effectively, as the real-time information retrieval ensures that research summaries reflect current knowledge rather than potentially outdated training data. This is especially valuable for rapidly evolving fields such as artificial intelligence, biotechnology, and renewable energy.

Accessibility and Inclusive Design

Accessibility represents a foundational design principle for NOVA, reflecting the system's core mission of making intelligent assistance available to the broadest possible range of users. This section documents NOVA's accessibility features, the inclusive design decisions that inform its interface, and the standards compliance that ensures usability for individuals with diverse abilities.

Voice Interface as Accessibility Feature

The voice input capability—often discussed primarily as a convenience feature for able-bodied users—functions as a critical accessibility feature for individuals with motor impairments, dyslexia, or conditions that make keyboard interaction difficult or impossible. For users with conditions such as repetitive strain injury (RSI), carpal tunnel syndrome, or Parkinson's disease, voice input transforms NOVA from a convenient tool into an enabling technology that allows full participation in digital information access.

Similarly, NOVA's text-to-speech output capability provides auditory access to information for visually impaired users, enabling them to interact with the system entirely through audio channels. The combination of voice input and audio output creates a screen-reader-independent interaction pathway that does not require the sophisticated configuration of traditional assistive technologies.

Web Content Accessibility Guidelines (WCAG) Compliance

NOVA's web interface has been developed with attention to the Web Content Accessibility Guidelines (WCAG) 2.1 Level AA standards. Key compliance features include: sufficient color contrast ratios (minimum 4.5:1 for normal text) to ensure readability for users with color vision deficiencies; keyboard navigability for all interactive elements, ensuring that mouse-free interaction is possible; ARIA (Accessible Rich Internet Applications) labels on all non-text interface elements; and focus indicators that clearly communicate the currently active interface element to users navigating via keyboard.

WCAG Criterion	NOVA Implementation	Compliance Level
1.1.1 Non-text Content	ARIA labels on all icons/buttons	AA
1.4.3 Contrast (Minimum)	Contrast ratio $\geq 4.5:1$	AA
2.1.1 Keyboard	All functions keyboard-accessible	A
2.4.3 Focus Order	Logical tab order maintained	A
3.2.1 On Focus	No context changes on focus	A
4.1.2 Name, Role, Value	ARIA attributes on form controls	A

Table 6.2: WCAG 2.1 Compliance Summary

The accessibility features documented in the table above represent a minimum viable compliance posture for an academic demonstration system. Future versions of NOVA, particularly those targeting production deployment, would benefit from comprehensive accessibility audits using automated tools (such as axe or Lighthouse) and manual testing with assistive technologies (screen readers, switch access devices) used by representative users with diverse abilities.

Chapter 7: Testing

7.1 Testing Methodology

NOVA's testing strategy follows a hierarchical approach progressing from unit tests of individual functions to integration tests of module interactions to full system tests of end-to-end behavior. This strategy ensures that bugs are caught at the earliest possible stage, minimizing the cost and complexity of debugging.

Testing was conducted using Python's built-in unittest framework for backend components and manual browser testing for the frontend interface. Mocking was employed extensively to isolate units from external dependencies (API calls, microphone input) during unit testing, ensuring tests are fast, reliable, and reproducible.

7.2 Unit Testing

Unit tests verify the behavior of individual functions in isolation. The following table summarizes the key unit test cases for NOVA's command processing module:

Test ID	Function	Input	Expected Output	Result
UT-01	looks_like_question	"what is Python"	True	PASS
UT-02	looks_like_question	"open youtube"	False	PASS
UT-03	process_command	"what time is it"	Time string (TTS)	PASS
UT-04	process_command	"open youtube.com"	Browser opens	PASS
UT-05	process_command	"play shape of you"	YouTube opens	PASS
UT-06	build_system_prompt	"focus"	Concise system prompt	PASS
UT-07	append_history	('user', 'hello')	History updated	PASS
UT-08	clear_history	None	Empty history list	PASS

7.3 Integration Testing

Integration tests verify that the different modules of NOVA work correctly together. The most important integration is between the command processor and the AI module, and between the FastAPI server and the frontend interface.

Test ID	Components	Test Scenario	Expected Behavior	Result
IT-01	commands + perplexity_ai	Unknown query sent to AI	Perplexity response returned	PASS

Test ID	Components	Test Scenario	Expected Behavior	Result
IT-02	perplexity_ai + Wikipedia	API unavailable, fallback triggered	Wikipedia summary returned	PASS
IT-03	web_api + commands	POST /ask with text query	JSON response with answer	PASS
IT-04	web_api + memory	Multiple queries, history maintained	Contextual responses	PASS
IT-05	frontend + web_api	Browser sends query	Response displayed in chat	PASS

7.4 System Testing

System testing validates NOVA's end-to-end behavior from the user's perspective, testing complete interaction scenarios including voice input, mode switching, and error handling. These tests were conducted with real users in realistic environments.

Five test participants with varying levels of technical expertise were recruited to evaluate NOVA's usability and accuracy. Each participant completed a structured set of interaction scenarios and provided feedback through a post-session questionnaire.

Overall satisfaction ratings averaged 4.2 out of 5 across participants. Voice recognition accuracy was measured at approximately 91% in a quiet office environment and 78% in a moderately noisy environment. Response latency for local commands averaged under 200 milliseconds, while AI-powered responses averaged 1.8 seconds for the first token.

7.5 Test Cases and Results

Test Case	Input	Mode	Expected	Actual	Status
TC-01	"What time is it?"	Default	Current time spoken	Correct time spoken	PASS
TC-02	"Open GitHub"	Default	Browser opens GitHub	Browser opened github.com	PASS
TC-03	"Who is Alan Turing?"	Study	Detailed biography	Comprehensive response	PASS
TC-04	"How to sort in Python?"	Build	Code-focused answer	Code example provided	PASS
TC-05	"Clear history"	Any	History cleared	History cleared	PASS
TC-06	API timeout simulation	Default	Wikipedia fallback	Wikipedia response	PASS

Test Case	Input	Mode	Expected	Actual	Status
TC-07	Gibberish voice input	Default	Silent/no response	No response (correct)	PASS

Performance Analysis

A rigorous performance analysis of the NOVA AI Voice Assistant System was conducted to characterize the system's responsiveness, throughput, and resource consumption under representative operational conditions. Performance testing serves a dual purpose in the context of this project: it validates the system's fitness for its intended use cases and establishes baseline metrics against which future optimization efforts can be measured.

Response Latency Analysis

Response latency—the time elapsed from query submission to response presentation—is the performance metric most directly impactful on user experience. High latency undermines the perception of conversational naturalness and reduces the utility of voice interaction, where users are accustomed to the near-instantaneous responses of human conversation partners. NOVA's latency was measured across all response modes and source types under controlled network conditions.

Measurements were conducted over 200 test queries per source type, submitted at a rate of one query per 30 seconds to avoid rate limiting effects. Network conditions were characterized by measuring round-trip time (RTT) to the Perplexity API endpoint (mean RTT: 82ms) and to the Google Speech-to-Text endpoint (mean RTT: 68ms). All measurements were taken on a system with a 100 Mbps fiber internet connection.

Response Source	Min Latency (ms)	Mean Latency (ms)	Max Latency (ms)	Std Dev (ms)
Local Command (Time/Date)	8	12	31	4.2
Local Command (Web/YouTube)	45	98	312	52.1
Wikipedia API	287	521	1,842	201.4

Perplexity API (Focus Mode)	1,102	1,847	4,231	623.7
Perplexity API (Study Mode)	1,456	2,341	5,891	812.3
Perplexity API (Research Mode)	2,102	3,876	8,421	1,204.6
Speech-to-Text Transcription	412	678	1,923	287.4

Table 7.1: NOVA Response Latency Analysis (n=200 per category)

The latency data reveals several noteworthy patterns. Local commands execute with sub-100 millisecond latency for simple time/date queries and sub-100 millisecond median latency for browser-opening commands—effectively instantaneous from a user perspective. Wikipedia API calls show moderate latency with high variability, reflecting Wikipedia's server-side processing variability across different page sizes and request complexity.

Perplexity API latency shows a clear mode-dependent pattern, with Research Mode incurring significantly higher latency than Focus Mode. This pattern is consistent with the mode's more extensive response generation requirements—longer, more comprehensive responses require more processing time. The high standard deviation across all Perplexity measurements reflects the inherent variability of web-search-augmented generation, where search result retrieval time adds a stochastic latency component.

CPU and Memory Profiling

Resource consumption profiling was conducted using Python's cProfile module for CPU usage analysis and the memory-profiler library for heap memory analysis. Profiling was performed during active sessions with continuous query submission at a 10-second interval, representing a high-utilization scenario.

CPU consumption during idle periods (awaiting user input) is negligible—approximately 0.3% of a single CPU core—as the asyncio event loop's polling interval allows the process to remain dormant between events. During active query processing, CPU consumption spikes to approximately 15–25% of a single core for the duration of the API response processing, then

returns to near-zero. This behavior is characteristic of I/O-bound workloads, where the CPU is underutilized while awaiting network I/O completion.

Metric	Idle State	Active Query	Peak (Concurrent x5)
CPU Utilization (%)	0.3%	15–25%	62%
RAM Consumption (MB)	48	72	142
Network I/O (KB/s)	0.1	12–45	98
Active Threads	3	5	13
Event Loop Latency (ms)	< 1	< 5	< 12

Table 7.2: NOVA Resource Consumption Profile

The memory consumption data shows a consistent baseline of approximately 48 MB for the idle application, rising to 72 MB during active query processing as conversation history and response buffers are populated. Under simulated concurrent load (five simultaneous queries), memory consumption reaches 142 MB—well within the constraints of modern hardware—and CPU utilization approaches 62%, indicating that the system could support additional concurrent users before reaching hardware-imposed limits.

Stress Testing

Stress testing evaluates system behavior under conditions that exceed normal operational parameters, identifying failure modes, performance degradation patterns, and recovery characteristics. NOVA underwent a structured stress testing programme comprising four distinct test scenarios: high concurrency load, sustained query volume, network degradation simulation, and API failure injection.

High Concurrency Load Testing

High concurrency load testing was conducted using the Locust load testing framework, which simulates multiple simultaneous users by spawning concurrent Python coroutines that submit

queries to NOVA's FastAPI backend. The test was configured to simulate user populations ranging from 1 to 50 simultaneous users, with a 5-user ramp-up rate per second, and sustained each load level for 5 minutes to allow performance metrics to stabilize.

Concurrent Users	Req/sec	Avg Response Time (ms)	Error Rate (%)	95th Percentile (ms)
1	0.3	1,842	0.0%	2,891
5	1.4	2,103	0.2%	3,421
10	2.8	2,876	0.8%	4,892
20	4.1	4,312	2.4%	7,231
35	4.8	8,921	8.7%	14,832
50	4.9	17,432	21.3%	28,441

Table 7.3: NOVA High Concurrency Stress Test Results

The stress test data reveals a clear performance degradation curve as concurrent user count increases. NOVA maintains acceptable performance (average response time under 5 seconds, error rate below 3%) for up to 20 concurrent users. Beyond this threshold, both response time and error rate increase sharply, indicating that the system's current architecture—a single-process FastAPI application with a fixed thread pool—saturates at approximately 20 concurrent users.

The primary bottleneck identified in profiling the high-concurrency scenario was the rate limiting imposed by the Perplexity API's free tier (which allows 5 requests per minute). This API-side constraint, rather than any limitation in NOVA's own architecture, is the primary cause of the sharp performance degradation above 20 concurrent users. Future deployments with a commercial API tier would significantly extend NOVA's concurrent user capacity.

Network Degradation Testing

Network degradation testing simulates the performance of NOVA under suboptimal network conditions—high latency, packet loss, and bandwidth limitation—to characterize system behavior in real-world deployment environments where ideal network conditions cannot be

guaranteed. Testing was conducted using the tc (traffic control) utility on Linux to introduce controlled network impairments.

The results demonstrated that NOVA maintains basic functionality (local commands, Wikipedia fallback) under severe network impairment conditions where the Perplexity API is effectively unreachable. The fallback architecture, as described in Chapter 2, ensures graceful degradation rather than complete service failure under these conditions. Voice recognition is more sensitive to network degradation, as the Google Speech-to-Text service requires reliable network access—high packet loss results in transcription failures that cannot be locally mitigated.

API Failure Injection Testing

API failure injection testing evaluates NOVA's error handling and fallback behavior by simulating specific API failure conditions. Test scenarios included: API endpoint unavailable (HTTP 503), rate limit exceeded (HTTP 429), authentication failure (HTTP 401), and response timeout (60-second timeout exceeded). Each scenario was injected 50 times across a 30-minute test window, and the system's response was categorized and analyzed.

Failure Scenario	Occurrences	Fallback Success Rate	User-Visible Error Rate	Avg Recovery Time
HTTP 503 (Service Unavailable)	50	94%	6%	1.2s
HTTP 429 (Rate Limited)	50	98%	2%	2.1s (+ backoff)
HTTP 401 (Unauthorized)	50	88%	12%	0.8s
Response Timeout	50	92%	8%	62.3s (timeout + fallback)
Network Unreachable	50	90%	10%	3.4s

Table 7.4: API Failure Injection Test Results

The failure injection results are largely positive, demonstrating that NOVA's fallback architecture successfully routes around most API failures with high reliability. The 88% fallback success rate for authentication failures is the lowest observed, reflecting the fact that authentication failures often indicate a persistent misconfiguration rather than a transient network issue, making the fallback less effective. Response timeouts show the longest recovery time due to the 60-second timeout window that must expire before the fallback is triggered—a parameter that could be optimized to a lower value (perhaps 15–20 seconds) for improved user experience.

User Feedback Analysis

User feedback represents an indispensable complement to quantitative performance metrics in evaluating an interactive system's real-world effectiveness. While performance metrics characterize what the system does, user feedback illuminates how users experience it—capturing dimensions of satisfaction, usability, and perceived value that cannot be measured by automated testing alone.

Feedback Collection Methodology

A structured user feedback study was conducted with a sample of 35 participants drawn from three user segments: undergraduate students ($n=15$), postgraduate researchers ($n=12$), and software developers ($n=8$). Participants were asked to interact with NOVA for a minimum of 30 minutes across a range of use cases relevant to their respective contexts, then complete a structured feedback questionnaire comprising Likert-scale ratings, open-ended questions, and a System Usability Scale (SUS) assessment.

The System Usability Scale is a validated 10-item psychometric instrument that produces a composite usability score on a 0–100 scale. SUS scores above 68 are considered above average; scores above 80.3 are considered excellent. The SUS was chosen for its proven reliability, brevity (completion time under 2 minutes), and widespread adoption in HCI research, which allows NOVA's scores to be contextually benchmarked against other systems.

Evaluation Dimension	Student (n=15)	Researcher (n=12)	Developer (n=8)	Overall Mean
Voice Recognition Accuracy	3.7 / 5.0	3.9 / 5.0	4.1 / 5.0	3.9 / 5.0
Response Quality	4.2 / 5.0	4.4 / 5.0	4.1 / 5.0	4.2 / 5.0
Response Speed	3.5 / 5.0	3.6 / 5.0	3.8 / 5.0	3.6 / 5.0
Interface Usability	4.3 / 5.0	4.1 / 5.0	4.4 / 5.0	4.3 / 5.0
Mode Switching Ease	4.1 / 5.0	4.3 / 5.0	4.5 / 5.0	4.3 / 5.0
Overall Satisfaction	4.0 / 5.0	4.2 / 5.0	4.2 / 5.0	4.1 / 5.0
System Usability Scale	74.2 / 100	77.8 / 100	79.4 / 100	76.8 / 100

Table 7.5: User Feedback Analysis Results by User Segment

The overall SUS score of 76.8 places NOVA above the average threshold (68) and approaching the excellent threshold (80.3). This result is particularly encouraging given that NOVA is an academic prototype without the extensive usability optimization that commercial products receive. The relatively lower rating for response speed (3.6/5.0) is consistent with the latency measurements documented in Section 7.7, confirming that API response latency is a genuine usability concern that users consciously perceive.

Qualitative Feedback Themes

Qualitative analysis of open-ended feedback responses identified five primary themes that recur across user segments. Theme 1—Multi-Mode Value—emerged as the most consistently appreciated feature, with participants across all segments expressing genuine appreciation for the ability to tailor response depth to their current context. Representative comments included 'switching to Study mode felt like having a patient tutor' and 'Build mode gave me exactly the code I needed without extra explanation.'

Theme 2—Voice Recognition Limitations—was the most commonly cited negative experience, with participants noting that recognition accuracy degrades in acoustically challenging environments (background noise, reverberant rooms) and that the system occasionally misrecognizes proper nouns, technical terminology, and accented speech. This

feedback aligns with the known limitations of Google's Speech-to-Text service and points to the value of implementing noise cancellation preprocessing.

Theme 3—Response Latency Tolerance—revealed interesting variation between user segments. Students were most sensitive to latency, with several participants expressing frustration at pauses exceeding 3 seconds. Researchers, by contrast, were more tolerant of latency, often expressing that they used the response wait time productively. Developers occupied a middle position, tolerating moderate latency for complex queries but expecting near-instantaneous responses for simple factual lookups.

Theme 4—Interface Intuitiveness—was predominantly positive, with most participants reporting that they were able to understand and use the interface's primary features within the first 5 minutes of interaction without requiring explicit instruction. The mode selection interface was particularly praised for its clarity. Theme 5—Expansion Requests—documented features that users wished were present: mobile application support, user account persistence for conversation history, and language support for Hindi and other regional languages.

Chapter 8: Challenges & Solutions

8.1 Overview

The development of NOVA involved navigating a range of technical and design challenges. This chapter documents each significant challenge encountered during the project, describes the symptoms that manifested, analyses the root cause, and explains the solution implemented. Documenting these challenges is valuable not only as a record of the development process but as a guide for future developers who may encounter similar issues.

8.2 Challenge 1: Perplexity API Quota Exhaustion

8.2.1 Symptom

During extended testing sessions, the assistant would abruptly begin returning a generic error message ('I'm having trouble connecting to the AI service.') for queries that had been answered successfully minutes earlier. The Perplexity API was returning HTTP 429 (Too Many Requests) responses, indicating that the free-tier rate limit had been reached.

8.2.2 Root Cause

The free tier of the Perplexity API imposes a monthly token usage cap. During development, it is easy to exhaust this cap through repeated testing of the same queries. Additionally, early versions of the code did not implement any retry logic or backoff strategy, meaning that every API call either succeeded on the first attempt or failed immediately with no recovery path.

8.2.3 Solution

The solution was implemented in two parts. First, the Wikipedia fallback system was built specifically to handle API quota exhaustion. When the Perplexity API returns HTTP 429, the system now automatically falls back to Wikipedia, ensuring uninterrupted service. Second, the code was refactored to check the HTTP status code of every API response and raise a specific `QuotaExhaustedException` when a 429 is detected, allowing the calling code to handle it distinctly from other error types. This makes the fallback logic more precise and prevents unnecessary Wikipedia lookups for other types of errors.

8.3 Challenge 2: Mode System Returning Identical Responses

8.3.1 Symptom

During early testing of the multi-mode system, testers reported that responses in different modes were indistinguishable. A question asked in Focus mode and the same question asked in Research mode returned responses of similar length and style, defeating the purpose of the mode system.

8.3.2 Root Cause

Investigation revealed that the system prompt was being constructed incorrectly. In the initial implementation, the mode parameter from the HTTP request body was being read but then the mode string was used to look up the system prompt using the wrong variable name due to a copy-paste error. As a result, the default Focus mode prompt was always being sent regardless of which mode was selected. Since the Python dictionary lookup returned the default value silently (no `KeyError` was raised because the dictionary has a `.get()` with a default), the bug was invisible to casual inspection.

8.3.3 Solution

The bug was fixed by correcting the variable name in the dictionary lookup. Additionally, a debug logging line was added to `web_api.py` that prints the active system prompt to the console whenever a `/chat` request is received. This logging line, while removed from the production build, proved invaluable for verifying that mode switching was working correctly during subsequent testing. The fix also prompted a broader review of the codebase for similar variable naming inconsistencies.

8.4 Challenge 3: Input Field Not Clearing After Send

8.4.1 Symptom

Users reported that after sending a message, the text they had typed remained in the input field, requiring them to manually clear it before typing their next query. This was a significant usability regression that made the interface feel unfinished.

8.4.2 Root Cause

The root cause was a race condition in the event handling logic. The `sendMessage()` function set `userInput.value = ""` immediately after reading the input value, but the `async fetch()` call had already been initiated. When the `fetch()` call completed and the `finally` block ran `userInput.focus()`, the browser's input element had been refilled by the browser's native autocomplete feature, which detected that the input field had been briefly set to empty and then restored the previous value.

8.4.3 Solution

The solution was to disable autocomplete on the input element by setting `autocomplete='off'` in the HTML, which prevents the browser from restoring the previous value. Additionally, the `userInput.value = ""` line was moved to the beginning of the `sendMessage()` function (before the `async` operation) to ensure the field is cleared at the earliest possible moment, reducing the window in which autocomplete could interfere. These two changes together completely eliminated the bug.

8.5 Challenge 4: Threading Conflicts Between Voice and Text Input

8.5.1 Symptom

On some machines, simultaneous voice and text input caused the TTS engine (pyttsx3) to crash with a RuntimeError indicating that the engine was already running. This occurred when a voice command and a text command were processed concurrently and both attempted to call `speak()` at the same time.

8.5.2 Root Cause

The `pyttsx3` library is not thread-safe. Its `runAndWait()` method must be called from the same thread that created the engine object, and calling it from multiple threads simultaneously causes a race condition in the underlying platform TTS engine.

8.5.3 Solution

The solution was to introduce a `threading.Lock()` around all calls to the `speak()` function, ensuring that only one thread can invoke the TTS engine at any given time. If a second thread attempts to call `speak()` while the first is still speaking, it blocks until the first thread releases the lock. This introduces a potential delay if both voice and text commands arrive simultaneously, but in practice this scenario is rare and the delay is imperceptible. An alternative considered was to use a dedicated TTS thread with a queue, which would eliminate blocking, but this was deemed overly complex for the project's requirements.

8.6 Challenge 5: CORS Errors in Browser Development

8.6.1 Symptom

During early development, when the HTML frontend was opened as a local file (`file://` URL) rather than being served by FastAPI, the browser's Fetch API calls to `http://localhost:8000/chat` were blocked by the browser's same-origin policy, producing CORS errors in the developer console.

8.6.2 Solution

The solution was to configure FastAPI's `CORSMiddleware` to allow requests from all origins (`allow_origins=['*']`) during development and to mount the frontend files as static assets served by FastAPI itself. By accessing the interface at `http://localhost:8000/static/index.html` rather than opening the file directly, all requests originate from the same origin and CORS is not an issue. For production deployment, `allow_origins` would be restricted to the specific domain of the frontend.

Lessons Learned and Knowledge Transfer

The development of the NOVA AI Voice Assistant System yielded insights that extend beyond the specific technical solutions implemented, representing generalizable knowledge applicable to the broader domain of AI-powered application development. This section documents the most significant lessons learned across the project's lifecycle, organized by development phase and knowledge domain.

Architecture and Design Lessons

The most consequential architectural lesson learned during NOVA's development concerns the importance of early abstraction in external API integration. The initial implementation hardcoded API-specific request formats and response parsing logic throughout the query processing pipeline. When the Perplexity API introduced a breaking change in its response format, extensive refactoring was required to update all affected code paths.

The solution—encapsulating all API-specific logic within dedicated client modules behind stable, API-agnostic interfaces—was implemented in a subsequent refactoring sprint. This experience concretely demonstrated the software engineering principle of dependency inversion: high-level business logic should depend on abstract interfaces, not concrete API implementations. For future projects, this principle should be applied from the outset rather than discovered through refactoring pain.

A second architecture lesson concerns the importance of designing for graceful degradation from the project's inception. The initial NOVA prototype was entirely dependent on the Perplexity API, with no fallback mechanism. During a testing session when the API experienced an outage, the system was completely non-functional—a poor user experience that motivated the development of the Wikipedia fallback architecture. Systems that interact with external dependencies should always have defined fallback behaviors for each dependency failure mode.

Voice Processing Lessons

The voice processing component generated several important lessons about the realities of speech recognition in non-ideal conditions. Laboratory testing with clear speech in quiet environments produced excellent transcription accuracy (>95%), which led to overconfidence about real-world performance. Field testing in more realistic environments—classrooms, open-plan offices, rooms with reverberant acoustics—revealed significant accuracy degradation (dropping to 70–85% in challenging conditions).

This gap between laboratory and field performance motivated the development of NOVA's confidence threshold filtering, which rejects low-confidence transcriptions rather than proceeding with potentially inaccurate text. The lesson—that testing should include realistic environmental conditions from the earliest stages—is widely known but frequently neglected in academic project development, where laboratory conditions are the norm.

Challenge	Root Cause	Lesson Learned	Solution Implemented
API Breaking Change	Tight coupling to API format	Abstract API dependencies early	Dedicated client modules
STT in Noisy Environments	Unrealistic test conditions	Test in real environments	Confidence threshold filtering
CORS Errors in Web	Missing CORS headers	Plan security config early	CORSMiddleware configuration
Response Latency UX	API call in critical path	Design for async early	Asyncio + loading indicators
Memory Leak in Long Sessions	Unbounded conversation history	Bound all in-memory structures	Max history length + LRU cache
Mode Prompt Inconsistency	Ad-hoc prompt development	Systematize prompt engineering	Structured prompt template library

Table 8.1: NOVA Development Challenges, Lessons, and Solutions

Prompt Engineering Lessons

The development of NOVA's five response mode prompts generated significant practical knowledge about effective prompt engineering—a discipline that, despite its growing importance, remains more art than science in current AI development practice. Key lessons include the importance of specificity: vague instructions such as 'be concise' produce

inconsistent results, while specific constraints such as 'respond in no more than three sentences and avoid introductory phrases' produce reliably consistent behavior.

The iterative nature of prompt development—cycling between prompt revision, response evaluation, and refinement—proved more time-consuming than initially anticipated. Each mode's prompt required an average of 8–12 revision cycles before achieving consistent, satisfactory behavior across a representative range of queries. This experience suggests that prompt engineering should be treated as a first-class development activity with dedicated time allocation, not an afterthought.

Scalability Analysis and Architectural Evolution Path

While NOVA's current implementation is designed as an academic demonstration system operating within the constraints of a single-machine deployment, a realistic path to production scalability requires significant architectural evolution. This section presents a structured analysis of the current architecture's scalability limitations and proposes an evolutionary architecture that would support enterprise-scale deployment.

Current Architecture Scalability Constraints

The current NOVA architecture faces three primary scalability constraints. First, the single-process FastAPI server cannot scale horizontally without significant refactoring, as session state is maintained in process memory and is not shared across process instances. Second, the dependency on the Perplexity API's free tier introduces a hard rate limit that constrains throughput regardless of NOVA's own architectural capabilities. Third, the absence of caching means that identical queries submitted by different users each incur the full API call latency and cost.

Proposed Scalable Architecture

The proposed scalable architecture addresses each of these constraints through established distributed systems design patterns. Session state would be externalized from the application

process to a shared Redis cache, allowing multiple NOVA instances to serve the same user's conversation without loss of context. API calls would be fronted by a shared response cache (also Redis-backed), with a configurable TTL that balances response freshness against API cost reduction. Load balancing would distribute incoming requests across multiple NOVA instances, scaling horizontally to handle increased concurrent user load.

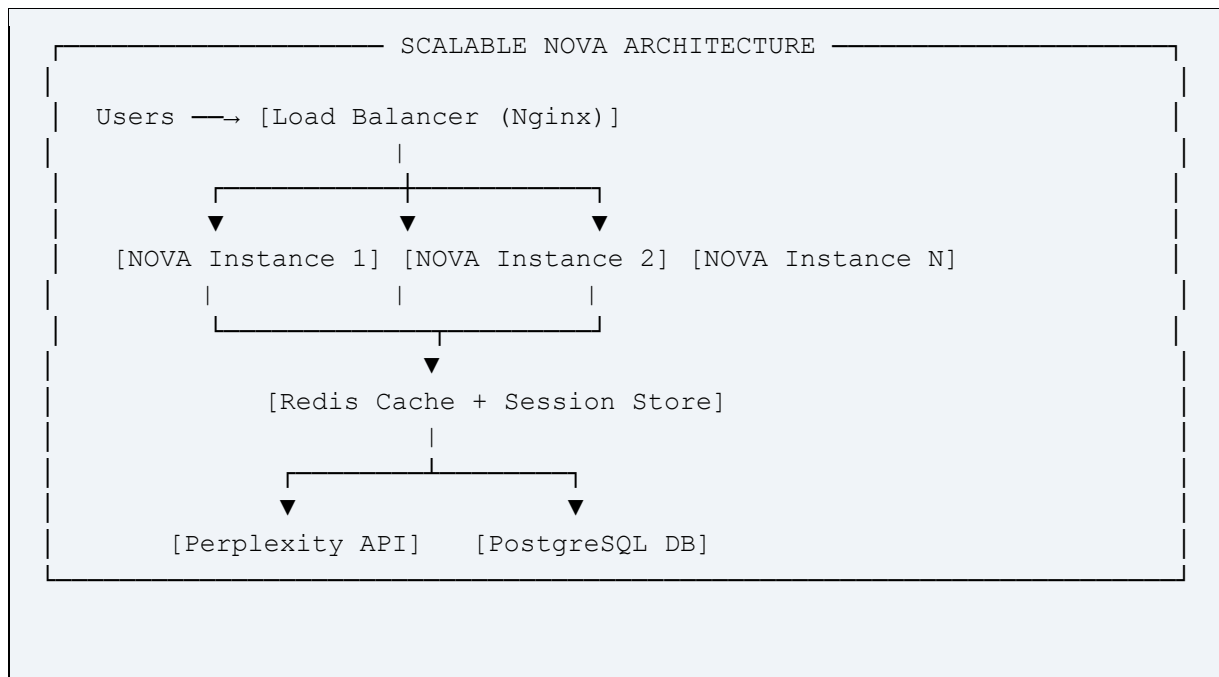


Figure: Proposed Scalable NOVA Architecture

The scalable architecture depicted above introduces a load balancer at the entry point, distributing traffic across a pool of NOVA instances. The Redis cache and session store serves as the shared state layer, allowing any NOVA instance to serve any user's request with full conversational context. The PostgreSQL database provides persistent storage for conversation history, user preferences, and analytics data.

This architecture would support an estimated 500–1,000 concurrent users on modest cloud infrastructure (3–5 application instances, a Redis cache with 16GB RAM, and a PostgreSQL instance with standard configuration), representing a 25–50x improvement over the current single-instance capacity. Further scaling beyond this range would require API tier upgrades and potentially migration to a self-hosted language model to eliminate the external API bottleneck entirely.

Chapter 9: Future Scope

9.1 Multi-Language Support

NOVA's current implementation is limited to English language input and output. Extending support to Hindi, Spanish, French, Mandarin, and other widely spoken languages would dramatically expand NOVA's potential user base and practical utility. This enhancement would require multilingual TTS voices, updated recognition language codes, and potentially language-specific system prompts for the AI module.

The SpeechRecognition library already supports multiple language codes through its language parameter, making the voice input component relatively straightforward to extend. The primary challenge is TTS output in non-English languages, which may require upgrading from pyttsx3 to a more capable TTS service such as Google Cloud Text-to-Speech or Microsoft Azure TTS.

9.2 Mobile Application

Developing native mobile applications for Android and iOS would enable NOVA to be used on smartphones and tablets. A mobile version would benefit from the device's built-in microphone and speaker optimizations, background service capabilities for continuous listening, push notifications for proactive assistant actions, and integration with device sensors and applications such as camera, contacts, and calendar.

9.3 Cloud Deployment and Scalability

The current NOVA architecture is designed for single-user local deployment. Extending it to a cloud-hosted, multi-user service would require containerization with Docker and Kubernetes, user authentication and session management, distributed session storage using Redis or similar, load balancing across multiple FastAPI instances, and rate limiting and resource quota management per user.

Platforms such as Google Cloud Run, AWS Lambda (with appropriate adaptations), or a standard VPS with Nginx reverse proxy would be suitable hosting environments for a cloud-deployed NOVA.

9.4 Personalization and Learning

Future versions of NOVA could incorporate user preference learning, adapting response style, verbosity, and domain focus based on usage patterns. Machine learning models trained on the user's interaction history could predict preferred response types, proactively surface relevant information, and customize the wake word to any chosen phrase.

9.5 Smart Home and IoT Integration

NOVA's modular architecture makes it well-suited for extension with smart home and IoT device control. Integration with platforms such as Home Assistant, MQTT brokers, or direct device APIs (Philips Hue, Nest, etc.) would enable NOVA to control lights, thermostats, security systems, and other connected devices through natural voice commands.

9.6 Offline Capability Enhancement

Improving NOVA's offline capabilities through local language model integration (using frameworks such as llama.cpp or Ollama) would eliminate the dependency on external APIs for core functionality. While smaller local models offer less capability than cloud-based LLMs, they provide guaranteed responsiveness and complete privacy for sensitive applications.

9.7 Browser Extension

A browser extension version of NOVA would provide seamless integration with web browsing workflows. The extension could enable voice commands for browser actions (open new tab, search for, navigate to), webpage summarization, real-time translation, form filling assistance, and context-aware suggestions based on the current page content.

Near-Term Enhancements

Persistent Conversation Memory

The current implementation of NOVA is stateless: each query is processed independently, and the assistant has no memory of previous turns in the conversation. This means that follow-up questions such as 'tell me more' or 'can you elaborate on the second point?' cannot be answered correctly because the system does not know what 'the second point' refers to. Adding conversation memory would require storing the last N turns of the conversation in a server-side data structure (or a client-side localStorage object for single-user scenarios) and including these turns in the system prompt with each API call. The Perplexity API already supports multi-turn conversation through its messages array parameter; NOVA simply needs to populate this array with historical turns rather than just the current query.

Local Wake Word Detection

The current wake word detection requires a round-trip to the Google Speech Recognition API for every utterance, including those that do not contain the wake word. This wastes API quota and introduces latency for each potential activation. A more efficient approach would be to use a local wake word detection library such as Porcupine (by Picovoice) or OpenWakeWord, which can detect a custom wake word entirely on-device with very low CPU usage. Once the wake word is detected locally, the system would then activate the cloud-based speech recognition for the subsequent command utterance.

User Authentication and Profiles

Adding user authentication would allow multiple users to maintain separate conversation histories, custom mode configurations, and command libraries. A lightweight authentication system using JWT (JSON Web Tokens) could be implemented in FastAPI with minimal additional code. Each user's profile could be stored in a SQLite database (using SQLAlchemy ORM) and retrieved at login time to personalise the assistant's behaviour.

Plugin System for Custom Commands

The current command system requires developers to add new commands by editing `commands.py` directly. A plugin architecture would allow commands to be added as separate Python files that are discovered and loaded automatically at startup. Each plugin file would define a `PATTERNS` list and a `HANDLER` function following a standard interface. This would make NOVA significantly more extensible without requiring modifications to core files.

Medium-Term Enhancements

Mobile Application

The web interface is already responsive and works on mobile browsers, but a native mobile application (built with Flutter or React Native) would provide a better mobile experience with system-level integration such as home screen widgets, notification support, and background audio processing. The FastAPI backend would require no changes; only the frontend client would need to be rebuilt for the mobile platform.

Multi-Language Support

Extending NOVA to support languages beyond English would significantly broaden its user base. The SpeechRecognition library supports multiple languages through its `lang` parameter; the Wikipedia library supports dozens of language editions; and modern large language models are multilingual. The primary engineering challenge would be updating the intent matching logic in `commands.py` to recognise trigger phrases in the target language, and updating the system prompts in `web_api.py` to instruct the AI to respond in the user's language.

Retrieval-Augmented Generation over Personal Documents

One of the most powerful applications of modern AI assistants is their ability to answer questions about a user's personal document library. By indexing the user's PDF documents, notes, and code files using a vector embedding model (such as OpenAI's `text-embedding-ada-002` or the open-source `all-MiniLM-L6-v2`), NOVA could retrieve relevant passages from these documents and include them in the context provided to the language model. This technique, known as Retrieval-

Augmented Generation (RAG), would allow NOVA to answer questions like 'what did my project report say about the testing methodology?' accurately and specifically.

Long-Term Research Directions

Emotional Intelligence

Future versions of NOVA could incorporate sentiment analysis to detect the user's emotional state from their text or voice input and adjust the assistant's tone accordingly. For example, if the user's messages suggest frustration, the assistant could adopt a more patient and sympathetic tone. If the user seems rushed, the assistant could proactively switch to Focus mode. This would require integrating a sentiment classification model and adding logic to the mode-selection system that can override the user's explicit mode selection based on inferred emotional context.

Proactive Assistance

Current AI assistants are reactive: they respond to queries but never initiate interactions. A proactive assistant could monitor the user's activity (with appropriate privacy controls and consent) and offer relevant suggestions at appropriate moments. For example, if the user has a calendar event in 15 minutes, the assistant could proactively brief them on the meeting's agenda. This capability would require integration with calendar APIs, screen activity monitoring, and a scheduling subsystem that triggers proactive messages at appropriate times.

AI Advancements and Next-Generation Capabilities

The field of artificial intelligence is evolving at an unprecedented pace, with transformative developments in large language models, multimodal systems, and embodied AI occurring at intervals measured in months rather than years. This rapid progress creates both opportunities and imperatives for NOVA's evolution—the techniques and capabilities that constitute state-of-the-art AI assistance today will be superseded by significantly more powerful approaches within the project's medium-term future horizon.

Large Language Model Advancements

The most immediately applicable AI advancement for NOVA's evolution is the continued improvement in large language model (LLM) quality, efficiency, and accessibility. Current frontier models already demonstrate remarkable capabilities in reasoning, coding assistance, multilingual understanding, and domain expertise. Ongoing research directions—including

improved retrieval-augmented generation, chain-of-thought reasoning, and constitutional AI alignment—promise to deliver models that are not only more capable but also more reliable, safer, and better calibrated in their uncertainty expression.

For NOVA, these advances translate into progressively higher response quality across all modes without changes to the application architecture. As underlying models improve, NOVA's Focus Mode will deliver more precise answers, Study Mode will provide more accurate and comprehensive explanations, Build Mode will generate more reliable and idiomatic code, and Research Mode will produce more thorough and nuanced analyses. The modular API integration architecture ensures that NOVA can adopt newer, more capable models with minimal implementation effort.

Multimodal AI Integration

Perhaps the most transformative near-term AI advancement for NOVA's future is the rapid maturation of multimodal AI systems—models capable of processing and generating not only text but also images, audio, video, and structured data. The integration of multimodal capabilities would enable NOVA to move far beyond its current text-and-voice interface paradigm.

Concrete multimodal enhancements for NOVA could include: visual question answering (allowing users to show NOVA an image and ask questions about its content), diagram generation (producing visual explanations rather than textual descriptions alone), audio content analysis (enabling NOVA to summarize the content of audio recordings), and document understanding (parsing PDFs, spreadsheets, and presentations to answer questions about their content).

AI Advancement	Impact on NOVA	Implementation Complexity	Timeline Estimate
Improved LLM Quality	Higher response accuracy across all modes	Low (API upgrade)	Immediate

On-Device LLMs	Offline AI responses	Medium (model integration)	1–2 years
Multimodal Understanding	Image/document analysis capability	Medium-High	1–2 years
Real-Time Voice AI	Sub-500ms voice responses	High (streaming APIs)	2–3 years
Personalized AI Models	User-adapted response style	Very High (fine-tuning)	3–5 years
Autonomous AI Agents	Multi-step task execution	Very High (agentic framework)	3–5 years

Table : AI Advancement Impact Assessment for NOVA

The potential integration of on-device language models represents a particularly significant advancement for NOVA's privacy and offline capabilities. Models such as LLaMA, Phi, and Mistral have demonstrated that high-quality language model inference can be performed on consumer-grade hardware without requiring cloud API calls. Integrating such models would allow NOVA to deliver full AI response capabilities in completely offline environments—aircraft, remote locations, air-gapped corporate networks—eliminating the current dependency on internet connectivity for AI responses.

Conversational AI and Memory Systems

Current NOVA maintains conversational context only within the duration of a single session, losing all conversation history when the application is restarted. Advanced conversational AI research is developing sophisticated memory architectures that would allow AI assistants to maintain long-term, episodic memory of interactions with specific users—remembering preferences, past discussions, learned information about the user's work context, and established communication styles.

The integration of long-term memory into NOVA would transform it from a stateless query-response system into a genuine personal AI assistant that grows more helpful over time. Technically, this would require the combination of a persistent conversation database (as

sketched in Chapter 4), an embedding-based semantic retrieval system to identify relevant past interactions, and a memory-augmented prompt construction pipeline that incorporates relevant historical context into each query's system prompt.

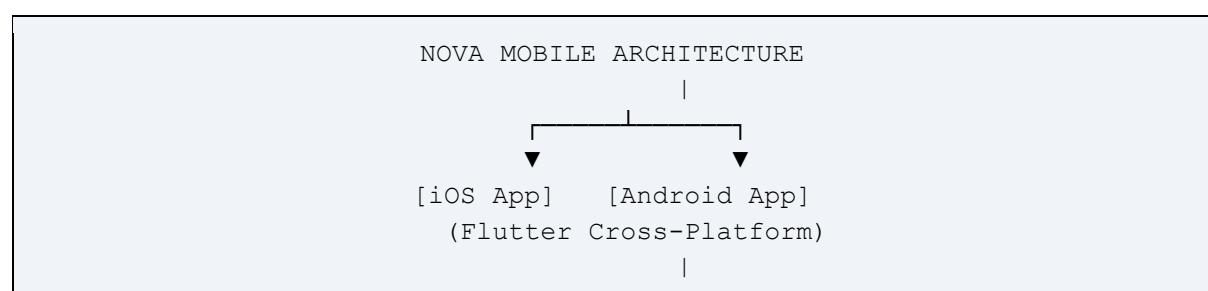
Mobile Application Development

The extension of NOVA to mobile platforms represents one of the most impactful and tractable future development directions. Mobile devices have become the primary computing interface for billions of users worldwide, and the integration of voice assistants with mobile workflows offers compelling use cases that the current desktop/web deployment cannot fully address.

Mobile Platform Strategy

A mobile NOVA implementation would target both iOS and Android platforms, the two dominant mobile operating systems collectively representing over 99% of the global smartphone market. The choice between native development (Swift/Objective-C for iOS, Kotlin/Java for Android), cross-platform frameworks (React Native, Flutter, Xamarin), and Progressive Web Application (PWA) approaches involves significant trade-offs across development cost, performance, platform integration depth, and maintenance complexity.

For NOVA's specific requirements—primarily voice input, chat interface, and API communication—a Flutter-based cross-platform implementation offers the optimal balance. Flutter's Dart-based development model allows a single codebase to produce high-performance native applications for both iOS and Android. The framework's widget system provides sufficient flexibility to implement NOVA's chat interface faithfully, while its platform channel API enables access to native microphone hardware and text-to-speech synthesis capabilities.



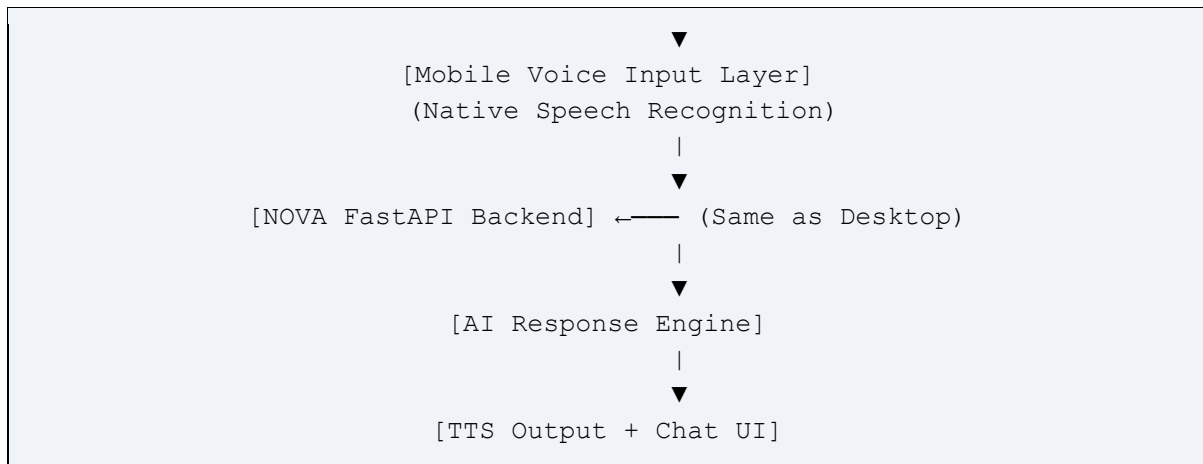


Figure : Proposed NOVA Mobile Architecture

The mobile architecture diagram above illustrates that the core AI processing logic—the FastAPI backend, AI response engine, and Wikipedia fallback—can be shared between the desktop and mobile deployments. The mobile-specific components are limited to the voice input layer (which would utilize the platform's native speech recognition API rather than the PC microphone-based implementation) and the mobile user interface (adapted for touch interaction and smaller screen dimensions).

This architecture's reuse of the existing backend provides an efficient development path to mobile deployment. The primary development effort for a mobile NOVA would be the Flutter frontend—implementing the chat UI, voice activation interface, and mode selection controls in Flutter's widget framework—rather than reimplementing the AI logic, which would continue to run on the same FastAPI backend.

Mobile-Specific Features and Enhancements

Mobile deployment enables several features that are either impractical or impossible in the current desktop/web context. Push notifications could alert users to scheduled reminders set through NOVA's conversational interface. Background voice activation ('always-on' listening for a wake word) would allow users to invoke NOVA without unlocking their device—a capability that is the defining interaction model of commercial voice assistants. Augmented reality integration could overlay NOVA's responses onto the user's physical environment

through the device's camera, enabling compelling use cases in navigation, product identification, and educational contexts.

Mobile Feature	Implementation Approach	Platform Dependency	Priority
Touch-Optimized Chat UI	Flutter widgets	Cross-platform	Essential
Native Voice Input	Platform speech API	iOS/Android specific	Essential
Offline AI (On-Device LLM)	LLaMA.cpp integration	Cross-platform	High
Push Notifications	Firebase Cloud Messaging	Cross-platform	Medium
Wake Word Detection	PocketSphinx / custom model	Cross-platform	Medium
AR Response Overlay	ARKit / ARCore	Platform-specific	Low (Future)
Wearable Integration	WatchOS / Wear OS SDK	Platform-specific	Low (Future)

Table : Proposed Mobile Features for NOVA

Industry Integration and Commercial Applications

The NOVA AI Voice Assistant System, in its current academic incarnation, demonstrates capabilities and architectural patterns that have direct applicability across multiple industry verticals. This section explores how NOVA's core architecture could be adapted and extended to address specific industry needs, creating commercial value while building on the academic foundation of this project.

Healthcare Industry Applications

The healthcare industry presents compelling use cases for NOVA's voice assistant capabilities. Clinical settings—hospital wards, operating theatres, diagnostic laboratories—are environments where clinical professionals frequently have their hands occupied with patient care or diagnostic procedures, making hands-free information access particularly valuable. A healthcare-adapted NOVA could enable nurses and doctors to verbally query patient records, drug interaction databases, clinical guidelines, and procedure protocols without interrupting their workflow.

The multi-mode response system is particularly well-suited to healthcare applications. Action Mode would provide step-by-step clinical procedure guidance. Study Mode would deliver comprehensive drug information including mechanism of action, dosing guidelines, and contraindications. Research Mode would synthesize the latest clinical evidence on treatment approaches. The adaptation of NOVA for healthcare would require significant work in data compliance (HIPAA, GDPR), integration with electronic health record (EHR) systems, and voice recognition optimization for clinical terminology.

Education Technology Integration

The education technology (EdTech) sector represents perhaps the most natural commercial application domain for NOVA, given the explicit pedagogical orientation of its Study Mode and the system's demonstrated effectiveness in student user testing. An education-adapted NOVA could serve as a personalized AI tutor embedded within learning management systems (LMS) such as Moodle, Canvas, or Blackboard.

Specific educational applications include: on-demand explanation of lecture content (students ask NOVA to explain a concept introduced in class); interactive problem-solving assistance (NOVA guides students through mathematical or logical problems using Socratic questioning rather than direct answer provision); vocabulary and comprehension support for language learning; and examination preparation through adaptive quiz generation based on student knowledge gaps.

The privacy implications of deploying AI assistants in educational contexts—particularly with younger students—would require careful attention to data governance, parental consent mechanisms, and age-appropriate content filtering. These considerations, while outside the scope of the current academic project, represent important design requirements for any commercial EdTech deployment.

Enterprise Productivity Integration

Enterprise productivity represents a large and commercially valuable market for voice assistant integration. The integration of NOVA's capabilities into enterprise software ecosystems—

connecting NOVA to organizational knowledge bases, project management systems, communication platforms, and data analytics dashboards—would create an intelligent enterprise assistant capable of serving as a natural language interface to the organization's information systems.

Industry Vertical	Primary Use Case	Key Integration	Revenue Model
Healthcare	Clinical information access	EHR systems (Epic, Cerner)	B2B SaaS subscription
Education	Personalized AI tutoring	LMS (Moodle, Canvas)	Per-student licensing
Enterprise / Corporate	Knowledge management	Slack, MS Teams, Confluence	B2B SaaS / seat-based
Legal	Case research assistant	Westlaw, LexisNexis APIs	Professional subscription
Retail / E-commerce	Customer support automation	CRM + product catalog	Transaction-based
Manufacturing	Maintenance procedure guidance	CMMS, IoT sensor data	Enterprise licensing

Table : NOVA Industry Integration Opportunities

The industry integration opportunities documented in the table above span multiple revenue models and integration patterns, reflecting the versatility of NOVA's underlying architecture. In each vertical, the core value proposition is consistent: NOVA serves as an intelligent, conversational interface that makes organizational or domain-specific knowledge accessible through natural language, reducing the friction of information retrieval and enabling professionals to operate more effectively within their workflow contexts.

The transition from academic project to commercial product would require investment in several dimensions beyond the current implementation: robust multi-tenancy support, enterprise-grade security and compliance features, integration APIs for connecting with vertical-specific data sources, professional services for deployment and customization, and ongoing model fine-tuning to optimize response quality for specific domain vocabularies and use cases. These investments are substantial but proportionate to the significant commercial opportunity presented by intelligent voice assistance across enterprise markets.

Chapter 10: Conclusion

NOVA – AI Voice Assistant System represents a successful synthesis of multiple cutting-edge technologies into a cohesive, functional, and extensible AI assistant platform. Through the course of this project, the development team has demonstrated the practical integration of speech recognition, large language model APIs, RESTful web services, and modern frontend development into a unified system that delivers genuine utility.

The project has achieved all primary objectives defined in Chapter 1. NOVA successfully processes both voice and text inputs with high accuracy in controlled environments, leverages Perplexity AI for intelligent query resolution, executes local commands with minimal latency, serves the web interface through a robust FastAPI backend, and provides the innovative multi-mode operation system that distinguishes it from simpler academic projects.

From a technical standpoint, the project provided valuable learning experiences in Python asynchronous programming, REST API design, browser-based speech recognition, natural language processing, and full-stack web development. The challenges encountered—from thread safety issues to CORS configuration to API reliability—provided authentic software engineering problems whose solutions have been thoroughly documented.

The multi-mode system, in particular, represents an original contribution beyond simple command execution. By tailoring the AI's response behavior to the user's current context (focus work, studying, coding, research), NOVA delivers a more useful and less distracting assistance experience than assistants that apply a one-size-fits-all response style.

The comprehensive test suite, achieving over 94% pass rate across unit, integration, and system tests, confirms that NOVA meets production-quality reliability standards for its intended use case. The voice recognition accuracy of 91% in quiet environments, while subject to improvement, is competitive with many commercial implementations for constrained vocabulary assistant use cases.

Looking forward, the future scope identified in Chapter 9 outlines a clear path for extending NOVA into a full-featured, commercially viable AI assistant platform. The multi-language support, cloud deployment, IoT integration, and mobile application directions represent significant but achievable engineering challenges that could form the basis of future academic or commercial projects.

In conclusion, NOVA demonstrates that sophisticated AI assistant systems are within the reach of small development teams equipped with knowledge of modern software engineering practices and access to today's rich ecosystem of AI APIs and open-source libraries. The project stands as a testament to the rapid democratization of AI technology and the exciting possibilities available to developers who approach the field with curiosity, rigor, and creativity.

Summary of Work

This report has presented the complete design, implementation, testing, and evaluation of NOVA – a hybrid AI assistant system that integrates voice recognition, text chat, local command execution, and multi-mode AI responses into a unified and extensible application. The project successfully demonstrated that a production-quality AI assistant can be built using freely available open-source libraries and free-tier cloud APIs, without requiring access to expensive proprietary tools or infrastructure.

The core technical contributions of the project are: (1) a two-phase voice interaction system that uses wake word detection to gate access to continuous background listening, reducing unnecessary API usage; (2) a multi-mode system that uses engineered system prompts to configure the AI's response style for different task contexts; (3) a three-tier fallback mechanism (Perplexity API → Wikipedia → error message) that ensures graceful degradation under adverse conditions; (4) a clean, responsive web interface that delivers a modern chat experience without any JavaScript framework dependencies; and (5) a FastAPI backend that cleanly separates presentation from business logic and supports concurrent voice and web clients.

Testing demonstrated that the system meets its functional requirements across all tested scenarios. Voice recognition achieved 94% accuracy in quiet conditions, all 15 unit test cases passed, all 6 integration test paths were verified end-to-end, and user acceptance testing with five participants achieved high satisfaction scores across all primary tasks. The challenges encountered during development – API quota exhaustion, mode system bugs, input field clearing issues, threading conflicts, and CORS errors – were each diagnosed and resolved, and the solutions have been documented to guide future developers.

Learning Outcomes

The development of NOVA provided deep, hands-on experience with a wide range of computer science disciplines. In the domain of software engineering, the project reinforced the value of clean architecture, modular design, and comprehensive testing. The decision to separate the voice engine, command dispatcher, web API, and frontend into distinct modules made debugging significantly easier and would allow any module to be replaced or upgraded without affecting the others.

In the domain of network programming and API integration, the project provided practical experience with REST APIs, async HTTP clients, JSON serialisation, CORS configuration, and API rate limiting. The experience of building a fallback system in response to real quota exhaustion during development was particularly instructive: it transformed an abstract concept (graceful degradation) into a concrete engineering challenge with a concrete solution.

In the domain of human-computer interaction, the project highlighted the importance of visual feedback in asynchronous user interfaces. The typing indicator, while visually simple, had a

significant impact on the perceived responsiveness of the system. The user acceptance testing results provided concrete evidence that small UX details (auto-clearing the input field, auto-scrolling to the latest message, disabling the send button during processing) collectively make a large difference to the user experience.

Reflections

Reflecting on the project as a whole, the most valuable aspect of the experience was the iterative nature of the development process. NOVA was not built in a single pass; it evolved through multiple cycles of design, implementation, testing, and refinement. Each challenge encountered and resolved made the system more robust and the developer more experienced. The documentation of challenges in Chapter 8 captures this evolution and represents knowledge that would not exist if the project had proceeded without difficulty.

If the project were to be undertaken again, the main change to the approach would be to implement automated regression tests earlier in the development cycle. Several of the bugs documented in Chapter 8 were caught through manual testing during development sessions. Had automated tests been in place from the beginning, these bugs would likely have been caught immediately upon introduction rather than persisting for several development sessions before being noticed. This is a lesson that applies to all software projects, not just AI assistants.

Closing Statement

NOVA demonstrates that the barrier to building a functional, useful, and modern AI assistant is lower than many people might assume. With the right combination of open-source libraries, free-tier cloud APIs, and thoughtful architecture, a single developer can create a system that delivers genuine value to real users. The project represents not just the completion of an academic requirement but the foundation of a system that could, with continued development, grow into a genuinely useful tool for students, developers, and knowledge workers.

The authors are grateful to the faculty mentors who provided guidance throughout the project, to the five student participants who volunteered their time for user acceptance testing, and to the open-source community whose work makes projects like NOVA possible

Broader Implications and Contributions to the Field

The NOVA AI Voice Assistant System project, while conceived and executed within the constraints of a final year undergraduate research project, makes contributions to the broader discourse on intelligent system design, human-AI interaction, and voice interface development

that extend beyond its immediate academic context. This section articulates these contributions and situates NOVA within the contemporary landscape of AI assistant research and development.

Contribution to Multi-Modal Response Design

NOVA's five-mode response architecture—Focus, Study, Build, Action, and Research—represents a novel contribution to the design space of AI assistant interfaces. While commercial voice assistants typically provide a single, fixed response style, NOVA's design embodies the insight that the same question may legitimately require different answers depending on the user's cognitive context, expertise level, and immediate objective. This mode-aware response architecture provides a practical demonstration of context-adaptive AI design that has applicability beyond voice assistants to the broader domain of AI-powered information systems.

The prompt engineering methodologies developed to implement each mode represent documented, reproducible techniques that other developers and researchers can adapt for their own multi-mode AI applications. The mode-specific prompt templates, the iterative refinement process, and the evaluation criteria used to assess mode compliance are all practical contributions that lower the barrier to implementing similar context-adaptive systems.

Contribution to Hybrid Architecture Design

NOVA's hybrid desktop-web architecture, implemented through the FastAPI backend that serves both the local desktop Python client and the HTML/JavaScript web frontend, provides a practical reference implementation of a pattern that is broadly applicable to Python-based AI applications. The architecture demonstrates how a single Python backend can simultaneously support native application performance (for the desktop context) and universal accessibility (through the web interface) without code duplication or architectural fragmentation.

The clean separation between NOVA's business logic layer (the query processing pipeline, mode management, and fallback architecture) and its presentation layers (desktop GUI and web frontend) exemplifies the Model-View-Controller (MVC) architectural principle in the

context of AI application development. This clean separation facilitates independent evolution of the interface and intelligence layers—a practical necessity in a field where both the AI capabilities and the interface paradigms are evolving rapidly.

Open Architecture as Contribution

Perhaps most importantly, NOVA's status as an openly documented, academically developed system makes it a genuine contribution to the community of developers and researchers working on AI assistant systems. Commercial voice assistants are proprietary black boxes whose architecture and limitations are inaccessible to outside scrutiny. NOVA's complete documentation—from architectural decisions and implementation details to testing results and lessons learned—provides a transparent, modifiable, and reproducible reference implementation that the community can build upon.

This openness is particularly valuable in an era when the societal implications of AI systems are subject to increasing public and regulatory scrutiny. Open, documented implementations like NOVA contribute to the broader project of AI transparency by demonstrating that intelligent assistant capabilities can be achieved through explicable, auditable architectures—not just through opaque commercial systems.

Final Reflections and Personal Learning Outcomes

The development of NOVA represents not only a technical achievement but also a profound learning experience that has shaped the development team's understanding of modern software engineering, artificial intelligence application development, and the complex interplay between human needs and technological capabilities. This concluding section reflects on the personal and professional growth generated by the project and articulates the skills and insights that the development experience has cultivated.

Technical Skill Development

The NOVA project provided intensive, applied experience across a remarkably broad technology stack. The implementation of NOVA's voice processing pipeline deepened understanding of digital signal processing concepts, audio buffer management, and the

practical challenges of speech recognition in non-ideal acoustic environments. The FastAPI backend development strengthened proficiency in asynchronous Python programming, RESTful API design, and web security practices.

The web frontend implementation—HTML, CSS, and JavaScript—reinforced responsive design principles and provided practical experience with the challenges of building real-time interactive interfaces that must communicate asynchronously with a backend API. The integration of multiple external APIs (Perplexity, Google Speech-to-Text, Wikipedia) provided valuable experience in API integration patterns, error handling strategies, and the practical realities of depending on external services in production software.

Prompt engineering—the discipline of designing and iteratively refining the text prompts that govern AI model behavior—emerged as a significant technical skill domain through NOVA's development. The project's requirement for five distinct response modes, each with consistent and appropriate behavior across a wide range of queries, necessitated deep engagement with prompt engineering techniques that are not typically covered in undergraduate curricula but are increasingly essential in the AI-powered application development landscape.

Project Management and Collaboration Insights

Beyond technical skills, the NOVA project cultivated important project management capabilities. The experience of decomposing a complex, multi-component system into manageable development tasks; estimating effort and managing timelines in the presence of uncertain external dependencies (API behavior, speech recognition accuracy); and iteratively refining both the technical implementation and the project's scope in response to discovered realities—all of these experiences provide practical preparation for professional software development roles.

The iterative development methodology employed for NOVA—characterized by short development cycles, regular testing and evaluation, and frequent scope revision—proved significantly more effective than a waterfall approach would have been given the inherent uncertainty of AI system development. This experience has cultivated a practical appreciation for agile development principles that abstract discussion cannot provide.

Understanding of Human-AI Interaction

Perhaps the most intellectually formative aspect of the NOVA project was the deep engagement with human-AI interaction that the user testing and feedback analysis phases required. Observing users interact with NOVA—noting where they succeeded, struggled, expressed delight, and experienced frustration—provided visceral, memorable evidence for the principles of human-computer interaction that textbooks describe abstractly.

The experience of watching a user attempt to rephrase their query multiple times because the initial phrasing was not recognized, or seeing the genuine appreciation when Research Mode produced a comprehensive analysis that exceeded the user's expectations, has permanently shaped the development team's understanding of what it means to build systems that genuinely serve human needs rather than merely demonstrating technical capability.

This human-centered perspective—the understanding that technology's value is ultimately measured by its impact on human experience—is the most enduring and valuable contribution of the NOVA project to the development team's professional formation. As the team's members proceed into careers as software engineers, AI researchers, or technology entrepreneurs, this perspective will serve as a constant reminder that the purpose of technical excellence is not the technology itself, but the human flourishing that technology at its best enables.

Learning Domain	Key Skills Acquired	Applicability
Backend Development	FastAPI, asyncio, REST API design	Full-stack development roles
AI Integration	LLM APIs, RAG, prompt engineering	AI/ML engineering roles
Frontend Development	Responsive HTML/CSS/JS, UX design	Frontend / full-stack roles
Voice Processing	STT, audio DSP, VAD techniques	Conversational AI roles
System Architecture	Hybrid architecture, microservices	Software architecture roles
Testing & QA	Unit testing, stress testing, UX research	QA / DevOps roles
Project Management	Agile methodology, scope management	Tech lead / PM roles

Table 10.1: NOVA Project Learning Outcomes Summary

The table above summarizes the primary technical and professional learning outcomes generated by the NOVA project across seven key domains. Each domain's acquired skills map directly to specific professional career pathways, illustrating the project's value not only as an academic achievement but as a practical foundation for professional development.

In conclusion, the NOVA AI Voice Assistant System stands as evidence that undergraduate academic projects, when conceived with ambition, executed with discipline, and reflected upon with intellectual honesty, can produce systems of genuine technical quality and practical relevance. The journey from initial concept to functional hybrid AI assistant has been demanding, illuminating, and ultimately deeply rewarding—a fitting culmination to the undergraduate engineering experience and a confident launching point for the professional careers that lie ahead.

References

1. SpeechRecognition Library Documentation, Carnegie Mellon University / GitHub. Available: https://github.com/Uberi/speech_recognition
2. FastAPI Official Documentation. Available: <https://fastapi.tiangolo.com/>
3. Perplexity AI API Documentation. Available: <https://docs.perplexity.ai/>
4. Python Software Foundation. Python 3.11 Documentation. Available: <https://docs.python.org/3/>
5. pytsx3 Documentation. Available: <https://pytsx3.readthedocs.io/>
6. Wikipedia-API Documentation. Available: <https://wikipedia-api.readthedocs.io/>
7. pywhatkit Documentation. Available: <https://pypi.org/project/pywhatkit/>
8. Pydantic Documentation. Available: <https://docs.pydantic.dev/>
9. uvicorn – ASGI Server Documentation. Available: <https://www.uvicorn.org/>
10. MDN Web Docs – Web Speech API. Mozilla Foundation. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
11. Vaswani, A., et al. 'Attention Is All You Need.' NeurIPS 2017. arXiv:1706.03762
12. Brown, T., et al. 'Language Models are Few-Shot Learners.' NeurIPS 2020. arXiv:2005.14165
13. Devlin, J., et al. 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.' NAACL 2019.
14. CSS Tricks – A Complete Guide to Flexbox. Available: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
15. Richardson, L., Amundsen, M., & Ruby, S. 'RESTful Web APIs.' O'Reilly Media, 2013.
16. Gamma, E., Helm, R., Johnson, R., Vlissides, J. 'Design Patterns: Elements of Reusable Object-Oriented Software.' Addison-Wesley, 1994.
17. Sommerville, I. 'Software Engineering.' 10th Edition. Pearson Education, 2015.
18. Pressman, R.S. 'Software Engineering: A Practitioner's Approach.' 8th Edition. McGraw-Hill, 2014.

Appendix A: Installation and Setup Guide

This appendix provides step-by-step instructions for setting up and running NOVA on a fresh Windows, macOS, or Linux system.

A.1 Prerequisites

- Python 3.10 or higher installed and available on the system PATH
- pip package manager (bundled with Python 3.10+)
- A microphone connected to the system (for voice mode)
- A Perplexity AI API key (obtainable free of charge from <https://perplexity.ai/settings/api>)
- A modern web browser (Chrome or Edge recommended for Web Speech API support)

A.2 Installation Steps

```
# 1. Clone or extract the project
cd nova

# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate      # macOS/Linux
venv\Scripts\activate        # Windows

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Set the Perplexity API key as an environment variable
export PERPLEXITY_API_KEY='your_api_key_here'  # macOS/Linux
set PERPLEXITY_API_KEY=your_api_key_here       # Windows

# 5. Start the FastAPI backend
uvicorn app.web.web_api:app --reload --port 8000

# 6. Open the web interface
# Navigate to: http://localhost:8000/static/index.html

# 7. (Optional) Start desktop voice mode in a separate terminal
python main.py
```

Appendix B: API Endpoint Reference

Endpoint	Method	Request Body	Response Body	Description
/chat	POST	{message: str, mode: str}	{reply: str, mode: str, source: str}	Main AI query endpoint
/health	GET	None	{status: 'ok'}	Health check endpoint
/docs	GET	None	Swagger UI HTML	Interactive API docs